

AI First Principles

12 周完整学习教程

从 Linear Regression 到 Mini GPT

Week 2 与邮件版 Week 3 原文保留

其余章节按关键知识重新审查与衔接

XeLaTeX / AMSMath 专业公式排版

目录索引

阅读说明	1
章节索引	1
Week 1 - 必要数学直觉：先学会读 AI 公式	2
Week 1 核心目标	2
本章怎样学习每个概念	2
1. AI 首先处理的是数字	2
2. Scalar：一个数字	3
3. Vector：一组有顺序的数字	3
为什么 Vector 不只是普通 List	3
4. Shape：数据的结构契约	4
Dimension、Axis 与 Shape 的区别	4
5. Weight：模型怎样看待不同输入	4
Weight 为什么必须存在	5
6. Dot Product：把多个输入合成一个分数	5
Dot Product 为什么要求长度一致	6
7. Bias：额外的可学习偏移	6
用门槛理解 Bias	6
8. Function：输入经过规则得到输出	6
Function 与 Model 的关系	7
9. Variable、Parameter 与 Hyperparameter	7
Variable	7
Parameter	7
Hyperparameter	7
10. Matrix：同时处理很多向量	7
11. Matrix Multiplication 的最重要规则	8
输出 Shape 为什么是 [m,p]	8
12. 求和符号 Σ	8
13. 平方、指数、概率：先建立最低限度直觉	9
平方	9
指数	9
概率	9
为什么平方适合表示误差	9
为什么指数适合生成概率	9
概率不等于确定事实	9
14. 读 AI 公式的固定顺序	9
15. Week 1 Python 小实验	10
16. Week 1 最应该理解的 6 件事	10
17. 理解测试	11
18. Week 1 $\rightarrow$ Week 2	11
Week 2 — Linear Regression：从 Prediction 到 Gradient Descent	12
1. Linear Regression 解决什么问题？	12
2. Model、Feature、Parameter	12
3. Prediction 是怎么产生的？	13
4. Error 与 Loss 的区别	13
5. MSE 什么时候用？	13
6. 为什么还需要 Gradient？	14
7. Derivative：先理解“坡度”	14
8. Gradient 是什么？	14
9. Gradient Descent	15
10. Learning Rate	15
11. 完整手算一次 w 更新	16
Step 1 — Prediction	16
Step 2 — Loss	16

Step 3 — Gradient	16
Step 4 — Update	17
12. Chain Rule 为什么重要?	17
13. Bias b 与 Partial Derivative	17
14. 同时更新 w 和 b	18
15. Second Derivative 二阶导数	18
16. Least Squares 与 Gradient Descent	19
17. Python 从零实现	19
18. 代码与数学——对应	20
19. Forward Pass 与 Backward Pass	21
20. Epoch 与 Batch	21
21. Linear Regression 与 Neural Network 的关系	21
22. 从 Linear Regression 到 GPT	21
23. Week 2 必须掌握的词	22
24. Week 2 最重要的 5 个公式	23
25. 最终 Mental Model	23
Week 2 补充深入理解: 把“公式结论”变成可运行的直觉	24
1. Loss 不是 Gradient	24
2. 一个 parameter 与多个 parameters	24
3. 为什么 Gradient 为负时参数会增加	25
4. Chain Rule 为什么是 Backprop 的桥	25
5. 为什么本周不需要二阶导数	25
6. 用有限变化检查导数方向	25
7. 一句话闭环	26
Week 3 — Neural Network: 从一个 Neuron 到多层神经网络	27
1. 先回答最重要的问题: 为什么需要 Neural Network?	27
2. 现实世界通常不是一条直线	28
3. 从 Week 2 的 Linear Regression 开始	28
4. Weight 到底是什么意思?	29
5. 这其实已经是一个 Neuron	29
Artificial Neuron	29
6. 为什么叫 Neuron?	30
7. 手算一个 Neuron	30
8. 但是这里有一个严重问题	31
9. 为什么多个 Linear Layer 还是 Linear?	32
10. Activation Function	32
Activation Function	33
11. Activation Function 为什么重要?	33
Non-linearity	33
12. 最容易理解的 Activation: ReLU	33
ReLU	34
13. ReLU 到底干什么?	34
14. 一个 Neuron 现在完整了	35
15. 手算一次带 ReLU 的 Neuron	35
16. 如果改变 Weight 呢?	36
17. 一个 Neuron 能做什么?	37
Layer	37
18. 什么是 Layer?	37
19. 为什么一个 Layer 要有很多 Neurons?	38
20. Hidden Layer 是什么?	38
21. Feature Learning	39
Representation	39
22. Deep Learning 的 Deep 是什么意思?	39
Deep Neural Network	40
23. 现在手算一个小型 Neural Network	40
24. Hidden Neuron 1	40
25. Hidden Neuron 2	41
26. 注意这里发生了什么	42
Representation Transformation	42

27. Output Layer	42
Forward Pass	43
28. Forward Pass 完整过程	43
29. Prediction 出来了，然后呢？	43
30. 这就是 Neural Network Training	44
31. 那么问题来了：这么多 Weight 怎么求 Gradient？	45
Backpropagation	45
32. 在进入 Backprop 前，先理解 Computational Graph	45
Computational Graph	46
33. 为什么 Computational Graph 很重要？	46
Forward	46
Backward	46
Chain Rule	47
34. Matrix 为什么会出现？	47
35. 不要被 Matrix 吓到	47
36. 用程序员角度理解 Matrix	48
37. 用纯 Python 实现一个 Neuron	48
38. 实现一个 Layer	49
39. 再连接 Output Layer	50
Neural Network	50
40. Neural Network 到底“学”的是啥？	50
Parameters	51
41. 为什么 Parameter 越多能力可能越强？	51
42. Parameters 是怎么数出来的？	52
43. 再举一个例子	52
44. Neural Network 和 Linear Regression 的真正关系	53
Linear Regression	53
Neural Network	53
45. 一个非常重要的公式	54
x	54
W	54
b	54
$Wx+b$	54
f	54
a	54
46. ReLU 不是唯一 Activation	54
Sigmoid	55
Tanh	55
ReLU	55
GELU	55
47. 为什么 Sigmoid 长这样？	55
48. Classification 和 Regression	56
Regression	56
Classification	56
49. Loss Function 也会改变	56
Cross Entropy Loss	57
50. 一个容易犯的错误：Neuron 是不是一个 if？	57
51. Traditional Programming vs Machine Learning	57
52. 那么 GPT 里的 Neuron 也是这些东西吗？	58
53. 从 Week 2 到 GPT 的完整地图	58
54. Week 3 最应该理解的 7 件事	59
55. Week 3 三个最重要公式	60
① Linear Calculation	60
② Activation	60
③ 完整 Neuron	60
56. 最后做一个理解测试	60
57. 用 Software Engineer 的视角总结 Week 3	61
58. Week 3 → Week 4：最关键的问题	62
Backpropagation	63
Week 3 补充深入理解：为什么多层 Neuron 能表达更多模式	63

1. Neuron 的输入、参数和输出分别是什么	63
2. 为什么 Weighted Sum 后还要 Activation	63
3. Hidden Layer 为什么有用	64
4. XOR 为什么是经典例子	64
5. Classification 的输出为什么不同	64
6. Parameter Count 为什么重要	64
7. 从 Forward 到 Learning 的连接	64
Week 4 - Backpropagation: 让网络里的所有参数一起学习	65
Week 4 核心目标	65
1. Backpropagation 不是新的 Training 方法	65
2. Computational Graph	65
3. Forward Pass: 先保存中间结果	66
4. Local Derivative: 每个小操作只负责自己	67
5. Chain Rule: 把局部影响串起来	69
6. 从 Loss 开始反向计算	70
7. Output Layer 的 gradients	70
8. 把 Gradient 继续传给 Hidden Layer	71
9. Gradient 经过 ReLU	71
10. Hidden Layer 的 gradients	71
11. 更新 Parameters	72
12. 再做一次 Forward	72
13. 用代码对应完整计算	72
14. 如果一个 Parameter 有多条路径	73
15. Batch 中为什么也要累加 Gradient	74
16. Autograd 做了什么	74
17. Vanishing 与 Exploding Gradient: 知道概念即可	74
18. Week 4 最应该理解的 6 件事	75
19. 理解测试	75
20. Week 4 → Week 5	75
Week 5 - Tensor 与 PyTorch: 把手算交给计算机	76
Week 5 核心目标	76
1. Tensor 到底是什么	76
2. Dimension 与 Shape	76
3. Dtype 与 Device	77
4. 用 Tensor 完成一个 Layer	77
5. Broadcasting: 为什么 [2] 能加到 [4,2]	78
6. 用 Tensor 重写 Week 2	79
7. requires_grad=True	79
8. .backward()	80
9. 为什么要清空 Gradient	80
10. 手动更新 Parameter	81
11. 完整的 Tensor Training Loop	81
12. nn.Module: 组织 Parameters 和 Forward	81
13. Loss Function 与 Optimizer	82
14. 标准 PyTorch Training Loop	82
15. Shape Debugging 的固定方法	83
16. train() 与 eval(): 知道用途即可	83
17. Week 5 最应该理解的 6 件事	83
18. Week 5 → Week 6	84
Week 6 - Embedding 与 Language Model: 把文字变成预测问题	85
Week 6 核心目标	85
1. Token ID 只是编号	85
Token ID 为什么仍然必不可少	85
2. 为什么需要 Embedding	85
为什么不直接把 ID 当作一个数输入	86
为什么不一律使用 One-hot	86
3. Embedding Matrix	86
4. Batch 进入 Embedding 后的 Shape	87

5. Embedding 是怎样学到的	87
一个 Embedding Row 为什么会更新	87
6. Language Model 到底是什么	88
为什么选择 Next-token Prediction	88
7. Autoregressive	88
8. Input 和 Target 为什么错开一位	89
为什么不能把 Input 和 Target 完全相同	89
9. Model 输出的不是 Probability	89
为什么 Model 先输出 Logits	89
10. Softmax: 把 Logits 变成 Probability	89
Softmax 为什么使用指数和归一化	90
11. Cross Entropy: 正确答案的 Probability 有多高	90
为什么使用 $-\log(p_{\text{correct}})$	91
12. 为什么 PyTorch CrossEntropyLoss 接收 Logits	91
13. 最简单的 Bigram Language Model	91
这张 Table 学到的是什么	92
14. 训练仍然是同一个 Loop	92
15. Sampling: 怎样从 Probability 生成 Token	92
Argmax	92
Sampling	92
16. Bigram Model 的关键局限	93
17. Perplexity: 知道含义即可	93
18. Week 6 最应该理解的 7 件事	93
19. Week 6 → Week 7	93
Week 7 - Attention: 让 Token 主动查找相关上下文	95
Week 7 核心目标	95
Attention 为什么会出现在	95
1. 最简单的 Context Aggregation	95
2. 用搜索系统理解 Q、K、V	95
Projection 为什么是可学习的	96
3. 为什么要有三份表示	96
4. Query 与 Key 的 Dot Product	96
Dot Product 为什么能表示匹配	97
5. Softmax 把 Scores 变成 Weights	97
6. 加权汇总 Values	97
7. Scaled Dot-Product Attention	98
8. 为什么除以 $\sqrt{d_k}$	98
一个数字直觉	98
9. Self-Attention 的 Shape	98
[T,T] Matrix 应该怎样读	99
10. 为什么 GPT 需要 Causal Mask	99
Training 为什么尤其需要 Mask	99
11. 为什么 Mask 在 Softmax 前使用 $-\infty$	100
12. 一个 Attention Head 的 PyTorch 实现	100
13. 代码与公式逐行对应	100
Projection	100
Similarity 与 Scaling	101
Weighting	101
14. 为什么叫 Self-Attention	101
15. Multi-Head Attention	101
为什么一头不够	102
16. Attention 不是什么	102
17. Attention 的计算成本	102
18. Week 7 最应该理解的 7 件事	102
19. Week 7 → Week 8	102
Week 8 - Transformer: 把 Attention 组装成可训练的网络	104
Week 8 核心目标	104
1. Attention 还缺什么	104

2. Token Embedding 不包含位置	104
Attention 自己为什么不能可靠知道顺序	104
3. Position Embedding	105
Position Embedding 学到什么	105
4. Transformer Block 的两个主要计算单元	105
Self-Attention	105
Feed-Forward Network	105
5. Feed-Forward Network	105
为什么先扩大到 4C	106
6. 为什么需要 GELU	106
Activation 为什么放在两次 Linear 之间	106
7. Residual Connection	106
Residual 为什么既帮助信息也帮助 Gradient	107
8. Residual 对 Shape 的要求	107
9. Layer Normalization 的直觉	107
LayerNorm 到底在哪个范围计算	108
10. Pre-Norm Transformer Block	108
为什么使用 Pre-Norm	108
11. 完整 Transformer Block	108
12. 堆叠多个 Blocks	109
13. 从 Transformer Output 到 Vocabulary Logits	109
14. GPT Forward Pass 的完整 Shape	110
15. Encoder、Decoder 和 Decoder-only	110
Encoder-only	110
Encoder-Decoder	110
Decoder-only	110
16. Transformer 的能力来自哪里	110
17. 需要知道但暂不展开的概念	111
18. Week 8 最应该理解的 7 件事	111
19. Week 8 → Week 9	111
Week 9 - Tokenizer: 文字怎样变成 GPT 的输入	112
Week 9 核心目标	112
Tokenizer 为什么是独立组件	112
1. Token 不等于 Word	112
Character-level	112
Word-level	112
Subword-level	112
Token 是计算单位，不是语言学真理	112
2. 为什么不简单按 Word 切	113
3. Vocabulary Size 的 Trade-off	113
Vocabulary 较小	113
Vocabulary 较大	113
用 Parameter Count 看 Trade-off	113
4. Character Tokenizer: 先理解完整闭环	113
为什么 Character Tokenizer 适合教学	114
5. Unicode 与 UTF-8	114
6. 未见 Token 问题	114
7. BPE 的核心直觉	115
手工再走一轮 BPE	115
8. Tokenizer Training 与 Text Encoding 不同	115
Tokenizer Training	115
Encoding	115
9. Tokenizer 与 Model 必须绑定	116
10. Special Tokens	116
11. Padding 与 Attention Mask	116
12. 把 Corpus 变成 Token Tensor	117
13. Train / Validation Split	117
14. 从 Token Stream 取一个 Example	117
15. 构造一个 Batch	118

16. Tokenizer 不负责理解语言	118
17. Week 9 最应该理解的 7 件事	118
18. Week 9 → Week 10	118
Week 10 - GPT Architecture: 把所有零件组装起来	120
Week 10 核心目标	120
1. 先定义 Configuration	120
Configuration 为什么需要集中定义	120
2. Configuration 的关键约束	121
3. GPT 的顶层数据流	121
为什么同一个 Forward 同时支持 Training 与 Inference	121
4. Causal Self-Attention Class	122
5. 拆分 Heads 时的 Shape	123
view 与 transpose 分别改变什么	123
6. FeedForward 与 Block	123
7. 完整 GPT Model	124
这段 Model 代码的 Ownership Map	125
8. ModuleList 为什么重要	125
9. Parameter 到底在哪里	125
10. 手算主要 Parameter Count	126
Token Embedding	126
Position Embedding	126
每个 Block 的 Attention	126
每个 Block 的 FFN	126
带入一个具体配置	127
11. Weight Tying: 知道用途即可	127
12. 正确性首先检查 Shape	127
13. 必须检查 Causal 性	128
14. Architecture 与 State Dict	128
15. 从 Week 2 的公式到 GPT	128
16. Week 10 最应该理解的 7 件事	129
17. Week 10 → Week 11	129
Week 11 - Training 与 Inference: 让 Mini GPT 学会生成文本	130
Week 11 核心目标	130
1. 一个 Batch 提供很多训练信号	130
2. Step 与 Epoch	130
Training Step	130
Epoch	131
3. 为什么常用 AdamW	131
AdamW 为什么比 “直接乘当前 Gradient” 更稳定	131
w 代表什么	132
4. 一个标准 Training Step	132
为什么这个顺序不能打乱	132
5. 完整 Training Loop	133
6. 初始 Loss 大约应该是多少	133
7. Validation 为什么必要	134
Validation 为什么不能更新 Parameters	134
8. 正确的 Validation 模式	134
9. Gradient Clipping	135
10. Learning Rate 仍然是关键	135
11. Overfit One Batch Test	135
12. Checkpoint 应保存什么	136
13. Inference 与 Training 的区别	136
14. 为什么生成时只取最后一个位置	137
15. Temperature	137
16. 手算 Temperature 的直觉	137
17. Top-k Sampling	138
18. 完整 Generation Loop	138
19. 为什么要截断 Context	139

20. 常见 Bug Checklist	139
Loss 不下降	139
输出提前看到答案	139
加载后乱码或完全异常	139
Device Error	139
Validation 波动很大	139
21. Week 11 最应该理解的 7 件事	140
22. Week 11 → Week 12	140
Week 12 - 完整 Mini GPT: 从 Text 到 Training 再到 Generation	141
Week 12 核心目标	141
1. 项目结构	141
2. 第一次运行先验证 Pipeline	141
3. 从一个字符串开始	142
4. 形成 Training Batch	142
5. Embedding Lookup	143
6. 进入 Transformer Block	143
7. Q/K/V Shape Trace	143
8. LM Head 产生 Logits	144
9. Cross Entropy 产生一个 Loss	144
10. Backward Trace	144
11. 为什么只有部分 Embedding Rows 更新	145
12. Optimizer Step	145
13. Training 前后的分布变化	145
14. Autoregressive Generation Trace	146
15. 为什么当前实现会重复计算过去	146
16. 三阶段实验方法	146
Stage 1: Pipeline Test	147
Stage 2: Learning Test	147
Stage 3: Generalization Experiment	147
17. 换成自己的 Corpus	147
18. 从 Character Tokenizer 升级 Subword	147
19. 如何安全地扩大 Model	148
20. Mini GPT 的测试清单	148
Tokenizer	148
Batch	148
Model	148
Gradient	148
Checkpoint	149
Causal Test	149
21. 十二周知识怎样汇合	149
Week 1	149
Week 2	149
Week 3	149
Week 4	149
Week 5	149
Week 6	149
Week 7	149
Week 8	149
Week 9	149
Week 10	149
Week 11	150
Week 12	150
22. 最终 Mental Model	150
23. 最终理解测试	150
24. 整个课程最重要的一句话	151
附录 A - 完整 Mini GPT 参考代码	152

阅读说明

这份深度扩展版以 Week 2 与你指定的 20:53 邮件《AI Learning Week 3 - Neural Network: 从一个 Neuron 到多层神经网络》为写作基准。

- Week 2: 保留原文;
- Week 3: 保留指定邮件原文, 公式仅转换为专业数学排版;
- Week 1、Week 4-12: 重新审查和编写;
- 关键知识: 使用“直觉 → 公式 → 手算 → 代码 → 下一章连接”详细解释;
- 相关但非前置知识: 只说明用途, 不打破主线。

整套课程只沿一条主线前进:

Data → Representation → Parameters → Prediction → Loss
→ Backpropagation → Parameter Update → Better Prediction

到 GPT 后, 数据与 architecture 变复杂, 但学习机制没有被替换。

章节索引

周次	章节	本周回答的核心问题
Week 1	必要数学直觉	怎样把 AI 公式翻译成数据、shape 和代码?
Week 2	Linear Regression	模型怎样根据错误学习 parameters?
Week 3	Neural Network	怎样从一个 $wx+b$ 扩展成多层网络?
Week 4	Backpropagation	Loss 怎样把责任传回每个 weight?
Week 5	Tensor 与 PyTorch	怎样让计算机自动执行大规模 forward/backward?
Week 6	Embedding 与 Language Model	文字怎样变成 next-token prediction?
Week 7	Attention	当前 token 怎样动态寻找相关 context?
Week 8	Transformer	Attention 怎样组成可堆叠的训练架构?
Week 9	Tokenizer	文字怎样稳定转换为模型 IDs?
Week 10	GPT Architecture	怎样把所有组件组合成 Mini GPT?
Week 11	Training 与 Inference	怎样训练、验证、保存并生成文本?
Week 12	完整 Mini GPT	整个系统怎样从 corpus 运行到 generated text?

Week 1 - 必要数学直觉：先学会读 AI 公式

Week 1 核心目标

这套教程不是要先补完一本高等数学。

我们的目标更实用：当你看到一个 AI 公式时，能够把它翻译成数据、代码和 shape。

本周只详细理解五件事：

1. Scalar、Vector、Matrix 分别是什么；
2. Shape 为什么像程序里的类型契约；
3. Dot Product 为什么是 neuron 的核心计算；
4. Function、Parameter 和 Input 的区别；
5. 如何把公式逐步翻译成 Python。

本周的主线是：

```
现实信息
↓
数字
↓
Vector / Matrix
↓
加权计算
↓
一个 Prediction
```

Week 2 会继续回答：Prediction 错了以后，模型怎样自动修改参数。

本章怎样学习每个概念

后面每个数学对象都不要只记名字。固定问四个问题：

1. 它是什么数据结构？
2. 为什么模型需要它？
3. 它在公式中做什么？
4. 它在代码中的 shape 和类型是什么？

数学在这里不是一组孤立公式，而是描述程序中数据流和变化关系的精确语言。

1. AI 首先处理的是数字

我们看到一句话：

这套房子离 CBD 很近

计算机不能直接对“很近”做矩阵乘法。我们需要把现实信息表示成数字，例如：

```
面积      = 100
卧室数量   = 3
距离 CBD   = 8
房龄       = 12
```

这些数字不是知识本身，而是模型能够处理的 representation（表示）。

为什么必须做 representation？因为现实概念没有统一的机器运算接口。“离 CBD 很近”“房屋较新”“用户很喜欢”都需要先变成可比较、可组合的数值。Representation 决定模型能够看见什么：如果输入里从未表达“距离”，再复杂的模型也无法直接利用这个信息。

因此 AI 的第一层设计问题往往不是选择多少 layers，而是：

怎样把问题中的对象和关系，转换成不会丢失关键含义的数字？

到了 GPT，文字也会经历类似过程：

文字 → Token ID → Embedding Vector → Neural Network

所以理解 Vector 和 Matrix，不是为了做纯数学，而是为了看懂数据怎样进入模型。

2. Scalar：一个数字

Scalar（标量）就是一个单独的数字：

房价 = 800000
温度 = 22.5
Loss = 0.18

Python 中：

```
price = 800000  
temperature = 22.5  
loss = 0.18
```

以后你会经常看到一个训练 batch 最后产生一个 scalar loss。因为只有一个最终 Loss，才能方便地从它开始向后计算所有参数的 gradient。

Scalar 的作用是表示“一个量”。它没有内部方向或位置结构。例如一个模型可以输出很多 predictions，但训练通常把它们对应的 errors 汇总成一个 scalar loss，作为本次 update 的统一目标。

这并不表示每个 example 的差异消失了。每个 example 对这个 scalar 都有贡献，Backward 会再把总 Loss 的影响分配回各个 parameters。

3. Vector：一组有顺序的数字

同一套房子的多个 feature 可以放进一个 Vector：

$$x = \begin{bmatrix} 100 \\ 3 \\ 8 \end{bmatrix}$$

分别表示：

x₁ = 面积
x₂ = 卧室数量
x₃ = 距离 CBD

代码里可以先把它看成 list：

```
x = [100, 3, 8]
```

Vector 的两个重点：

- 数字有固定顺序；
- 每个位置代表一个确定含义。

如果训练时顺序是 [面积, 卧室, 距离]，预测时却传入 [距离, 面积, 卧室]，程序可能不会报错，但含义已经完全错了。

为什么 Vector 不只是普通 List

List 强调“装了多个值”；Vector 还强调这些值共同表示同一个对象，并且可以作为整体参加加法、缩放、点积等运算。

例如用户画像：

[购买频率, 平均金额, 最近活跃度]

这三个数字的组合才是完整 representation。Vector 的 dimension 表示模型用多少个数字描述对象；dimension 越多不一定越好，关键是每个维度能否通过 training 承载有用信息。

4. Shape: 数据的结构契约

上面的 Vector 有三个元素，所以 shape 是：

```
[3]
```

把四套房子放在一起：

$$X = \begin{bmatrix} 100 & 3 & 8 \\ 80 & 2 & 15 \\ 140 & 4 & 6 \\ 95 & 3 & 11 \end{bmatrix}$$

它的 shape 是：

```
[4, 3]
```

意思是：

4 rows = 4 个 examples

3 columns = 每个 example 有 3 个 features

对软件工程师来说，可以把 shape 看成数据的 type signature：

```
Tensor[batch_size, feature_count]
```

很多 AI bug 不是公式错，而是 shape 不符合契约。

Dimension、Axis 与 Shape 的区别

[4,3] 是 shape；它有两个 dimensions（也常叫 axes）。第 0 维长度是 4，第 1 维长度是 3。

重要的不只是数字，还要写出语义：

```
X.shape = [4, 3]
```

```
axis 0 = examples
```

```
axis 1 = features
```

两个 tensors 即使 shape 相同，语义也可能不同。例如 [8,32] 可能是“8 个 tokens，每个 32 个 features”，也可能是“8 个 examples，每个 32 个 features”。框架只检查数字是否兼容，业务语义要由我们保证。

5. Weight: 模型怎样看待不同输入

假设模型给三个 feature 分配三个 weights：

$$w = \begin{bmatrix} 0.7 \\ 0.5 \\ -0.4 \end{bmatrix}$$

直觉上：

面积 weight 为正

卧室数量 weight 为正

距离 CBD weight 为负

Weight 可以先理解为：

某个输入影响结果的方向和强度。

这些 weights 不是程序员逐个写出来的。Training 会根据数据自动寻找它们。

Weight 为什么必须存在

如果所有 inputs 都直接相加，模型只能认为每个 feature 同样重要，而且只能产生固定规则。Weight 为每条输入路径提供一个可调旋钮：

- 正数表示 input 增大时倾向推高结果；
- 负数表示 input 增大时倾向压低结果；
- 接近 0 表示当前位置该 input 的影响很弱；
- 绝对值较大表示影响倍率较强。

Weight 的含义还与 feature 的尺度有关。面积用平方米、价格用万元或元，会改变数字大小，所以不能脱离 input scale 机械比较两个 weight 的绝对值。

6. Dot Product: 把多个输入合成一个分数

Dot Product（点积）是本周最关键的计算。

它之所以反复出现在神经网络中，是因为它能把“多个证据及各自重要程度”压缩成一个综合分数。每个 $w_i x_i$ 是一项证据，求和后得到 neuron 对当前 input 的总体响应。

公式：

$$w \cdot x = w_1 x_1 + w_2 x_2 + w_3 x_3$$

它做的事情非常朴素：

```
每个 input × 对应 weight
↓
全部相加
↓
得到一个数
```

为了方便手算，假设：

```
x = [0.8, 0.6, 0.2]
w = [0.7, 0.5, -0.4]
```

那么：

$$w \cdot x = (0.7)(0.8) + (0.5)(0.6) + (-0.4)(0.2)$$

$$= 0.56 + 0.30 - 0.08$$

$$= 0.78$$

Python：

```
x = [0.8, 0.6, 0.2]
w = [0.7, 0.5, -0.4]

score = 0.0

for input_value, weight in zip(x, w):
    score += input_value * weight

print(score) # 0.78
```

数学符号 $\cdot$ ，对应的就是“对应相乘，再全部相加”。

Dot Product 为什么要求长度一致

每个 input 必须能找到唯一对应的 weight:

```
x1 ↔ w1
x2 ↔ w2
x3 ↔ w3
```

如果 x 有三个 features、w 只有两个 weights，第三个 feature 没有明确的影响规则。这就是 shape mismatch 背后的业务含义，不只是矩阵库的语法限制。

7. Bias: 额外的可学习偏移

模型通常还会加一个 Bias:

$$z = w \cdot x + b$$

假设:

```
w · x = 0.78
b = 0.10
```

那么:

$$z = 0.78 + 0.10 = 0.88$$

代码:

```
bias = 0.1
z = score + bias
```

Bias 让模型不必在所有输入都为 0 时强制输出 0。它和 weight 一样，也是模型需要学习的 parameter。

用门槛理解 Bias

假设 neuron 在 $z > 0$ 时被认为“触发”。没有 bias 时，判断边界必须穿过原点。加入 bias 后:

$$w \cdot x + b > 0$$

等价于允许模型移动触发门槛。例如综合证据分数是 0.78，但只有超过 0.90 才希望触发，那么模型可以学到 $b = -0.90$ 。Weight 控制边界方向，Bias 控制边界位置；二者缺一会限制模型能表达的规则。

8. Function: 输入经过规则得到输出

Function 可以理解成一个有明确输入和输出的处理过程:

$$y = f(x)$$

代码:

```
def f(x):
    return 2 * x + 1
```

对于模型:

```
def model(x, w, b):
    return w * x + b
```

这里:

- x 是 Input;
- w、b 是 Parameters;

- 函数结构是 Model;
- 返回值是 Prediction。

这四个词在 Week 2 会反复出现。

Function 与 Model 的关系

所有 model 都可以看作 function，但不是所有 function 都会学习。普通函数的规则由程序员固定；可训练 model 的结构由程序员定义，其中一部分数值 parameters 由数据决定。

固定代码：return $w * x + b$

可学习状态：w, b

运行时输入：x

产生结果：prediction

Training 改变的是 w 和 b，不是每一步重新改写 forward 代码。

9. Variable、Parameter 与 Hyperparameter

三者很容易混淆。

Variable

计算过程中可能变化的值，例如 input、prediction、loss。

Parameter

模型通过 training 学到的值，例如 weights 和 biases。

Hyperparameter

由训练者先选择的设置，例如 learning rate、batch size、layer count。

Parameter → model learns

Hyperparameter → human chooses

为什么 Hyperparameter 不能简单交给同一次 training 自动学习？因为它们往往定义“学习过程本身”或 architecture 结构，例如 layer count 会决定 graph 有多少节点，learning rate 会决定 parameter update 的步长。它们可以通过外层实验、搜索或其他算法调优，但不属于普通 forward/backward 中直接更新的 model parameters。

10. Matrix：同时处理很多向量

如果一个 neuron 有一组 weights，那么多个 neurons 就有多组 weights。

例如三个 inputs 进入两个 neurons：

$$W = \begin{bmatrix} 0.7 & 0.2 \\ 0.5 & -0.1 \\ -0.4 & 0.8 \end{bmatrix}$$

这里每一列可以看成是一个 neuron 的 weights。

输入：

$$x = [0.8 \quad 0.6 \quad 0.2]$$

计算：

$$z = xW + b$$

本质上是在同时做两个 Dot Products。

Shape:

```
x: [1, 3]
W: [3, 2]
z: [1, 2]
```

中间的 3 必须一致：三个 input features 对应每个 neuron 的三个 weights。

Matrix 存在的工程价值是批量化。与其用 Python loop 为每个 example、每个 neuron 逐个计算 dot product，不如把同类运算排列成矩阵乘法，让底层库在 CPU/GPU 上并行执行。神经网络的大部分算力最终都花在这类密集的矩阵乘法上。

11. Matrix Multiplication 的最重要规则

不用现在背完整矩阵乘法，只记住 shape:

$$[m, n] \times [n, p] \rightarrow [m, p]$$

中间维度必须相同。

例如：

$$[4, 3] \times [3, 2] \rightarrow [4, 2]$$

意思是：

```
4 个 examples
每个有 3 个 features
进入 2 个 neurons
得到每个 example 的 2 个 outputs
```

这比死记矩阵计算步骤更重要。

输出 Shape 为什么是 [m,p]

左侧 matrix 提供 m 个 examples，右侧 matrix 提供 p 组 output weights。每个 example 都会与每组 weights 做一次 dot product，因此最终得到：

```
m 个 examples × p 个 outputs
```

中间的 n 是每次 dot product 要配对并消掉的 feature dimension，所以不会出现在输出 shape 中。

12. 求和符号 Σ

下面两个公式是同一件事：

$$w_1x_1 + w_2x_2 + w_3x_3$$

$$\sum_{i=1}^3 w_i x_i$$

Σ 只是在说：让 i 从 1 走到 3，把每一项加起来。

它对应代码：

```
total = 0

for i in range(3):
    total += w[i] * x[i]
```

看到 Σ 时，先把它翻译成 loop。

Σ 存在的原因只是让重复模式写得更紧凑。它不引入新的 operation；下标告诉 loop 从哪里开始，上标告诉 loop 到哪里结束，右侧表达式就是每次迭代累加的内容。

13. 平方、指数、概率：先建立最低限度直觉

这些概念后面会出现，但本周不需要展开。

平方

$$x^2 = x \times x$$

Week 2 会用平方表示 error 的大小。

指数

$$e^x$$

它能把任意实数转换成正数。Week 6 的 Softmax 会使用它。

概率

$$0 \leq p \leq 1$$

概率越接近 1，代表模型越倾向某个结果。Language Model 会为下一个 token 产生一整组概率。

现在只需要知道它们将在哪里使用，不需要提前记公式。

为什么平方适合表示误差

正误差和负误差不应互相抵消，平方会把它们都变成非负数；同时较大错误会受到更强惩罚。Week 2 会把这一点发展成 MSE。

为什么指数适合生成概率

Softmax 需要把任意 logits 转为正数，再归一化到总和为 1。指数函数始终为正，而且会保留“更大的 logit 应得到更大权重”的顺序。

概率不等于确定事实

$p=0.8$ 表示模型在当前参数和上下文下给予某结果更高权重，不代表现实世界中有 80% 的客观真理。模型概率是对其学到的数据分布的表达。

14. 读 AI 公式的固定顺序

以后看到公式，例如：

$$z = Wx + b$$

按下面顺序读：

1. 每个符号代表什么？
2. 每个对象的 shape 是什么？
3. 运算顺序是什么？
4. 输出 shape 是什么？
5. 哪些值是 parameters？
6. 代码里是哪一行？

例如：

```
x : [batch, input_features]
W : [input_features, neurons]
b : [neurons]
z : [batch, neurons]
```

然后才看代码：

```
z = x @ W + b
```

如果只看符号而不看 shape， Wx 和 xW 很容易被当成同一件事；如果只看代码而不问参数含义，又会把能运行的 tensor 运算误当成正确模型。固定的六步阅读顺序，就是在“数学正确、shape 正确、业务语义正确”之间建立检查链。

15. Week 1 Python 小实验

```
inputs = [0.8, 0.6, 0.2]

weights = [
    [0.7, 0.5, -0.4],
    [0.2, -0.1, 0.8],
]

biases = [0.1, -0.2]

outputs = []

for neuron_weights, bias in zip(weights, biases):
    z = bias

    for x, w in zip(inputs, neuron_weights):
        z += x * w

    outputs.append(z)

print(outputs)
```

这段代码已经完成了一个 neural network layer 的核心计算：

```
Inputs
↓
Dot Products
↓
Add Biases
↓
Layer Outputs
```

我们还没有加入 activation，也没有 training。Week 2 和 Week 3 会依次补上。

16. Week 1 最应该理解的 6 件事

1. 现实信息要先转换成数字表示。
2. Vector 是一个 example 的一组有顺序的 features。
3. Matrix 可以同时装下多个 examples 或多组 weights。
4. Shape 是数据结构契约。

5. Dot Product 是“对应相乘再相加”。
6. $Wx+b$ 是后续 neuron、layer、Transformer 的基础计算。

17. 理解测试

假设：

```
x = [2, 3]
w = [0.5, -0.2]
b = 0.1
```

计算：

$$z = w \cdot x + b$$

答案：

$$\begin{aligned} z &= (0.5)(2) + (-0.2)(3) + 0.1 \\ &= 1 - 0.6 + 0.1 = 0.5 \end{aligned}$$

如果你能把它直接翻译成：

```
z = 0.5 * 2 + (-0.2) * 3 + 0.1
```

本周的核心已经掌握。

18. Week 1 → Week 2

我们现在可以用 parameters 做 prediction：

$$\hat{y} = wx + b$$

但仍有一个关键问题：

如果 prediction 错了，计算机怎样知道应该修改哪个 parameter、修改方向是什么、每次修改多少？

Week 2 会用 Loss、Derivative、Gradient 和 Gradient Descent 回答这个问题。

Week 2 — Linear Regression: 从 Prediction 到 Gradient Descent

本周目标不是背公式，而是彻底理解这条主线：

Data → Parameters → Prediction → Error → Loss → Gradient → Update → Better Prediction

只要这条链真正理解了，后面的 Neural Network、Backpropagation，甚至 Transformer / GPT 的训练都会顺很多。

1. Linear Regression 解决什么问题？

Linear Regression（线性回归）用来预测连续数值，例如房价、销量、温度、距离等。

假设历史数据：

x（面积）	y（房价）
1	3
2	5
3	7
4	9

规律是：

$$y = 2x + 1$$

如果 $x = 5$ ，则预测 $y = 11$ 。

线性回归的核心就是：根据已有数据，找到最合适的一条直线，再用它预测新的数据。

2. Model、Feature、Parameter

模型：

$$\hat{y} = wx + b$$

其中：

- x : Feature, 输入特征
- y : 真实值 / Target
- $\hat{y}$: Prediction, 预测值
- w : Weight, 权重
- b : Bias, 偏置
- w 、 b : Parameters, 模型需要通过训练找到的参数

程序员视角：

```
def predict(x, w, b):  
    return w * x + b
```

训练的本质，就是找到最合适的 w 和 b 。

3. Prediction 是怎么产生的?

假设:

$w = 1$
 $b = 0$
 $x = 2$

那么:

$$\hat{y} = \mathbf{w}\mathbf{x} + b = 1 \times 2 + 0 = 2$$

但真实答案可能是:

$$y = 5$$

于是:

Prediction = 2
Actual = 5

模型预测错了。

4. Error 与 Loss 的区别

最简单的 Error:

$$\text{Error} = \hat{y} - y$$

例如:

$$2 - 5 = -3$$

说明预测低了 3。

但直接把多个 Error 相加会有正负抵消的问题, 所以训练时通常用 Loss Function。

最常见的一种:

$$\text{SquaredError} = (\hat{y} - y)^2$$

例如:

$$(2 - 5)^2 = 9$$

平方有两个作用:

1. 消除正负号;
2. 更严重地惩罚大误差。

5. MSE 什么时候用?

有多个样本时, 通常把所有 Squared Error 取平均:

$$\text{MSE} = (1/n) \times \sum (\hat{y}_i - y_i)^2$$

MSE = Mean Squared Error, 均方误差。

例如:

真实值: 3, 5, 7
预测值: 2, 4, 8
误差: -1, -1, +1
平方误差: 1, 1, 1

$$\text{MSE} = (1 + 1 + 1) / 3 = 1$$

所以可以这样记:

- 单个样本: Squared Error
- 多个样本: MSE

MSE 主要用于 Regression, 即预测连续数值; 分类问题通常会使用 Cross Entropy 等 Loss。

6. 为什么还需要 Gradient?

现在模型知道 Loss 很大, 但它还不知道:

w 和 **b** 应该往哪个方向改? 改多少?

Gradient 就是用来回答这个问题的。

7. Derivative: 先理解 “坡度”

导数 Derivative 可以先理解成:

当前位置的 slope (坡度 / 变化率)。

例如:

- derivative > 0: 向右走, 函数值上升
- derivative < 0: 向右走, 函数值下降
- derivative = 0: 当前位置是平的

假设:

$$f(x) = x^2$$

导数是:

$$f'(x) = 2x$$

当 $x = 2$:

$$f'(2) = 4$$

表示此处 x 稍微增加一点, y 大约以 4 倍速度变化。

8. Gradient 是什么?

对一个参数 w 来说:

$$\text{Gradient} = \frac{d\text{Loss}}{dw}$$

它告诉我们: 如果稍微改变 w , Loss 会怎么变化。

最重要的是符号:

```
gradient > 0
→ w 增加会让 Loss 增加
→ 应该减少 w

gradient < 0
→ w 增加会让 Loss 减少
→ 应该增加 w
```

因此参数更新公式是：

$$w_{\text{new}} = w_{\text{old}} - \text{learning}_{\text{rate}} \times \text{gradient}$$

为什么是减号？因为 Gradient 指向 Loss 增长最快的方向，我们要向它的反方向走。

9. Gradient Descent

Gradient Descent（梯度下降）可以想象成：

你站在山上，看不到整座山，只能感觉脚下坡度。

目标是走到最低点。

```
calculate gradient
↓
move downhill
↓
calculate gradient again
↓
move downhill again
↓
repeat
```

在机器学习里，最低点代表 Loss 尽可能小。

10. Learning Rate

参数更新：

$$\text{parameter}_{\text{new}} = \text{parameter}_{\text{old}} - \text{learning}_{\text{rate}} \times \text{gradient}$$

Learning Rate 控制每一步走多大。

- 太大：可能跨过最低点，震荡甚至发散
- 太小：训练很慢

Learning Rate 是 Hyperparameter。

注意区别：

- Parameter：模型自己学出来，如 w、b
 - Hyperparameter：人设置，如 learning_rate、batch_size、epochs
-

11. 完整手算一次 w 更新

只用一个训练样本：

$x = 2$
 $y = 5$

模型先简化为：

$$\hat{y} = wx$$

初始：

$w = 1$
 $\text{learning_rate} = 0.01$

Step 1 — Prediction

$$\hat{y} = 1 \times 2 = 2$$

Step 2 — Loss

$$\text{Loss} = (\hat{y} - y)^2 = (2 - 5)^2 = 9$$

Step 3 — Gradient

Loss：

$$\text{Loss} = (\hat{y} - y)^2$$

Prediction：

$$\hat{y} = wx$$

使用 Chain Rule：

$$\frac{d\text{Loss}}{dw} = (d\text{Loss}/d\hat{y}) \times (d\hat{y}/dw)$$

第一部分：

$$d\text{Loss}/d\hat{y} = 2(\hat{y} - y)$$

代入：

$$2(2 - 5) = -6$$

第二部分：

$$d\hat{y}/dw = x = 2$$

所以：

$$\frac{d\text{Loss}}{dw} = -6 \times 2 = -12$$

Gradient = -12。

负数说明增加 w 可以让 Loss 下降。

Step 4 — Update

$$w_{\text{new}} = 1 - 0.01 \times (-12)$$

所以：

$$w_{\text{new}} = 1.12$$

新的 Prediction：

$$\hat{y} = 1.12 \times 2 = 2.24$$

新的 Loss：

$$(2.24 - 5)^2 = 7.6176$$

之前 Loss = 9，现在约等于 7.62。

模型真的变好了。

12. Chain Rule 为什么重要？

依赖关系是：

```
w
↓
prediction ŷ
↓
Loss
```

所以要知道 w 如何影响 Loss，需要沿 dependency chain 往回算。

这就是未来 Backpropagation 的核心思想。

13. Bias b 与 Partial Derivative

完整模型：

$$\hat{y} = \mathbf{w}\mathbf{x} + b$$

现在 Loss 同时依赖 w 和 b：

$$\text{Loss} = f(w, b)$$

我们分别计算：

$$\frac{\partial \text{Loss}}{\partial w}$$

和：

$$\frac{\partial \text{Loss}}{\partial b}$$

Partial Derivative（偏导数）可以理解为：多个变量时，一次只研究其中一个变量变化对结果的影响。

对于单个样本：

$$\frac{\partial \text{Loss}}{\partial w} = 2(\hat{y} - y)x$$

$$\frac{\partial \text{Loss}}{\partial b} = 2(\hat{y} - y)$$

多个参数的这些偏导数组合起来，就是 Gradient。

14. 同时更新 w 和 b

假设：

```
w = 1
b = 0
gradient_w = -12
gradient_b = -6
learning_rate = 0.01
```

更新：

```
w = 1 - 0.01 × (-12) = 1.12
b = 0 - 0.01 × (-6) = 0.06
```

新模型：

$$\hat{y} = 1.12x + 0.06$$

这就是模型“学习了一小步”。

15. Second Derivative 二阶导数

一阶导数描述：

变化率 / slope

二阶导数描述：

变化率本身变化得有多快。

一个直观类比：

```
Position
↓ derivative
Velocity
↓ derivative
Acceleration
```

在优化里，二阶导数还能帮助描述 Loss 曲面的弯曲程度。

Week 2 只需建立这个直觉，不需要深入 Hessian。

16. Least Squares 与 Gradient Descent

这两个概念不要混淆。

Least Squares 回答：

我们想优化什么？

也就是让 squared errors 总和尽可能小。

Gradient Descent 回答：

我们怎么一步步找到较小的 Loss？

因此：

Least Squares = Objective

Gradient Descent = Optimization method

简单线性回归也可以直接通过数学公式得到最优参数，但 Neural Network 有几百万、几十亿参数时，通常必须靠迭代优化。

17. Python 从零实现

不使用 sklearn / PyTorch / TensorFlow：

```
x_data = [1, 2, 3, 4]
y_data = [3, 5, 7, 9]

w = 0.0
b = 0.0
learning_rate = 0.01

for epoch in range(1000):
    dw = 0.0
    db = 0.0
    loss = 0.0

    n = len(x_data)

    for x, y in zip(x_data, y_data):
        # Forward pass
        prediction = w * x + b

        # Error
        error = prediction - y

        # Squared error
        loss += error ** 2

    # Gradients
    dw += 2 * error * x
    db += 2 * error

    # MSE / mean gradients
    loss /= n
    dw /= n
    db /= n

    # Gradient descent
    w -= learning_rate * dw
    b -= learning_rate * db

print('w:', w)
print('b:', b)
```

最终会接近：

$w \approx 2$
 $b \approx 1$

也就是说模型自己从数据中学习出了：

$$y \approx 2x + 1$$

18. 代码与数学一一对应

Model:

`prediction = w * x + b`

对应：

$$\hat{y} = \mathbf{w}\mathbf{x} + b$$

Error:

`error = prediction - y`

对应：

$$\hat{y} - y$$

Loss:

`loss += error ** 2`

对应：

$$(\hat{y} - y)^2$$

Gradient for w:

`dw += 2 * error * x`

对应：

$$\frac{\partial \text{Loss}}{\partial w} = 2(\hat{y} - y)x$$

Gradient for b:

`db += 2 * error`

对应：

$$\frac{\partial \text{Loss}}{\partial b} = 2(\hat{y} - y)$$

Parameter update:

`w -= learning_rate * dw`
`b -= learning_rate * db`

对应：

`parameter = parameter - learning_rate × gradient`

19. Forward Pass 与 Backward Pass

Forward Pass:

```
Input
↓
Model
↓
Prediction
↓
Loss
```

Backward Pass:

```
Loss
↓
Gradients
↓
Update Parameters
```

这已经是最简单版本的 Backpropagation 雏形。

20. Epoch 与 Batch

Epoch: 模型完整看完训练数据一次。

例如:

```
for epoch in range(1000):
```

表示训练数据被重复学习 1000 次。

Batch: 一次只拿部分训练数据计算 Forward → Loss → Gradient → Update。

例如一百万条数据, 可以按 batch_size = 32 或 64 分批训练。

21. Linear Regression 与 Neural Network 的关系

Linear Regression:

$$\hat{y} = \mathbf{w}\mathbf{x} + b$$

一个 Neural Network neuron 的核心计算也是:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

之后通常再接 Activation Function。

所以 Linear Regression 可以看作理解 neuron 的最简单入口。

22. 从 Linear Regression 到 GPT

Linear Regression:

Input
↓
Parameters
↓
Prediction
↓
Loss
↓
Gradient
↓
Update Parameters

Neural Network:

Input
↓
Many layers / weights
↓
Prediction
↓
Loss
↓
Backpropagation
↓
Update weights

GPT:

Tokens
↓
Transformer
↓
Next-token Prediction
↓
Loss
↓
Backpropagation
↓
Update huge numbers of parameters

结构和规模变复杂了，但最底层的训练逻辑仍然是同一条主线。

23. Week 2 必须掌握的词

English	中文	核心理解
Feature	特征	输入 x
Target / Label	目标 / 标签	正确答案 y
Model	模型	$wx+b$
Parameter	参数	w 、 b
Prediction	预测	$\hat{y}$
Error	误差	$\hat{y}-y$
Loss	损失	模型有多差
MSE	均方误差	多样本 squared error 平均
Derivative	导数	slope / 变化率
Partial Derivative	偏导	单个参数对 Loss 的影响
Gradient	梯度	各参数偏导组成的方向信息
Gradient Descent	梯度下降	向 Loss 更小的方向走
Learning Rate	学习率	每步走多远
Epoch	训练轮次	完整看一次数据集
Batch	批次	一次处理部分样本

English	中文	核心理解
Forward Pass	前向传播	Input → Prediction → Loss
Backward Pass	反向传播	Loss → Gradients
Chain Rule	链式法则	沿依赖关系求导

24. Week 2 最重要的 5 个公式

1. Prediction

$$\hat{y} = \mathbf{w}\mathbf{x} + b$$

2. MSE

$$\text{MSE} = (1/n) \times \sum (\hat{y}_i - y_i)^2$$

3. Gradient for w

$$\frac{\partial \text{Loss}}{\partial w} = 2(\hat{y} - y)x$$

4. Gradient for b

$$\frac{\partial \text{Loss}}{\partial b} = 2(\hat{y} - y)$$

5. Gradient Descent

$$\text{Parameter}_{\text{new}} = \text{Parameter}_{\text{old}} - \text{LearningRate} \times \text{Gradient}$$

其中第 5 个最重要。

25. 最终 Mental Model

请把下面这条链记牢：

```

DATA
↓
PARAMETERS
↓
MODEL
↓
PREDICTION
↓
COMPARE WITH TARGET
↓
LOSS
↓
GRADIENT
↓
UPDATE PARAMETERS
↓
BETTER MODEL

```


一句话概括 Machine Learning:

Learning = finding parameters that minimize loss.

程序员版本:

```
while training:
    prediction = model(input, parameters)
    loss = calculate_loss(prediction, actual)
    gradients = calculate_gradients(loss, parameters)
    parameters -= learning_rate * gradients
```

这就是 Week 3 Neural Network、Backpropagation，以及后面 Transformer / GPT 的地基。

Week 2 补充深入理解：把“公式结论”变成可运行的直觉

本节不改写 Week 2 原文，而是在原文之后补充几个最容易卡住的连接。原文介绍了 Prediction、Loss、Derivative、Gradient 和 Gradient Descent；下面把它们放回同一条因果链。

1. Loss 不是 Gradient

Loss 是一个数，表示当前参数下模型有多差。例如：

$$Loss(w) = (2w - 5)^2$$

当 $w=1$ 时，prediction 是 2，Loss 是 9。

Gradient 则是 Loss 对参数的变化率：

$$\frac{dLoss}{dw} = -12$$

它回答的是另一个问题：如果只把 w 增加一点，Loss 会增加还是减少、变化有多快？

Loss → 当前位置有多差

Gradient → 参数往哪边改会变好

没有指定“对谁求导”时，不能只说“Loss 的导数”。训练中通常是 Loss 对 weights、biases 等 trainable parameters 的导数。

2. 一个 parameter 与多个 parameters

如果只有一个参数 w ：

$$gradient = \frac{dLoss}{dw}$$

这是一个数字，可以想成一维曲线在当前位置的坡度。

如果有 w 和 b ：

$$\nabla Loss = \begin{bmatrix} \frac{\partial Loss}{\partial w} \\ \frac{\partial Loss}{\partial b} \end{bmatrix}$$

Gradient 变成向量。它描述 Loss 曲面上每个参数方向的局部坡度，整个向量指向 Loss 增长最快的方向。Gradient Descent 使用负方向：

$$\theta_{new} = \theta_{old} - \alpha \nabla Loss$$

3. 为什么 Gradient 为负时参数会增加

假设：

```
w = 1
gradient = -12
learning_rate = 0.01
```

更新：

$$w_{new} = 1 - 0.01(-12) = 1.12$$

负号来自 Gradient Descent 的“反坡度”原则。Gradient 为负表示向右增加 w 会让 Loss 下降；公式中的减号把这个负数变成正的 update。

4. Chain Rule 为什么是 Backprop 的桥

在模型中，Loss 通常不是直接由 weight 得到：

$w \rightarrow \text{weighted sum} \rightarrow \text{activation} \rightarrow \text{prediction} \rightarrow \text{Loss}$

如果每一步都是一个 function， w 的改变会先影响中间值，中间值再影响后面的值。Chain Rule 把每一段局部影响相乘：

$$\frac{dLoss}{dw} = \frac{dLoss}{d\hat{y}} \frac{d\hat{y}}{dz} \frac{dz}{dw}$$

每一项都可以由对应 operation 单独提供，因此一个 node 不需要理解整个 network。

5. 为什么本周不需要二阶导数

一阶导数告诉我们当前位置的坡度；训练的第一步问题是“向左还是向右”。这已经足够执行：

$$parameter \leftarrow parameter - learningRate \times gradient$$

二阶导数：

$$\frac{d^2 Loss}{dw^2}$$

告诉我们坡度变化得多快，也就是曲面弯曲程度。它可以帮助某些方法估计更合适的步长，但对有数百万参数的网络，完整 Hessian matrix 非常昂贵。因此 GPT 训练主要使用一阶 gradient 的 SGD、Momentum、AdamW 等方法。

6. 用有限变化检查导数方向

导数也可以用一个很小的试探步近似检查：

$$\frac{dLoss}{dw} \approx \frac{Loss(w + \epsilon) - Loss(w)}{\epsilon}$$

以 $w=1$ 、 $\epsilon=0.001$ 为例：

```
Loss(1)    = 9.000000
Loss(1.001) ≈ 8.988004
变化       ≈ -0.011996
除以 0.001 ≈ -11.996
```

接近解析结果 -12。这种 finite-difference check 是工程中验证手写 gradient 的好工具，但不适合代替 Backprop 训练，因为每个 parameter 都需要额外 forward，成本很高。

7. 一句话闭环

Parameters 决定 prediction

Prediction 与 target 决定 Loss

Loss 对 parameters 的 gradient 提供方向

Gradient Descent 沿反方向更新 parameters

重复后, prediction 通常逐渐改善

Week 3 — Neural Network: 从一个 Neuron 到多层神经网络

Week 3 核心目标

Week 2 我们已经理解：

Data → Prediction → Loss → Gradient → Update Parameters

Week 3 不会推翻这些东西。

Neural Network 本质上只是把 Week 2 的：

$$\hat{y} = wx + b$$

从一个计算单元，扩展成大量相互连接的计算单元。

1. 先回答最重要的问题：为什么需要 Neural Network?

Week 2 学习了 Linear Regression：

$$\hat{y} = wx + b$$

例如预测房价：

$$\text{Price} = w \times \text{Area} + b$$

假设：

$w = 5000$

$b = 100000$

那么 100m² 房子：

$$\text{Price} = 5000 \times 100 + 100000$$

得到：

\$600,000

这种关系很简单：

面积增加

↓

价格基本按固定速度增加

也就是一个近似的线性关系 **Linear Relationship**。

但现实世界远没有这么简单。

2. 现实世界通常不是一条直线

比如预测房价，可能需要：

面积
卧室数量
房龄
距离 CBD 距离
学区
犯罪率
土地面积
装修情况
利率
市场环境
...

更麻烦的是，这些因素之间还会互相影响。

例如：

面积很大
+
距离 CBD 很远
可能并不值钱。

但：

面积大
+
好学区
+
距离 CBD 近

价格可能突然非常高。

也就是说：

现实问题通常是 **Non-linear**（非线性）的。

这就是 Neural Network 开始发挥作用的地方。

3. 从 Week 2 的 Linear Regression 开始

Week 2:

$$\hat{y} = wx + b$$

现在假设有三个 Feature：

- x_1 ：面积
- x_2 ：卧室数量
- x_3 ：距离 CBD

模型可以写成：

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

看起来复杂了一点。

但本质没变。

以前：

$$wx + b$$

现在只是：

$$w_1x_1 + w_2x_2 + w_3x_3 + b$$

也就是：

每个 input 都有自己的 weight。

4. Weight 到底是什么意思？

这是 Neural Network 中最重要的概念之一。

假设：

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

如果：

x1 = 面积

x2 = 卧室

x3 = 距离CBD

模型训练后可能得到：

w1 = 0.8

w2 = 0.4

w3 = -0.7

直觉上可以理解：

面积 → 正影响比较大

卧室数量 → 正影响

距离CBD → 负影响

所以 Weight 可以暂时理解成：

模型认为某个输入有多重要，以及它产生正影响还是负影响。

但注意：到了大型 Neural Network 后，单个 weight 通常不再具有这么直接的人类语义。

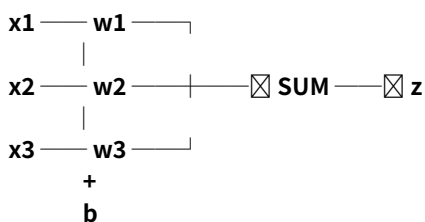
5. 这其实已经是一个 Neuron

下面这个东西：

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

就是神经网络最基础的计算单元。

可以画成：



这就是一个最简单的：

Artificial Neuron

人工神经元。

6. 为什么叫 Neuron?

它的灵感来自生物神经元，但不要把两者看得过于相同。

人工 neuron 的工作其实非常简单：

接收 Inputs



乘以 Weights



加起来



加 Bias



得到一个数

数学就是：

$$z = \sum_i w_i x_i + b$$

那个：

$$\Sigma$$

只是”全部加起来”的数学写法。

例如：

$$\sum_{i=1}^3 w_i x_i$$

就是：

$$w_1 x_1 + w_2 x_2 + w_3 x_3$$

所以不要被 Σ 吓到。

7. 手算一个 Neuron

假设我们做一个非常简单的”是否值得买房”模型。

三个输入：

x1 = 0.8 地段

x2 = 0.6 房屋状况

x3 = 0.2 距离问题

Weights:

w1 = 0.7

w2 = 0.5

w3 = -0.4

Bias:

b = 0.1

计算：

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

代进去：

$$z = (0.7)(0.8) + (0.5)(0.6) + (-0.4)(0.2) + 0.1$$

一步一步：

$$0.7 \times 0.8 = 0.56$$

$$0.5 \times 0.6 = 0.30$$

$$-0.4 \times 0.2 = -0.08$$

所以：

$$z = 0.56 + 0.30 - 0.08 + 0.1$$

$$z = 0.88$$

到目前为止，仍然没有什么神秘的。

就是：

$$z = x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + b$$

8. 但是这里有一个严重问题

如果 neuron 只有：

$$z = wx + b$$

那么即使我们放很多层：

Input
↓
Linear
↓
Linear
↓
Linear
↓
Output

最终仍然只是一个 Linear Model。

这是理解 Neural Network 的一个关键点。

9. 为什么多个 Linear Layer 还是 Linear?

假设第一层：

$$h = w_1x + b_1$$

第二层：

$$y = w_2h + b_2$$

把第一层代入：

$$y = w_2(w_1x + b_1) + b_2$$

展开：

$$y = w_2w_1x + w_2b_1 + b_2$$

把：

$$w_2w_1$$

看成一个新的 weight：

$$W$$

把：

$$w_2b_1 + b_2$$

看成新的 bias：

$$B$$

最后：

$$y = Wx + B$$

你会发现：

两层 Linear 仍然等价于一层 Linear。

10 层也一样。

100 层也一样。

所以仅仅堆 $wx+b$ 没意义。

我们缺一个东西。

10. Activation Function

这就是：

ReLU

全称：

Rectified Linear Unit

公式：

$$\text{ReLU}(x) = \max(0, x)$$

看起来像数学。

其实翻译成代码就是：

```
def relu(x):  
    return max(0, x)
```

就是这么简单。

13. ReLU 到底干什么？

如果：

$x = 5$

那么：

$$\text{ReLU}(5) = 5$$

如果：

$x = -3$

那么：

$$\text{ReLU}(-3) = 0$$

如果：

$x = 0.7$

那么：

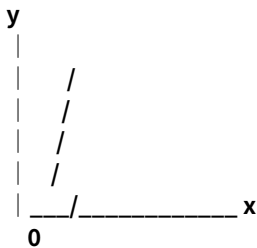
$$\text{ReLU}(0.7) = 0.7$$

所以：

negative $\rightarrow 0$

positive $\rightarrow$ keep it

图形大概：



14. 一个 Neuron 现在完整了

之前:

Inputs
↓
Weights
↓
Sum
↓
Bias
↓
Output

现在:

Inputs
↓
Weights
↓
Weighted Sum
↓
Bias
↓
 $z = wx + b$
↓
Activation Function
↓
 $a = \text{ReLU}(z)$
↓
Output

数学:

$$a = \text{ReLU}(wx + b)$$

15. 手算一次带 ReLU 的 Neuron

假设:

$$x_1 = 2$$

$$x_2 = 3$$

Weights:

$$w_1 = 0.5$$

$$w_2 = -0.8$$

Bias:

$$b = 0.2$$

先计算:

$$z = w_1x_1 + w_2x_2 + b$$

所以：

$$z = (0.5)(2) + (-0.8)(3) + 0.2$$

$$= 1 - 2.4 + 0.2$$

$$= -1.2$$

然后：

$$a = \text{ReLU}(-1.2)$$

得到：

$$a = 0$$

这个 neuron 在当前 input 下没有输出正信号。

16. 如果改变 Weight 呢？

例如把：

$$w_2 = -0.8$$

改成：

$$w_2 = 0.2$$

重新计算：

$$z = (0.5)(2) + (0.2)(3) + 0.2$$

$$= 1 + 0.6 + 0.2$$

$$= 1.8$$

ReLU：

$$\text{ReLU}(1.8) = 1.8$$

所以：

$$a = 1.8$$

这说明一个非常重要的事情：

Weights 决定 neuron 对输入如何响应。

而 Training 的工作，就是寻找这些 weights。

这和 Week 2 完全一样。

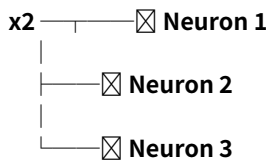
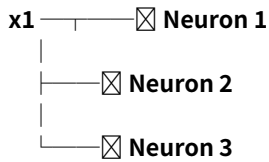
17. 一个 Neuron 能做什么？

一个 neuron 能表达的东西仍然有限。

真正强大的地方来自：

很多 neurons 一起工作。

例如：



这一组 neurons 就叫：

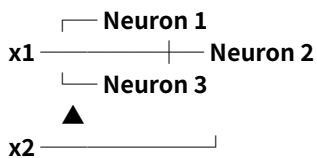
Layer

18. 什么是 Layer?

假设 Input:

$$x = [x_1, x_2]$$

Hidden Layer 有三个 neurons:



每个 neuron 都有自己的 weights。

Neuron 1:

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

Neuron 2:

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

Neuron 3:

$$z_3 = w_{31}x_1 + w_{32}x_2 + b_3$$

然后分别经过 ReLU:

$$a_1 = \text{ReLU}(z_1)$$

$$a_2 = \text{ReLU}(z_2)$$

$$a_3 = \text{ReLU}(z_3)$$

于是这一层输出：

$$[a_1 \quad a_2 \quad a_3]$$

19. 为什么一个 Layer 要有很多 Neurons?

因为不同 neuron 可以学习不同 pattern。

为了建立直觉，假设是图片识别。

一些 neuron 可能逐渐对：

竖线

敏感。

另一些：

横线

另一些：

边缘

更高层可能组合成：

眼睛

耳朵

轮廓

再组合：

猫脸

这不是程序员手工写：

```
if has_whiskers and has_ears:
    return "cat"
```

而是模型通过训练自己调整 weights。

20. Hidden Layer 是什么?

典型 Neural Network:

Input Layer

↓

Hidden Layer 1

↓

Hidden Layer 2

↓

Output Layer

为什么叫 Hidden?

因为这些层既不是原始输入，也不是最终答案。

它们是模型内部的：

intermediate representations

中间表示。

21. Feature Learning

这是 Neural Network 相比传统方法非常重要的地方。

例如输入一张猫的图片。

最原始的数据可能只是：

pixel
pixel
pixel
pixel
...

模型逐层转换：

Pixels
↓
Edges
↓
Shapes
↓
Parts
↓
Object concepts
↓
Cat

这里有个很重要的 AI 概念：

Representation

模型内部不断把数据转换成更有用的表示。

以后学习：

- Embedding
- Attention
- Transformer

会不断遇到这个思想。

22. Deep Learning 的 Deep 是什么意思？

假设：

Input
↓
Hidden
↓
Output

层很少。

如果：

Input
↓
Layer 1
↓
Layer 2
↓
Layer 3
↓

Layer 4

↓

Layer 5

↓

...

↓

Output

有很多层。

这就是所谓：

Deep Neural Network

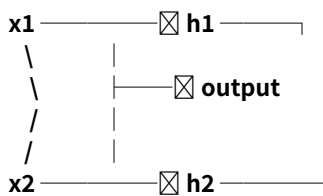
所以：

Deep Learning 的 Deep，主要就是网络有很多层。

并不是” AI 思考得很深”。

23. 现在手算一个小型 Neural Network

我们做一个非常小的网络：



两个 inputs：

$$x_1 = 1$$

$$x_2 = 2$$

Hidden Layer 两个 neurons。

24. Hidden Neuron 1

Weights:

$$w_{11} = 0.5$$

$$w_{12} = 0.4$$

Bias:

$$b_1 = 0.1$$

计算：

$$z_1 = 0.5(1) + 0.4(2) + 0.1$$

$$= 0.5 + 0.8 + 0.1$$

$$= 1.4$$

经过 ReLU:

$$h_1 = \mathbf{ReLU}(1.4)$$

所以:

$$h_1 = 1.4$$

25. Hidden Neuron 2

Weights:

$$w_{21} = -0.3$$

$$w_{22} = 0.8$$

Bias:

$$b_2 = -0.2$$

计算:

$$z_2 = (-0.3)(1) + (0.8)(2) - 0.2$$

$$= -0.3 + 1.6 - 0.2$$

$$= 1.1$$

ReLU:

$$h_2 = \mathbf{ReLU}(1.1)$$

所以:

$$h_2 = 1.1$$

Hidden Layer 输出:

$$[1.4 \quad 1.1]$$

26. 注意这里发生了什么

原始输入：

$$\begin{bmatrix} 1 & 2 \end{bmatrix}$$

经过 Hidden Layer：

$$\begin{bmatrix} 1.4 & 1.1 \end{bmatrix}$$

也就是说：

Original representation

$[1, 2]$

↓ Neural Network

New representation

$[1.4, 1.1]$

这就是刚才说的：

Representation Transformation

后面 Transformer 其实也一直在做这种事情。

27. Output Layer

现在 output neuron 接收：

$h_1 = 1.4$

$h_2 = 1.1$

假设：

$$v_1 = 0.7$$

$$v_2 = 0.2$$

Bias：

$$b = 0.1$$

计算：

$$y' = 0.7h_1 + 0.2h_2 + 0.1$$

代入：

$$= 0.7(1.4) + 0.2(1.1) + 0.1$$

$$= 0.98 + 0.22 + 0.1$$

$$y' = 1.30$$

这就是一次完整的：

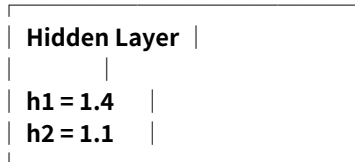
Forward Pass

28. Forward Pass 完整过程

刚才实际上做的是：

$x_1 = 1$

$x_2 = 2$



这就是：

Input → 一层一层计算 → Prediction

29. Prediction 出来了，然后呢？

这里马上重新连接 Week 2。

假设正确答案：

$$y = 2$$

模型预测：

$$\hat{y} = 1.3$$

Error：

$$1.3 - 2 = -0.7$$

如果使用 squared error：

$$\text{Loss} = (1.3 - 2)^2$$

$$= (-0.7)^2$$

$$0.49$$

然后呢？

和 Week 2 完全一样：

Prediction
↓
Loss
↓
Gradient
↓
Update Weights

只不过现在 parameters 不再只有：

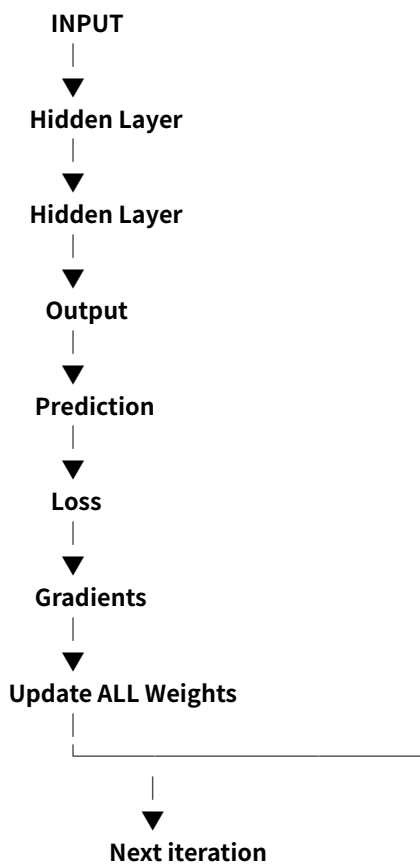
w
b

而是：

w11
w12
w21
w22
b1
b2
v1
v2
b3
...

30. 这就是 Neural Network Training

完整流程：



你会发现 Week 2 的核心完全没变：

Prediction → Loss → Gradient → Update

31. 那么问题来了：这么多 Weight 怎么求 Gradient?

Week 2:

$$\hat{y} = wx + b$$

只有：

w
b

求导还比较简单。

但 Neural Network:

x
↓
w1
↓
Neuron
↓
w2
↓
Neuron
↓
w3
↓
Output
↓
Loss

我们需要知道：

$$\frac{\partial \text{Loss}}{\partial w_1}$$

$$\frac{\partial \text{Loss}}{\partial w_2}$$

$$\frac{\partial \text{Loss}}{\partial w_3}$$

甚至几百万、几十亿个参数。

怎么做？

答案就是：

Backpropagation

这正是 Week 4 的核心。

32. 在进入 Backprop 前，先理解 Computational Graph

假设：

$$x = 2$$

$$w = 3$$

先：

$$z = xw$$

再：

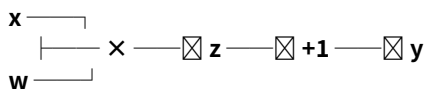
$$y = z + 1$$

程序：

```
z = x * w
```

```
y = z + 1
```

可以画成：



这叫：

Computational Graph

计算图。

Neural Network 本质上就是一个非常巨大的 computational graph。

33. 为什么 Computational Graph 很重要？

因为训练时：

Forward

沿着图向前：

```

Input
↓
Calculation
↓
Calculation
↓
Prediction
↓
Loss
  
```

Backward

从 Loss 反过来：

```

Loss
↑
gradient
↑
gradient
↑
gradient
↑
weights
  
```

利用 Week 2 已经见过的：

Chain Rule

一层一层计算 gradient。

这就是：

Backpropagation = 在 computational graph 上利用 chain rule 从后往前传播 gradients。

Week 4 我们会真正手算。

34. Matrix 为什么会出现？

目前我们一直写：

$$w_1x_1 + w_2x_2 + w_3x_3$$

但如果：

1000 inputs

1000 neurons

难道写：

neuron1 = $x_1 \cdot w_{11} + x_2 \cdot w_{12} + \dots$

neuron2 = $x_1 \cdot w_{21} + x_2 \cdot w_{22} + \dots$

...

当然不会。

我们会把 inputs 写成：

$$X$$

把 weights 写成：

$$W$$

于是整层计算：

$$Z = XW + b$$

然后：

$$A = \text{ReLU}(Z)$$

这就是以后 PyTorch 里看到的東西。

35. 不要被 Matrix 吓到

例如：

$$X = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

Weights:

$$W = \begin{bmatrix} 0.5 & -0.3 \\ 0.4 & 0.8 \end{bmatrix}$$

实际上只是把我们刚才：

Neuron 1:

$1 \times 0.5 + 2 \times 0.4$

Neuron 2:

$1 \times (-0.3) + 2 \times 0.8$

一次性计算。

所以 Matrix Multiplication 在 Neural Network 中并不是为了故意搞复杂。

它解决的是：

一次计算大量 neurons。

36. 用程序员角度理解 Matrix

你可以暂时把：

$$XW$$

理解成一种高效版本的：

for neuron in neurons:

 result = 0

 for input in inputs:

 result += input * weight

GPU 特别擅长这种大规模 matrix multiplication。

这也是为什么：

GPU 对 AI 如此重要。

37. 用纯 Python 实现一个 Neuron

不使用 PyTorch:

def relu(x):

 return max(0, x)

def neuron(inputs, weights, bias):

 total = 0

 for x, w in zip(inputs, weights):

 total += x * w

 total += bias

 return relu(total)

inputs = [1, 2]

```
weights = [0.5, 0.4]
```

```
bias = 0.1
```

```
output = neuron(inputs, weights, bias)
```

```
print(output)
```

得到：

1.4

和前面的手算一致。

38. 实现一个 Layer

现在多个 neurons：

```
def relu(x):  
    return max(0, x)
```

```
def neuron(inputs, weights, bias):
```

```
    total = sum(  
        x * w  
        for x, w in zip(inputs, weights)  
    )
```

```
    return relu(total + bias)
```

```
def layer(inputs, weights, biases):
```

```
    outputs = []
```

```
    for neuron_weights, bias in zip(weights, biases):
```

```
        output = neuron(  
            inputs,  
            neuron_weights,  
            bias  
        )
```

```
        outputs.append(output)
```

```
    return outputs
```

定义两个 neurons：

```
inputs = [1, 2]
```

```
weights = [  
    [0.5, 0.4],  
    [-0.3, 0.8]  
]
```

```
biases = [  
    0.1,  
    -0.2  
]
```

```
hidden = layer(  
    inputs,  
    weights,  
    biases  
)
```

```
print(hidden)
```

得到：

```
[1.4, 1.1]
```

这正是我们前面的手算。

39. 再连接 Output Layer

```
output_weights = [  
    [0.7, 0.2]  
]
```

```
output_biases = [  
    0.1  
]
```

```
prediction = layer(  
    hidden,  
    output_weights,  
    output_biases  
)
```

```
print(prediction)
```

概念上：

```
[1, 2]
```

↓

Hidden Layer

↓

```
[1.4, 1.1]
```

↓

Output Layer

↓

Prediction

我们已经从零写出了一个非常小的：

Neural Network

虽然它还不会 training。

Training 正是 Week 4 要解决的问题。

40. Neural Network 到底”学”的是什么？

这是必须彻底理解的问题。

很多人会说：

AI 学到了猫是什么。

这个说法从高层没问题。

但底层到底发生了什么？

实际上训练过程中真正被修改的是：

$$W$$

和：

$$b$$

也就是大量：

Parameters

例如：

0.12531
-0.88211
0.00341
1.27342
...

模型最终可能有几十亿这样的数字。

所谓：

模型学会了知识

从底层看，很大程度上意味着：

Training 找到了一组非常复杂的 parameter values。

41. 为什么 Parameter 越多能力可能越强？

Linear Regression：

2 parameters

w
 b

小 Neural Network：

几百
几千 parameters

现代 Deep Learning：

Millions
Billions

更多 parameters 意味着模型有更大的 capacity 去表示复杂关系。

但注意：

Parameters 多 $\neq$ 自动更聪明。

还需要：

- 足够的数据
- 合适的 architecture
- 合适的 training
- optimization
- regularization
- compute

等等。

42. Parameters 是怎么数出来的？

这是一个很实用的问题。

假设一层：

Input size = 3

Neuron count = 4

每个 neuron：

3 weights

1 bias

所以：

4 parameters per neuron

一共：

4 neurons $\times$ 4

= 16 parameters

公式：

$$\text{Parameters} = (\text{Inputs} \times \text{Neurons}) + \text{Neurons}$$

也就是：

$$3 \times 4 + 4 = 16$$

后面的 +4 就是四个 bias。

43. 再举一个例子

假设：

Input features = 100

Hidden neurons = 256

Weights：

$$100 \times 256 = 25,600$$

Bias：

$$256$$

总参数：

$$25,600 + 256$$

$$25,856$$

仅仅一层就有两万多个 parameters。

这也是为什么我们不可能手动调整 weights。

必须：

Loss
↓
Gradient
↓
Optimizer
↓
Automatic update

44. Neural Network 和 Linear Regression 的真正关系

现在把 Week 2 和 Week 3 并排看。

Linear Regression

$$\hat{y} = wx + b$$

Input
↓
Linear
↓
Prediction

Neural Network

$$h = \text{ReLU}(W_1x + b_1)$$

然后：

$$\hat{y} = W_2h + b_2$$

也就是：

Input
↓
Linear
↓
Activation
↓
Linear
↓
Prediction

再复杂：

Input
↓
Linear
↓
ReLU
↓
Linear
↓
ReLU
↓
Linear
↓
Output

所以 Neural Network 并没有突然出现某种魔法。
它只是把：

$$wx + b$$

大量组合起来，并在中间加入：

Activation

45. 一个非常重要的公式

Week 3 最值得记住：

$$a = f(Wx + b)$$

拆开：

x

Input / Feature

W

Weights

b

Bias

Wx+b

Linear Transformation

f

Activation Function

a

这一层的 output / activation

这一个公式会一路跟着你进入：

Deep Learning → Transformer → GPT。

46. ReLU 不是唯一 Activation

你以后还会看到：

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

输出范围：

$$0 \sim 1$$

所以很容易被理解成 probability。

Tanh

输出：

$$-1 \sim 1$$

ReLU

$$\max(0, x)$$

GELU

Transformer 中非常常见。

GPT 系列 architecture 中你会遇到它或其变体。

现在不用学公式。

只需要先知道：

Activation Function 的核心作用之一是引入 Non-linearity。

47. 为什么 Sigmoid 长这样？

公式：

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

目前不要试图背。

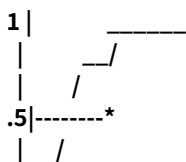
只看输入输出：

x 很负
→ 接近 0

x = 0
→ 0.5

x 很正
→ 接近 1

图形：



$$\begin{array}{c|c} 1 & _ \\ 0 & _ \\ + & \text{-----} x \end{array}$$

因此它非常适合表达：

0 → unlikely

1 → likely

例如 binary classification 的输出。

48. Classification 和 Regression

Week 2 主要是：

Regression

预测连续数值：

房价 = \$812,532

温度 = 27.3

销量 = 1052

Neural Network 还可以做：

Classification

例如：

Cat

Dog

Bird

或者：

Spam

Not Spam

或者：

Fraud

Not Fraud

模型 architecture 类似。

主要变化之一是：

Output 和 Loss Function。

49. Loss Function 也会改变

Week 2 使用：

MSE

它很适合 Regression。

但 Classification 常见：

Cross Entropy Loss

以后我们会专门解释。

现在只需要建立：

Regression
→ often MSE

Classification
→ often Cross Entropy

到了 GPT：

Next Token Prediction
→ Classification over vocabulary
→ Cross Entropy

所以你以后会再次遇到它。

50. 一个容易犯的错误：Neuron 是不是一个 if?

不是。

Neuron 不会直接写：

```
if image_has_ears:
    cat_score += 1
```

它做的是：

$$a = f(Wx + b)$$

Training 不断调整 W 和 b。

最终整个网络形成复杂行为。

所以它不是传统：

Rule-based Programming

51. Traditional Programming vs Machine Learning

传统程序：

```
Data
+
Rules written by programmer
↓
Output
```

例如：

```
if age >= 18:
    allow()
```

Machine Learning:

```
Data
+
Correct Answers
↓
Training
↓
Learn Parameters
```

↓
Model

然后：

New Data

+

Model

↓

Prediction

这个区别非常重要。

52. 那么 GPT 里的 Neuron 也是这些东西吗？

底层依然大量使用：

$$Wx + b$$

但 Transformer 的 architecture 要复杂很多。

例如：

Token

↓

Embedding

↓

Attention

↓

Linear

↓

Activation

↓

Linear

↓

...

其中仍然充满：

Matrix multiplication

Weights

Bias

Activation

所以你现在学的不是”一个过时的小 Neural Network”。

你是在学：

GPT 内部最基础的数学 building blocks。

53. 从 Week 2 到 GPT 的完整地图

现在你已经走到这里：

Week 2

Linear Regression

|

$$y = wx + b$$

▼

Parameters

|

▼

Prediction



Loss



Gradient Descent



Week 3

Neuron



$$a = f(Wx+b)$$



Activation



Layer



Multiple Layers



Neural Network



Week 4

Backpropagation



PyTorch



Embedding



Attention



Transformer



GPT

这条路线是连续的。

54. Week 3 最应该理解的 7 件事

这一周不要追求背很多公式。

真正应该掌握：

1. Neuron 本质上还是 $Wx+b$ 。
2. Weight 决定输入怎样影响 neuron。
3. Bias 给 neuron 一个额外可学习的偏移。
4. 只堆 Linear Layer 没用，因为整体仍然 Linear。
5. Activation Function 引入 Non-linearity。

6. 多个 Neurons 组成 Layer，多层组成 Neural Network。
7. Neural Network 学习，本质仍然是调整 Parameters 来降低 Loss。

55. Week 3 三个最重要公式

① Linear Calculation

$$z = Wx + b$$

意思：

Input
×
Weights
+
Bias

② Activation

$$a = f(z)$$

例如：

$$a = \text{ReLU}(z)$$

③ 完整 Neuron

合起来：

$$a = f(Wx + b)$$

如果你看到这个公式能够直接在脑子里转换成：

```
z = weighted_sum(inputs, weights) + bias  
a = activation(z)
```

Week 3 的核心已经掌握一大半。

56. 最后做一个理解测试

假设 neuron：

$$z = w_1x_1 + w_2x_2 + b$$

其中：

x1 = 2
x2 = 3

w1 = 0.5
w2 = -0.2

b = 0.1

那么：

$$\begin{aligned} z &= (2)(0.5) + (3)(-0.2) + 0.1 \\ &= 1 - 0.6 + 0.1 \\ &= 0.5 \end{aligned}$$

使用：

$$\text{ReLU}(z)$$

所以：

$$a = 0.5$$

如果把：

`w2 = -1`

那么：

$$\begin{aligned} z &= 2(0.5) + 3(-1) + 0.1 \\ &= 1 - 3 + 0.1 \\ &= -1.9 \end{aligned}$$

经过 ReLU：

$$\text{ReLU}(-1.9) = 0$$

所以 neuron output：

$$0$$

如果这个过程你能跟下来，那么你已经理解了一个最基础 Neural Network neuron 的 Forward Pass。

57. 用 Software Engineer 的视角总结 Week 3

可以把一个 neuron 看成：

```
class Neuron:
```

```
    def __init__(self):
        self.weights = ...
        self.bias = ...
```

```
    def forward(self, inputs):
```

```
        z = dot(inputs, self.weights) + self.bias
```

```
        return activation(z)
```

一个 Layer:

```
class Layer:
```

```
    neurons = [
        Neuron(),
        Neuron(),
        Neuron()
    ]
```

一个 Neural Network:

```
class NeuralNetwork:
```

```
    layers = [
        Layer(),
        Layer(),
        Layer()
    ]
```

Forward:

```
x = input
```

```
for layer in layers:
    x = layer.forward(x)
```

```
prediction = x
```

Training:

```
for epoch in range(...):
```

```
    prediction = model.forward(input)
```

```
    loss = loss_function(
        prediction,
        target
    )
```

```
    gradients = backward(loss)
```

```
    update_parameters(gradients)
```

如果用这种角度看:

Neural Network 就是一个巨大的、参数化的 Computational Graph。

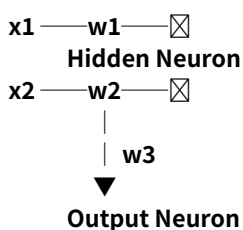
Training 则是在做:

不断修改 graph 中的 parameters, 让最终 Loss 越来越低。

58. Week 3 → Week 4: 最关键的问题

现在其实留下了一个巨大的未解决问题。

我们的网络:





如果最终：

Loss = 10

我们知道模型错了。

但：

到底应该修改 w_1 多少？

w_2 应该增加还是减少？

w_3 对最终错误负多少责任？

如果网络有 100 层，最前面的 weight 怎么知道自己应该怎么改？

这就是 Week 4 的核心：

Backpropagation

它会把 Week 2 的 **Derivative + Partial Derivative + Chain Rule + Gradient Descent** 全部串起来。

最终你会看到：

Loss → Gradient → Backward → Every Weight → Update

Week 2 教的是机器如何学习一个参数；Week 3 教的是如何把大量参数组织成 Neural Network；Week 4 则要解决——如何让这个网络里的所有参数一起学习。

Week 3 补充深入理解：为什么多层 Neuron 能表达更多模式

本节保留邮件版 Week 3 原文不变，只补充几个读完原文后最值得追问的“为什么”。

1. Neuron 的输入、参数和输出分别是什么

一个 neuron 可以写成：

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

这里：

- x_i 是当前 example 提供的 feature；
- w_i 是模型要学习的影响倍率；
- b 是移动触发边界的偏移；
- z 是 activation 之前的 raw score；
- $\text{activation}(z)$ 是 neuron 对这个 score 的非线性响应。

Neuron 存在的作用，是把多个输入证据压成一个可学习的响应。很多 neurons 并排后，每个 neuron 可以学习不同的证据组合。

2. 为什么 Weighted Sum 后还要 Activation

如果整张网络只有 Linear operations：

$$W_2(W_1x + b_1) + b_2$$

仍可合并成一个更大的 Linear operation。多层代码看起来更深，表达能力却没有真正增加。

Activation 打破这种可合并性，让不同输入区域可以走不同响应曲线。ReLU 通过门控正值，Sigmoid 把 raw score 映射成 0 到 1 的概率式输出。

3. Hidden Layer 为什么有用

Output neuron 直接从原始 features 组合，通常只能学一个相对简单的边界。Hidden layer 先产生中间 features，例如：

hidden 1 → “面积大且距离近”
hidden 2 → “卧室多但价格敏感”

这些不是程序员手写的标签，而是训练后 weights 形成的可用方向。下一层再组合这些中间 signals，就能表达原始输入空间中更复杂的区域。

4. XOR 为什么是经典例子

XOR 在输入 (0,0) 和 (1,1) 时输出 0，在 (0,1) 和 (1,0) 时输出 1。单个 linear neuron 无法用一条直线把这两组点分开。

两个 hidden neurons 可以先各自形成不同的半平面，再由 output neuron 组合它们。这个例子不是因为现代 GPT 仍在手工解决 XOR，而是因为它最小地说明：非线性和层叠会改变可表达的函数集合。

5. Classification 的输出为什么不同

Regression 通常输出一个连续数，例如房价；Binary classification 常输出一个 logit，再通过 Sigmoid 得到一个 probability；Multi-class classification 输出每个 class 一个 logit，再通过 Softmax 得到整组 probabilities。

共同结构仍是：

input → network → raw output → task-specific loss

区别在于 target 的形式和最后如何解释 output，不是“神经网络训练换了另一套原理”。

6. Parameter Count 为什么重要

如果一层有 n 个 inputs 和 m 个 neurons：

$$weights = n \times m$$

再加 m 个 biases。Parameter count 越大，模型有更多可调自由度，但也需要更多 data、memory 和 training signal。容量增加可能帮助 underfitting，也可能在数据不足时带来 overfitting。

7. 从 Forward 到 Learning 的连接

邮件版中的 forward 只产生 prediction；它还没有告诉 weights 应该怎样改变。Week 4 会把同一张 network graph 反向走一遍：

Loss
↓ local derivatives + Chain Rule
every weight and bias 的 gradient
↓ optimizer
updated parameters

所以 Week 3 的最后一句“这个网络还不会学习”非常关键：Architecture 决定模型能表达什么，Back-propagation 才提供让它根据 data 改善的机制。

Week 4 - Backpropagation: 让网络里的所有参数一起学习

Week 4 核心目标

Week 2 已经解释了一个参数怎样通过 Gradient Descent 学习。

Week 3 把一个参数扩展成大量 neurons、layers、weights 和 biases，并在最后留下四个问题：

w_1 应该修改多少？

w_2 应该增加还是减少？

某个 weight 对最终错误负多少责任？

很前面的 weight 怎样收到 Loss 的反馈？

Backpropagation（反向传播）就是答案。

本周只需要真正理解四件事：

1. Computational Graph;
2. Local Derivative;
3. Chain Rule;
4. Gradient 怎样从 Loss 一层层传回每个 parameter。

核心一句话：

Backpropagation 从 Loss 开始，沿计算依赖关系反向移动，用 Chain Rule 计算每个 parameter 对 Loss 的影响。

1. Backpropagation 不是新的 Training 方法

完整训练过程仍然是：

```
Forward Pass
↓
Prediction
↓
Loss
↓
Backpropagation
↓
Gradients
↓
Optimizer / Gradient Descent
↓
Updated Parameters
```

其中：

- Backpropagation 负责算 gradients;
- Gradient Descent 或 optimizer 负责用 gradients 更新 parameters。

不要把二者混成一件事。

2. Computational Graph

假设一个很小的网络：

$$z = w_1 x + b_1$$

$$h = \text{ReLU}(z)$$

$$\hat{y} = w_2 h + b_2$$

$$L = (\hat{y} - y)^2$$

依赖关系是：

```

x, w1, b1
↓
z
↓ ReLU
h
↓ with w2, b2
ŷ
↓ compare with y
L

```

这就是 Computational Graph：把一个大公式拆成一组小操作，并记录谁依赖谁。

对程序员来说，它像运行时建立的一张 dependency graph。

为什么要把公式拆成 Graph

如果直接把整个网络写成一个巨大公式，理论上也可以一次求导，但工程上几乎不可维护。真实网络里同一个 tensor 可能被多个 layer 使用，一个 parameter 也可能通过多条路径影响 Loss。Computational Graph 把复杂问题变成一组统一的小问题：

```

这个 operation 收到哪些 inputs?
它产生哪个 output?
它的 output 被谁继续使用?
它的 local derivative 是什么?

```

这很像编译器的中间表示或服务之间的 dependency graph：每个节点只描述自己的行为，图负责保存整体依赖关系。这样无论网络有十个操作还是十亿个操作，Backward 使用的仍然是同一套遍历规则。

Graph 中真正保存的是什么

Graph 不只是保存“调用顺序”，还要保存 backward 需要的上下文。例如乘法：

$$c = ab$$

Backward 时需要知道 forward 当时的 a 和 b，因为：

$$\frac{\partial c}{\partial a} = b, \quad \frac{\partial c}{\partial b} = a$$

因此，forward 中保存中间结果并不是浪费内存，而是在用内存换取 backward 所需的信息。这也是训练通常比纯 inference 占用更多显存的重要原因之一。

3. Forward Pass：先保存中间结果

设定：

```

x = 2
y = 5
w1 = 1
b1 = 0
w2 = 1
b2 = 0

```

Forward:

$$z = w_1x + b_1 = 1 \times 2 + 0 = 2$$

$$h = \text{ReLU}(2) = 2$$

$$\hat{y} = w_2h + b_2 = 1 \times 2 + 0 = 2$$

$$L = (2 - 5)^2 = 9$$

需要保存的中间值包括 z 、 h 和 $\hat{y}$ ，因为 backward 时计算局部导数会再次使用它们。

Forward 不只是在“做预测”

Training 时的 forward 同时完成两件事：

1. 计算 prediction 和 loss;
2. 为之后的 backward 留下计算路径和必要的中间值。

Inference 只需要第一件事，所以通常可以关闭 gradient tracking。Training 不能随意丢掉所有中间结果，否则走到 backward 时就不知道 ReLU 当时输入是正还是负，也不知道乘法节点应该把哪个值乘到上游 gradient 上。

4. Local Derivative: 每个小操作只负责自己

Backprop 不会一次求完整大公式。每个节点只回答：

我的 output 稍微变化时，我的 input 会受到怎样的影响？

更准确地说，Local Derivative（局部导数）回答的是：

在这个小 operation 内，某个 input 稍微变化一点，它的 output 会变化多快？

“Local” 的意思是它不需要知道整个 network，也不需要知道最终 Loss 是多少。乘法节点只懂乘法，ReLU 节点只懂 ReLU。最终 Loss 对某个 parameter 的影响，由这些局部关系沿 graph 组合出来。

可以把每个节点想成一个实现统一接口的组件：

```
forward(inputs) → output
backward(upstream_gradient) → input_gradients
```

forward 产生值；backward 接收后面传来的影响，再结合自己的 local derivative，把影响继续传给前面的 inputs。

几个常见局部导数：

加法

$$c = a + b$$

$$\frac{\partial c}{\partial a} = 1, \quad \frac{\partial c}{\partial b} = 1$$

为什么都是 1？假设 $c = a + b$ ，固定 b ： a 增加 0.01， c 也正好增加 0.01。因此 output 对这个 input 的变化倍率是 1。加法节点不会放大或缩小 gradient，只会把收到的 gradient 原样分给两个 inputs。

乘法

$$c = ab$$

$$\frac{\partial c}{\partial a} = b, \quad \frac{\partial c}{\partial b} = a$$

例如 $c = a \times b$, forward 时 $a = 3$ 、 $b = 4$ 。如果固定 b , 让 a 增加 0.01:

原来 $c = 3 \times 4 = 12$

现在 $c = 3.01 \times 4 = 12.04$

a 增加 0.01, c 增加 0.04, 变化倍率是 4, 正好等于另一个 input b 。这就是乘法节点的 local derivative, 不是需要死记的神秘规则。

平方误差

$$L = (\hat{y} - y)^2$$

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

这里的导数同时编码了方向和严重程度:

- 如果 prediction 高于 target, $\hat{y} - y$ 为正, gradient 为正;
- 如果 prediction 低于 target, $\hat{y} - y$ 为负, gradient 为负;
- 错得越远, gradient 的绝对值越大。

因此平方误差不仅告诉模型“错了多少”, 它的导数还告诉模型“prediction 应该往哪边移动”。

ReLU

$$ReLU(z) = \max(0, z)$$

$$ReLU'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$$

在 $z = 0$ 的特殊点, 框架会选择一个约定值; 当前不影响我们的核心理解。

ReLU 的 local derivative 像一个 gradient gate:

forward 时 $z > 0 \rightarrow$ gate 打开 $\rightarrow$ gradient 乘 1, 继续通过

forward 时 $z < 0 \rightarrow$ gate 关闭 $\rightarrow$ gradient 乘 0, 这条路径停止

注意, ReLU 并不是凭空“生成” gradient。它只是根据 forward 时的输入, 决定收到的上游 gradient 有多少可以继续传回去。

Local Derivative 怎样真正参与 Backward

一个节点在 backward 时执行的核心规则是:

$$input\ gradient = upstream\ gradient \times local\ derivative$$

例如乘法节点 $c = ab$ 收到:

upstream gradient = $\partial L / \partial c = -6$

forward 时 $a = 3$

forward 时 $b = 4$

那么它传给两个 inputs 的 gradients 是：

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \frac{\partial c}{\partial a} = (-6)(4) = -24$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial c} \frac{\partial c}{\partial b} = (-6)(3) = -18$$

这就是截图中几条 local derivative 公式真正的用途：它们不是孤立的求导表，而是每个节点把“Loss 的责任”继续向前传递时使用的倍率。

5. Chain Rule：把局部影响串起来

如果：

$w_1 \rightarrow z \rightarrow h \rightarrow \hat{y} \rightarrow \text{Loss}$

那么：

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial z} \frac{\partial z}{\partial w_1}$$

每一项只是相邻两个节点之间的局部影响。

为什么相乘？因为：

w_1 改一点
↓ 影响 z
 z 再影响 h
↓
 h 再影响 prediction
↓
prediction 再影响 Loss

最终影响等于每一段影响的乘积。

为什么是相乘，而不是相加

假设一个很小的变化经过三段：

w_1 增加 0.01
 z 大约增加 2×0.01
 h 大约增加 $1 \times z$ 的变化
 $\hat{y}$ 大约增加 $3 \times h$ 的变化

最终：

$$\Delta \hat{y} \approx 0.01 \times 2 \times 1 \times 3 = 0.06$$

每一段都是上一段变化的倍率，所以倍率自然连续相乘。Chain Rule 只是把这件事写成精确的导数形式。

用程序员熟悉的 Dependency Chain 理解

如果：

$\text{Tax} = \text{Tax}(\text{Subtotal}(\text{Price}(\text{quantity}, \text{unitPrice})))$

unitPrice 改动对最终 Tax 的影响，必须依次经过 Price、Subtotal、Tax。任何一段完全不受输入影响，最终影响就会变成 0。Neural Network 的 backward 做的也是沿依赖链追踪影响，只是 graph 更大、分支更多。

Chain Rule 计算的是一阶导数

Backpropagation 主要计算 Loss 对 parameters 的一阶导数。它不需要先计算二阶导数：一阶 gradient 已经能告诉 optimizer 哪个方向会让 Loss 上升，因此取反方向就能下降。

二阶导数描述 Loss 曲面的弯曲程度，某些高级优化方法会使用，但完整神经网络的二阶导数会形成巨大的 Hessian matrix，成本很高。GPT 训练通常使用 SGD、AdamW 等一阶优化方法。

6. 从 Loss 开始反向计算

Forward 得到：

```
 $\hat{y} = 2$ 
 $y = 5$ 
 $L = 9$ 
```

Backward 的逻辑起点其实是：

$$\frac{\partial L}{\partial L} = 1$$

一个值对自己的变化率当然是 1。可以理解成在 Loss 节点放入一个初始信号 1，然后让它通过每个节点的 local derivative 向前传播。

为什么最终通常要求 Loss 是 scalar？因为 scalar 只有一个明确的起点。如果 output 是一个 vector，就需要先说明我们想对这个 vector 做怎样的加权组合，才能决定 backward 的初始方向。训练中把一个 batch 的各项损失求和或求平均，正是为了得到一个 scalar objective。

接下来第一步是计算 Loss 对 prediction 的影响。

第一步：

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = 2(2 - 5) = -6$$

这个 -6 是从 Loss 向前一个节点传出的 gradient。

它表示：在当前位置稍微增加 prediction，会让 Loss 下降。

7. Output Layer 的 gradients

Output：

$$\hat{y} = w_2 h + b_2$$

对 w_2 ：

$$\frac{\partial \hat{y}}{\partial w_2} = h = 2$$

Chain Rule：

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_2} = (-6)(2) = -12$$

对 b_2 ：

$$\frac{\partial \hat{y}}{\partial b_2} = 1$$

所以：

$$\frac{\partial L}{\partial b_2} = (-6)(1) = -6$$

现在 Output Layer 的两个 parameters 都知道应该怎样改变。

8. 把 Gradient 继续传给 Hidden Layer

为了继续向前传，需要先计算 Loss 对 h 的影响：

$$\frac{\partial \hat{y}}{\partial h} = w_2 = 1$$

所以：

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} = (-6)(1) = -6$$

这个 gradient 不是 parameter gradient。它是一条继续向前传播的“责任信号”。

9. Gradient 经过 ReLU

Forward 时：

$z = 2 > 0$

因此：

$$\frac{\partial h}{\partial z} = 1$$

所以：

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial h} \frac{\partial h}{\partial z} = (-6)(1) = -6$$

如果 Forward 时 $z < 0$ ，ReLU 的局部导数为 0，这条路径的 gradient 就会被挡住。

10. Hidden Layer 的 gradients

Hidden linear calculation：

$$z = w_1 x + b_1$$

对 w_1 ：

$$\frac{\partial z}{\partial w_1} = x = 2$$

所以：

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial w_1} = (-6)(2) = -12$$

对 b_1 :

$$\frac{\partial z}{\partial b_1} = 1$$

$$\frac{\partial L}{\partial b_1} = (-6)(1) = -6$$

最终:

```
gradient_w1 = -12
gradient_b1 = -6
gradient_w2 = -12
gradient_b2 = -6
```

所有 parameters 都收到了来自同一个 Loss 的反馈。

11. 更新 Parameters

设 learning rate:

```
learning_rate = 0.01
```

更新公式仍然来自 Week 2:

$$parameter_{new} = parameter_{old} - learningRate \times gradient$$

因此:

```
w1 = 1 - 0.01 * (-12) = 1.12
b1 = 0 - 0.01 * (-6) = 0.06
w2 = 1 - 0.01 * (-12) = 1.12
b2 = 0 - 0.01 * (-6) = 0.06
```

12. 再做一次 Forward

新参数:

$$z = (1.12)(2) + 0.06 = 2.30$$

$$h = ReLU(2.30) = 2.30$$

$$\hat{y} = (1.12)(2.30) + 0.06 = 2.636$$

$$L = (2.636 - 5)^2 \approx 5.59$$

对比:

```
Before: Loss = 9.00
After: Loss ≈ 5.59
```

一次 backward + update 后, 模型已经向正确答案移动。

13. 用代码对应完整计算

```

x = 2.0
y = 5.0

w1, b1 = 1.0, 0.0
w2, b2 = 1.0, 0.0

learning_rate = 0.01

# Forward
z = w1 * x + b1
h = max(0.0, z)
prediction = w2 * h + b2
loss = (prediction - y) ** 2

# Backward: Loss → prediction
d_prediction = 2 * (prediction - y)

# Output layer
dw2 = d_prediction * h
db2 = d_prediction
d_h = d_prediction * w2

# ReLU
d_z = d_h * (1.0 if z > 0 else 0.0)

# Hidden layer
dw1 = d_z * x
db1 = d_z

# Update
w1 -= learning_rate * dw1
b1 -= learning_rate * db1
w2 -= learning_rate * dw2
b2 -= learning_rate * db2

```

每一行都能对应回计算图中的一条边。

14. 如果一个 Parameter 有多条路径

在真实网络中，一个 parameter 或 intermediate value 可能通过多条路径影响 Loss。

规则是：

同一个变量从不同路径收到的 gradients 要相加。

例如：

$$L = L_1 + L_2$$

那么：

$$\frac{\partial L}{\partial w} = \frac{\partial L_1}{\partial w} + \frac{\partial L_2}{\partial w}$$

这和一个 shared service 被多个调用路径影响的情况很像：总影响必须汇总。

为什么这里是相加？因为每条路径都造成了一部分真实影响。忽略任何一条，就会低估该变量对 Loss 的总责任。

可以记住 graph 的两条规则：

沿同一条连续路径：local derivatives 相乘

多条路径汇合到同一变量：path gradients 相加

“路径上相乘，分支处相加”几乎概括了 Backpropagation 的计算结构。

15. Batch 中为什么也要累加 Gradient

一个 batch 有多个 examples:

$$L = \frac{1}{n} \sum_{i=1}^n L_i$$

所以 parameter gradient 是每个 example 对它的影响的平均:

$$\frac{\partial L}{\partial w} = \frac{1}{n} \sum_{i=1}^n \frac{\partial L_i}{\partial w}$$

这解释了为什么训练代码经常对 batch loss 求 mean。

16. Autograd 做了什么

PyTorch 的 Autograd 自动完成三件事:

1. Forward 时记录运算和必要的中间值;
2. 从 scalar loss 开始反向遍历 graph;
3. 用每个 operation 的 local derivative 和 Chain Rule 累积 gradients。

它没有改变数学原理, 只是替我们执行大量重复计算。

Autograd 也不是保存一张永久的静态流程图。PyTorch 默认在每次 forward 时根据实际执行的 operations 动态建立 graph, 所以普通 Python 的 if、loop 和函数调用都可以参与模型计算。loss.backward() 完成后, graph 通常会被释放, 下一次 training step 再建立新的 graph。

这解释了三个常见现象:

- 改变本次 input 可能改变本次 graph;
- 不需要 gradient 的 inference 可以用 torch.no_grad() 节省内存;
- 如果错误地把旧 graph 跨 step 保留下来, 内存可能持续增长。

17. Vanishing 与 Exploding Gradient: 知道概念即可

当 gradient 穿过很多层时, 会连续相乘。

如果很多局部导数都小于 1, gradient 可能越来越小, 叫 Vanishing Gradient。

如果很多局部导数都大于 1, gradient 可能越来越大, 叫 Exploding Gradient。

后面会看到 ReLU、Residual Connection、LayerNorm 和 Gradient Clipping 等设计帮助训练稳定。现在不需要展开证明。

一个最小数字例子: 如果 gradient 连续经过 20 个 local derivative 0.5, 总倍率是:

$$0.5^{20} \approx 0.00000095$$

信号几乎消失。若连续经过 20 个 2, 总倍率则是:

$$2^{20} = 1,048,576$$

这说明问题不是“层数多”本身, 而是很多局部倍率连续相乘后可能变得极小或极大。后续 architecture 和 optimizer 中的稳定化设计, 很多都在处理这个后果。

Gradient、Derivative 与 Parameter Gradient 的关系

这三个词容易在代码里混在一起：

- Derivative 是一般概念：一个输出对一个输入的变化率；
- Local Derivative 是单个 operation 内部的变化率；
- Gradient 是 scalar Loss 对一组 variables 的所有偏导数组成的向量；
- Parameter Gradient 是其中针对可训练 parameters 的部分，例如 `w.grad`。

Intermediate tensor 也可能有 gradient，但 optimizer 通常只更新 parameters。Backpropagation 的任务，是从许多 local derivatives 组合出每个 parameter gradient。

18. Week 4 最应该理解的 6 件事

1. Forward 计算 prediction 和 loss，并保存中间值。
2. Backward 从 Loss 开始，沿 graph 反向移动。
3. 每个 operation 只提供自己的 local derivative。
4. Chain Rule 把局部影响相乘成最终影响。
5. 多条路径的 gradients 要相加。
6. Backprop 算 gradient；optimizer 使用 gradient 更新 parameter。

19. 理解测试

如果：

```
Loss → prediction 的 gradient = -4
prediction = w × h
h = 3
```

那么：

$$\frac{\partial \text{prediction}}{\partial w} = h = 3$$

因此：

$$\frac{\partial \text{Loss}}{\partial w} = (-4)(3) = -12$$

如果 learning rate 是 0.01：

```
w_new = w - 0.01 × (-12)
      = w + 0.12
```

20. Week 4 → Week 5

我们已经能手算一个小网络的 forward、backward 和 parameter update。

但真实网络可能有数百万甚至数十亿 parameters，人工维护计算图和导数不现实。

Week 5 会把同样的数学交给 PyTorch：

```
Python numbers
↓
Tensor
↓
Autograd
↓
nn.Module
↓
Optimizer
```

Week 5 - Tensor 与 PyTorch: 把手算交给计算机

Week 5 核心目标

Week 4 手算了:

Forward → Loss → Backward → Gradients → Update

PyTorch 不会替换这套原理。它只是提供更适合大规模计算的工具:

Tensor → 保存和批量计算数字

Autograd → 自动执行 Chain Rule

nn.Module → 组织 Parameters 和 Forward

Optimizer → 更新 Parameters

本周重点只有四个: Tensor、Shape、Autograd、Training Loop。

1. Tensor 到底是什么

Tensor 可以先理解为:

带有 shape、dtype、device 等信息的多维数字容器。

这个定义里的每一部分都有实际作用:

- 数字内容告诉 operation 要计算什么;
- shape 告诉 operation 数据怎样组织;
- dtype 决定每个数字怎样存储和可以参加哪些运算;
- device 决定运算在 CPU、GPU 或其他加速器上执行;
- autograd metadata 记录它是否需要 gradient 以及怎样回到前面的 operations。

import torch

```
scalar = torch.tensor(3.0)
vector = torch.tensor([1.0, 2.0, 3.0])
matrix = torch.tensor([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0],
])
```

对应 shape:

```
scalar: []
vector: [3]
matrix: [2, 3]
```

Tensor 不是一种新数学。它把 Week 1 的 scalar、vector 和 matrix 放进一个统一接口。

为什么 Python List 不够

Python list 可以保存数字,却不是为大规模数值计算设计的。对百万个元素逐个执行 Python loop,会产生大量解释器开销;list 也没有统一的 shape、device 和 automatic differentiation 语义。

Tensor 把同类数字排列成规则内存,并把整个 operation 交给经过优化的底层 kernel:

Python 发出一次 matrix multiplication 请求
底层并行计算大量乘加操作
Autograd 同时记录这次依赖关系

因此 Tensor 是数学、硬件加速和 Backpropagation 的共同数据载体。

2. Dimension 与 Shape

假设一个 batch 有四套房子,每套有三个 features:

```
x = torch.tensor([
    [0.8, 0.6, 0.2],
    [0.5, 0.4, 0.9],
    [0.9, 0.7, 0.1],
    [0.6, 0.8, 0.4],
])
```

```
print(x.shape) # torch.Size([4, 3])
```

dimension 0 = batch
dimension 1 = features

后面处理文本时常见：

[B, T, C]

- B: Batch size;
- T: Sequence length;
- C: Channel / Embedding dimension.

读 PyTorch 代码时，先追 shape，再追公式。

Axis 的语义不能靠猜

[B,T,C] 里的三个字母不是 PyTorch 强制规定的，只是项目中的约定。同一个 [4,8,32] tensor，如果某段代码认为 axis 1 是 token，而另一段认为 axis 1 是 channel，运算可能照样运行却产生错误含义。

建议在复杂代码旁写 shape trace：

```
# x: [batch, sequence, embedding]
x = token_embedding(token_ids)
```

它相当于给动态 tensor 增加可阅读的 type annotation。

3. Dtype 与 Device

Dtype

```
x = torch.tensor([1.0, 2.0], dtype=torch.float32)
token_ids = torch.tensor([3, 8, 2], dtype=torch.long)
```

神经网络计算通常使用浮点数；Embedding lookup 的 token IDs 通常使用整数 long。

为什么 token ID 要是整数？因为它用于选择 Embedding Matrix 的某一行，本质上是 index。为什么 weights 通常是浮点数？因为训练需要连续地做小幅 parameter update，整数无法表达这些细微变化。

Device

```
device = "cuda" if torch.cuda.is_available() else "cpu"
x = x.to(device)
```

参与同一个 operation 的 tensors 必须在兼容的 device 上。常见错误：model 在 GPU，input 还在 CPU。

把 tensor 移到 GPU 不是给它加一个标签，而是把真实数据复制到另一块内存。频繁在 CPU 和 GPU 之间搬运会有成本，所以常见做法是把 model、inputs、targets 长时间放在同一 device，只把最终统计值移回 CPU。

4. 用 Tensor 完成一个 Layer

公式：

$$Z = XW + b$$

假设：

X: [4, 3] 4 个 examples, 每个 3 个 features
 W: [3, 2] 3 个 inputs 连接到 2 个 neurons
 b: [2] 每个 neuron 一个 bias

输出:

Z: [4, 2]

代码:

```
X = torch.tensor([
    [0.8, 0.6, 0.2],
    [0.5, 0.4, 0.9],
    [0.9, 0.7, 0.1],
    [0.6, 0.8, 0.4],
])

W = torch.tensor([
    [0.7, 0.2],
    [0.5, -0.1],
    [-0.4, 0.8],
])

b = torch.tensor([0.1, -0.2])

Z = X @ W + b
print(Z.shape) # [4, 2]
```

@ 是 matrix multiplication。

一行代码实际完成了多少计算

$X @ W$ 中的四个 examples 会同时进入两个 neurons, 因此一共执行 $4 \times 2 = 8$ 个 dot products。Bias 再被加到每一行。代码虽然只有一行, 背后已经批量完成了整个 layer 的 forward。

这正是 Tensor abstraction 的价值: 业务代码描述高层数据变换, 底层 kernel 负责安排大量相同的小运算。

5. Broadcasting: 为什么 [2] 能加到 [4,2]

z 的 shape 是 [4,2], b 的 shape 是 [2]。

PyTorch 会把同一个 bias vector 应用到 batch 中每一行:

```
[4, 2]
+ [2]
-----
[4, 2]
```

这叫 Broadcasting。

现在只要记住: PyTorch 会尝试从最后一个 dimension 开始对齐; 维度相同或其中一个为 1 时通常可以广播。

Broadcasting 为什么存在

Bias 的语义是“同一组 neuron biases 应用于 batch 中每个 example”。如果没有 broadcasting, 需要先手工复制:

b [2] → 复制 4 次 → [4,2]

Broadcasting 用逻辑扩展表达相同意图, 通常不必真的复制整份数据。

Broadcasting 的危险

能广播不等于语义正确。例如 $[B,T,C] + [T,1]$ 可能运行成功，但你原本也许想加的是 $[C]$ bias。遇到意外结果时，先把两个 shapes 从右往左对齐，并逐维说明“这一维为什么应该复用”。

6. 用 Tensor 重写 Week 2

数据：

```
x = torch.tensor([[[1.0], [2.0], [3.0], [4.0]]])
y = torch.tensor([[[3.0], [5.0], [7.0], [9.0]]])
```

真实关系是：

$$y = 2x + 1$$

初始化 parameters：

```
w = torch.tensor([[0.0]], requires_grad=True)
b = torch.tensor([0.0], requires_grad=True)
```

Forward：

```
prediction = x @ w + b
loss = ((prediction - y) ** 2).mean()
```

数学仍然是：

$$\hat{y} = Xw + b$$

$$MSE = \frac{1}{n} \sum (\hat{y} - y)^2$$

7. requires_grad=True

```
w = torch.tensor([[0.0]], requires_grad=True)
```

意思是：

请追踪所有依赖 w 的 operations，以便以后计算 Loss 对 w 的 gradient。

Forward 时，PyTorch 动态建立 Computational Graph。

```
w, b
↓
prediction
↓
error
↓
square
↓
mean
↓
loss
```

哪些 Tensor 应该追踪 Gradient

通常只有需要学习的浮点 parameters 设置 `requires_grad=True`。Inputs 和 targets 默认不需要被 optimizer 更新，因此不必追踪它们的 gradient。

`requires_grad` 不是“这个 tensor 已经有 gradient”，而是“如果后面的 scalar loss 依赖它，请记录足够信息，以便未来计算 gradient”。真正的数值要等 `backward()` 后才会出现在 `.grad`。

Leaf Tensor 与中间 Tensor

直接创建并作为 parameter 使用的 w、b 通常是 leaf tensors。prediction、error、loss 是 operations 产生的中间 tensors，并带有 grad_fn 指向它们怎样被计算出来。

Optimizer 主要持有 leaf parameters；中间 tensors 用于连接 graph，不是长期保存的模型状态。

8. .backward()

```
loss.backward()
```

这行代码从 scalar loss 开始执行 Week 4 的反向传播。

结果保存在：

```
print(w.grad)
print(b.grad)
```

它们分别对应：

$$\frac{\partial Loss}{\partial w} \quad \frac{\partial Loss}{\partial b}$$

PyTorch 并没有“猜” gradient。它按照记录下来的 graph，调用每个 operation 的局部导数，再使用 Chain Rule。

为什么通常对 Scalar Loss 调用 Backward

loss.backward() 隐含的起点是：

$$\frac{\partial Loss}{\partial Loss} = 1$$

如果对一个多元素 tensor 直接 backward，就必须额外指定每个元素怎样被加权，因为“一个 vector 对 parameters 的 gradient”没有唯一方向。把 batch errors 求 mean 得到 scalar loss，既定义了统一目标，也提供了清晰的 backward 起点。

.grad 表示什么

w.grad 不是新 weight，也不是本次应该直接加多少。它是当前位置：

$$\frac{\partial Loss}{\partial w}$$

Optimizer 还会结合 learning rate、momentum 等规则决定真正 update。

9. 为什么要清空 Gradient

PyTorch 默认会累加 gradients：

```
new grad = old grad + current grad
```

这对某些算法有用，但标准 training loop 每一步通常要先清零：

```
w.grad.zero_()
b.grad.zero_()
```

使用 optimizer 时：

```
optimizer.zero_grad()
```

如果忘记，下一步 `update` 会混入前面 `batches` 的 `gradients`。

为什么默认设计成累加？因为同一个 `parameter` 可能通过多条路径被使用，或用户希望用多个小 `batches` 模拟一个大 `batch`。Autograd 无法假设你何时想结束累加，所以把清零时机交给 `training loop`。

标准顺序通常是先 `zero_grad()` 再 `forward`。只要每个 `step` 恰好清一次，放在上一步末尾也能工作；放在开头更容易读出“本 `step` 从干净 `gradient` 开始”。

10. 手动更新 Parameter

更新时不应该把“更新操作本身”继续记录到下一张 `graph`：

```
with torch.no_grad():
    w -= learning_rate * w.grad
    b -= learning_rate * b.grad
```

公式仍然是：

$$parameter := parameter - learningRate \times gradient$$

如果不使用 `torch.no_grad()`，`w -= ...` 可能被当成 `model computation` 的一部分继续记录，导致 `graph` 跨 `training steps` 连接起来。Parameter update 是 `optimizer` 的控制逻辑，不是 `model forward` 的可微分数据流。

11. 完整的 Tensor Training Loop

```
import torch

x = torch.tensor([[1.0], [2.0], [3.0], [4.0]])
y = torch.tensor([[3.0], [5.0], [7.0], [9.0]])

w = torch.tensor([[0.0]], requires_grad=True)
b = torch.tensor([0.0], requires_grad=True)

learning_rate = 0.01

for step in range(1000):
    # Forward
    prediction = x @ w + b
    loss = ((prediction - y) ** 2).mean()

    # Backward
    loss.backward()

    # Update
    with torch.no_grad():
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad

    # Reset gradients
    w.grad.zero_()
    b.grad.zero_()

print(w.item()) # approximately 2
print(b.item()) # approximately 1
```

这就是 Week 2 和 Week 4 的代码化版本。

12. nn.Module：组织 Parameters 和 Forward

真实模型不应手工管理大量 `parameters`。

```
import torch.nn as nn

class LinearModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(1, 1)

    def forward(self, x):
        return self.linear(x)
```

`nn.Linear(1,1)` 内部已经包含：

```
weight
bias
```

`model(x)` 会调用 `forward(x)`，同时保留 framework 所需的 hooks 和 graph 行为。

nn.Module 为什么不仅是代码封装

普通 class 也能写 `forward`，但 `nn.Module` 还维护一棵可遍历的模型对象树。把 `nn.Parameter` 或 `child module` 赋值给 `self` 后，PyTorch 才能统一完成：

```
model.parameters() 收集可训练参数
model.to(device) 递归移动参数和 buffers
model.state_dict() 保存可恢复状态
model.train()/eval() 切换子模块运行模式
```

因此 Module registration 是训练框架能够管理模型生命周期的基础。

13. Loss Function 与 Optimizer

```
model = LinearModel()
loss_function = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

它们的职责：

```
model    → 产生 prediction
loss_function → 衡量 prediction 有多差
autograd → 计算 gradients
optimizer → 更新 parameters
```

Loss function 和 optimizer 被分开，是因为它们回答不同问题：Loss 定义“什么叫变好”；optimizer 决定“根据 gradient 怎样走”。同一个 model 可以换训练目标，同一个 loss 也可以搭配不同 optimizer，而 Backprop 负责连接二者。

14. 标准 PyTorch Training Loop

```
for step in range(1000):
    optimizer.zero_grad()

    prediction = model(x)
    loss = loss_function(prediction, y)

    loss.backward()
    optimizer.step()
```

建议把它记成五个动作：

```
zero gradients
forward
loss
backward
step
```

这五步以后训练 Mini GPT 仍然存在。

每一步都不能随意省略：

```
zero → 防止混入旧 gradients
forward → 用当前 parameters 产生 prediction
loss → 把任务目标压成 scalar
backward → 计算所有 parameter gradients
step → 真正修改 parameters
```

只有 optimizer.step() 改变了参数；backward() 只计算并存放 gradients。

15. Shape Debugging 的固定方法

当代码出现 matmul、reshape 或 loss 错误时，不要盲猜。

在关键位置打印：

```
print("x:", x.shape)
print("weight:", model.linear.weight.shape)
print("prediction:", prediction.shape)
print("target:", y.shape)
```

逐项确认：

1. 每个 dimension 的业务含义；
2. Matrix multiplication 的中间维度是否相等；
3. Prediction 和 target 是否对齐；
4. Batch dimension 是否意外丢失；
5. dtype 和 device 是否一致。

Shape 在 AI 工程里相当于接口契约和数据 schema。

16. train() 与 eval()：知道用途即可

```
model.train()
model.eval()
```

它们不是控制 Autograd 开关，而是改变 Dropout、BatchNorm 等 layers 的运行方式。

Validation 常见写法：

```
model.eval()

with torch.no_grad():
    prediction = model(x)
```

后面训练 GPT 时再详细使用。

三个容易混淆的开关：

```
model.train() → layers 使用 training 行为
model.eval() → layers 使用 evaluation 行为
torch.no_grad() → 不记录 backward graph
```

eval() 不会自动关闭 gradient；no_grad() 也不会自动关闭 Dropout。Validation 通常同时使用 eval() 和 no_grad()，因为两种需求都存在。

17. Week 5 最应该理解的 6 件事

1. Tensor 是带 shape、dtype、device 的多维数字容器。
2. $X @ W + b$ 可以并行处理整个 batch。
3. Shape 是最重要的 debugging 信息之一。
4. requires_grad=True 让 PyTorch 记录计算依赖。
5. loss.backward() 执行 Chain Rule 并填充 .grad。
6. 标准 training loop 是 zero → forward → loss → backward → step。

18. Week 5 → Week 6

目前模型接收的是数字 features。

但 GPT 的 input 是文字。

下一个问题：

怎样把文字变成 tensor？模型怎样用这些 tensors 预测下一个 token？

Week 6 会建立这条路径：

```
Text
↓
Token IDs
↓
Embeddings
↓
Logits
↓
Cross Entropy Loss
```

Week 6 - Embedding 与 Language Model: 把文字变成预测问题

Week 6 核心目标

Week 5 已经能训练接收数字 tensor 的模型。

GPT 面对的新问题是：文字不是数字。

本周建立一条完整主线：

```
Text
↓ Tokenizer
Token IDs
↓ Embedding Lookup
Vectors
↓ Model
Logits
↓ Softmax
Probabilities
↓ compare with correct next token
Cross Entropy Loss
```

本周重点是 Embedding、Next-token Prediction、Logits、Softmax 和 Cross Entropy。

1. Token ID 只是编号

假设 vocabulary:

```
0 → 我
1 → 喜欢
2 → AI
3 → 学习
4 → 猫
```

句子：

我喜欢 AI

可以编码成：

```
[0, 1, 2]
```

但是 Token ID 的大小没有语义：

```
AI 的 ID 是 2
猫的 ID 是 4
```

不能因此说“猫是 AI 的两倍”。ID 只是数组索引。

Token ID 为什么仍然必不可少

字符串不适合直接作为矩阵索引，也无法高效组成固定 dtype 的 tensor。Tokenizer 先把有限 vocabulary 中的每个 token 映射到一个整数 ID，模型便能用 [B,T] 的整数 tensor 批量处理文本。

ID 的职责是稳定地回答“这是 vocabulary 中的哪一项”，不是表达含义。含义由该 ID 选中的 Embedding row 和后续 context computation 逐步形成。

2. 为什么需要 Embedding

神经网络需要连续数值 vector，而不是没有距离含义的编号。

Embedding 为每个 token 保存一个可学习 vector:

我 → [0.2, -0.1, 0.7]
喜欢 → [0.6, 0.3, -0.2]
AI → [-0.4, 0.8, 0.5]

这些数字最初通常是随机的，training 会不断更新它们。

所以 Embedding 不是人工编写的词义字典，而是一组 parameters。

为什么不直接把 ID 当作一个数输入

若把 猫=4、AI=2 直接当数值，Linear Layer 会被迫认为它们位于一条有顺序、有距离的数轴上。但这些编号通常只是按 vocabulary 建立顺序分配，换一种编号方式不应改变语言含义。

为什么不一直使用 One-hot

也可以把 ID 4 表示成：

[0,0,0,1]

这叫 one-hot vector。它避免了虚假的大小关系，但 vocabulary 若有 50,000 项，每个 token 就需要 50,000 维，而且任意两个不同 tokens 的距离都一样。

Embedding 相当于用 one-hot vector 选择矩阵的一行：

$$oneHot(token) \times E = E[token]$$

实际实现直接 lookup，不必构造巨大的 one-hot vector。

3. Embedding Matrix

如果：

vocabulary size = 5
embedding dimension = 3

Embedding Matrix:

$$E \in \mathbb{R}^{5 \times 3}$$

例如：

$$E = \begin{bmatrix} 0.2 & -0.1 & 0.7 \\ 0.6 & 0.3 & -0.2 \\ -0.4 & 0.8 & 0.5 \\ 0.1 & 0.2 & 0.9 \\ -0.7 & 0.4 & 0.3 \end{bmatrix}$$

Token ID 2 会选择第 2 行：

[-0.4, 0.8, 0.5]

代码：

```
embedding = torch.nn.Embedding(
    num_embeddings=5,
    embedding_dim=3,
)

token_ids = torch.tensor([0, 1, 2])
x = embedding(token_ids)
```

```
print(x.shape) # [3, 3]
```

每一行都是一个可训练 parameter vector。Vocabulary size 决定有多少行，embedding dimension 决定模型用多少个数字表示一个 token。Dimension 越大，representation capacity 通常越高，但 embedding、attention 和 FFN 的 parameter/compute 成本也随之上升。

4. Batch 进入 Embedding 后的 Shape

假设：

```
token_ids: [B, T]
```

其中 B 是 batch size，T 是 sequence length。

Embedding 后：

```
x: [B, T, C]
```

C 是 embedding dimension。

例如：

```
[4, 8] → Embedding → [4, 8, 32]
```

意思是四个 sequences，每个八个 tokens，每个 token 被表示成 32 个数字。

这个 shape 变化很关键：Tokenizer 提供离散 IDs，Embedding 把每个离散位置展开成连续 feature space。后续 Transformer 不再直接看“ID 7”，而是在 [C] 维 vector 上做 projection、dot product 和 nonlinear transformation。

5. Embedding 是怎样学到的

假设模型多次看到：

```
我喜欢 AI
我学习 AI
```

如果 Embedding 帮助模型更准确地预测下一个 token，Loss 会下降。

Backpropagation 会计算：

$$\frac{\partial Loss}{\partial E}$$

Optimizer 再更新被使用的 rows。

经过大量 contexts，具有类似使用方式的 tokens 可能形成相似 vectors。这不是提前规定的，而是训练目标的结果。

一个 Embedding Row 为什么会更新

假设当前 batch 出现 token AI。它的 embedding 参与后续 prediction，因此 Loss 依赖这一行。Backprop 会沿依赖路径计算：

$$\frac{\partial Loss}{\partial E[AI]}$$

Optimizer 更新这一行，使它在未来相似 contexts 中更有助于正确预测。没有出现在本次 lookup 中的 rows 没有这条依赖路径，通常不会在该 step 收到来自 token embedding 的 gradient。

Embedding 的“相似含义”不是单独的训练目标。它是大量 next-token tasks 共同施加压力后出现的有用结构。

6. Language Model 到底是什么

Language Model 的核心任务可以非常简单：

根据前面的 tokens，预测下一个 token。

例如：

Input: 我
Target: 喜欢

Input: 我 喜欢
Target: AI

数学：

$$P(token_t \mid token_1, token_2, \dots, token_{t-1})$$

意思是：已知前面的 tokens，第 t 个 token 出现的概率。

为什么选择 Next-token Prediction

这个目标有三个巨大优点：

1. 普通文本天然自带答案，不需要人工逐句标注；
2. 为了预测下一个 token，模型必须学习拼写、语法、事实关联、风格和上下文模式；
3. 同一个训练目标也能直接用于 generation。

例如语料本身是：

机器学习需要数据

把每个位置右边的 token 当 target，就自动产生多个 supervised examples。Language modeling 因此能把海量原始文本转成训练信号。

7. Autoregressive

GPT 是 Autoregressive Language Model：

根据已有内容预测一个 token

↓

把预测结果接到末尾

↓

再预测下一个 token

↓

重复

一整段文本的概率可以拆成一系列条件概率：

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

这里也叫 Chain Rule，但它是 Probability Chain Rule，不是 Week 4 求导的 Chain Rule。名字相同，内容不同。

Autoregressive 的“auto”不是自动训练，而是输出会再次成为自己的输入。模型一次只需要学会一个统一接口：

context → next-token distribution

重复调用这个接口，就能生成任意长度的 continuation。代价是生成具有顺序依赖，后一个 token 必须等前一个 token 产生后才能确定。

8. Input 和 Target 为什么错开一位

Token sequence:

[我, 喜欢, AI, 学习]

Training pair:

Input: [我, 喜欢, AI]

Target: [喜欢, AI, 学习]

每个位置都提供一个 next-token prediction:

我 → 喜欢

我 喜欢 → AI

我 喜欢 AI → 学习

一个长度为 T 的 sequence 可以同时提供 T 个训练目标。

为什么不能把 Input 和 Target 完全相同

如果当前位置的 target 也出现在当前位置 input 中，模型可能学会复制答案，而不是根据左侧 context 预测。向后错开一位，配合 causal mask，确保每个位置只能使用当时真正可获得的信息。

这也是一种常见 off-by-one bug：代码能运行、loss 甚至下降很快，但模型实际上看到了答案。检查 `inputs[:,1:] == targets[:, :-1]` 是数据管线的重要测试。

9. Model 输出的不是 Probability

模型最后为 vocabulary 中每个 token 输出一个原始分数：

`logits = [2.0, 1.0, 0.0]`

这些分数叫 Logits。

Logit:

- 可以为正或负；
- 不要求总和为 1；
- 数字越高，模型越倾向对应 token。

如果 vocabulary size 是 V ：

`logits shape = [B, T, V]`

为什么 Model 先输出 Logits

Linear Layer 最自然地产生不受范围限制的实数。让 network 直接维护“必须大于 0 且总和为 1”的约束会增加计算困难。Logits 把两个职责拆开：

Model 负责产生相对偏好分数

Softmax 负责把整组分数转成合法概率

Logit 的绝对值通常没有独立含义。给全部 logits 同时加 100，Softmax probabilities 不变；真正重要的是它们之间的差值。

10. Softmax：把 Logits 变成 Probability

Softmax:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

假设 logits:

$$[2, 1, 0]$$

指数:

$$e^2 \approx 7.39$$

$$e^1 \approx 2.72$$

$$e^0 = 1$$

$$\text{sum} \approx 11.11$$

所以 probabilities 约为:

$$[0.665, 0.245, 0.090]$$

它们都大于 0, 而且总和为 1。

Softmax 的关键不是绝对分数, 而是不同 logits 之间的相对差异。

Softmax 为什么使用指数和归一化

指数把任意实数变为正数, 并放大较大 logit 相对于较小 logit 的优势; 再除以全部指数和, 结果总和就严格为 1。

实际实现不会天真地直接计算巨大 e^z 。通常先减去最大 logit:

$$\text{softmax}(z) = \text{softmax}(z - \max(z))$$

这不会改变相对差值, 却能减少数值 overflow。框架内部已经处理了这种 numerical stability。

11. Cross Entropy: 正确答案的 Probability 有多高

假设正确 next token 是第一个 token, 模型给它的 probability 是:

$$p_{\text{correct}} = 0.665$$

Cross Entropy 对单个目标可理解为:

$$\text{Loss} = -\log(p_{\text{correct}})$$

所以:

$$\text{Loss} = -\log(0.665) \approx 0.408$$

如果模型给正确答案高概率, Loss 小; 给正确答案低概率, Loss 大。

$$p(\text{correct}) = 0.9 \rightarrow \text{low loss}$$

$$p(\text{correct}) = 0.1 \rightarrow \text{high loss}$$

这和 Week 2 的 MSE 角色相同: 都是衡量 prediction 与 target 的差距。只是任务不同, 所以 loss function 不同。

为什么使用 $-\log(p_{\text{correct}})$

$-\log$ 具有很合适的惩罚形状：

$p(\text{correct})$ 接近 1 $\rightarrow$ loss 接近 0
 $p(\text{correct})$ 变小 $\rightarrow$ loss 快速增大
 $p(\text{correct})$ 接近 0 $\rightarrow$ loss 非常大

模型不能只给正确 token 略高一点分数就满足；它会受到压力，把更多 probability mass 放到正确答案上。而且一段文本的条件概率连乘，取 $\log$ 后会变成求和，更适合稳定计算和对大量 token losses 求平均。Cross Entropy 同时具有概率解释和良好的优化性质。

12. 为什么 PyTorch CrossEntropyLoss 接收 Logits

PyTorch：

```
loss = torch.nn.functional.cross_entropy(logits, targets)
```

不要先手动执行 Softmax。

这个函数内部会以更稳定的方式组合 LogSoftmax 和 Negative Log Likelihood。

常见 shape：

```
logits: [B, T, V]
targets: [B, T]
```

计算 loss 前常见 reshape：

```
B, T, V = logits.shape
```

```
loss = F.cross_entropy(
    logits.reshape(B * T, V),
    targets.reshape(B * T),
)
```

现在一共有 $B \times T$ 个 classification tasks，每个 task 在 V 个 tokens 中选择正确答案。

如果先手动 Softmax，再把 probabilities 交给 cross_entropy，函数会把它们当成 logits 再处理一次，数学就错了。保留 logits 还允许函数用 $\log_{\text{softmax}}$ 的稳定实现，避免先得到极小 probability 再取 $\log$ 。

13. 最简单的 Bigram Language Model

Bigram model 只使用当前 token 预测下一个 token。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class BigramLanguageModel(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()

        self.token_table = nn.Embedding(
            vocab_size,
            vocab_size,
        )

    def forward(self, token_ids, targets=None):
        logits = self.token_table(token_ids)

        loss = None

        if targets is not None:
            B, T, V = logits.shape
```

```
loss = F.cross_entropy(
    logits.reshape(B * T, V),
    targets.reshape(B * T),
)
```

```
return logits, loss
```

如果 input shape 是 [4,8]，输出：

```
token_ids : [4, 8]
logits   : [4, 8, V]
```

每个 token ID 直接选择一行 next-token logits。

这张 Table 学到的是什么

token_table 有 [V,V] 个 parameters。第 i 行表示：当当前 token 是 i 时，下一个 token 的 logits 应该怎样分布。

它能学到诸如“机器 后常出现 学习”的局部统计，却无法区分两个更长 contexts 只因为它们最后一个 token 相同。这就是 Bigram 简单且有教育价值的原因：完整实现了 language-model training loop，同时把“缺少 context”这个问题暴露得很清楚。

14. 训练仍然是同一个 Loop

```
model = BigramLanguageModel(vocab_size)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
```

```
for step in range(1000):
    inputs, targets = get_batch()
```

```
optimizer.zero_grad()
```

```
logits, loss = model(inputs, targets)
loss.backward()
optimizer.step()
```

与 Week 5 相比，只是 input、model 和 loss function 变了：

```
Linear Regression → numeric features + MSE
Language Model    → token IDs + Cross Entropy
```

底层仍然是 Loss、Backpropagation、Optimizer。

15. Sampling: 怎样从 Probability 生成 Token

模型得到概率后，可以：

Argmax

总是选概率最高的 token。

```
next_id = torch.argmax(probabilities, dim=-1)
```

结果稳定，但可能机械和重复。

Sampling

按照概率随机抽取：

```
next_id = torch.multinomial(probabilities, num_samples=1)
```

高概率 token 更容易出现，但不是每次都相同。

Sampling 存在是因为模型输出的是分布，而语言本身常有多个合理 continuation。Argmax 把分布压成单一选择，可能反复进入同样的高概率循环；sampling 保留备选路径。后续还会用 temperature 和 top-k 控制分布的随机程度与候选范围。

16. Bigram Model 的关键局限

Bigram model 只看当前一个 token：

current token → next token

它无法有效处理：

- 很早以前出现的名字；
- 句子的语法结构；
- 代词指向谁；
- 长距离条件关系。

它缺少一种机制：

当前 token 应该从前面哪些 tokens 获取信息？

这正是 Attention 要解决的问题。

17. Perplexity：知道含义即可

Perplexity 常用于描述 language model 的平均不确定程度：

$$Perplexity = e^{Loss}$$

越低通常表示模型对正确 token 越有把握。但它必须在相同 tokenizer、dataset 和评估方式下比较才有意义。

例如 Loss = log(10) 时，Perplexity 是 10。直觉上可理解为模型平均像是在约 10 个同等可能的候选中犹豫。它不是模型真正每次只看 10 个 tokens，而是把平均 log loss 转回更容易感受的尺度。

18. Week 6 最应该理解的 7 件事

1. Token ID 只是索引，不包含大小语义。
2. Embedding 把 token ID 映射成可学习 vector。
3. Language Model 的核心任务是 next-token prediction。
4. Input 和 target 要错开一个位置。
5. Logits 是 vocabulary 中每个 token 的原始分数。
6. Softmax 把 logits 转换成 probabilities。
7. Cross Entropy 惩罚模型给正确 token 太低的 probability。

19. Week 6 → Week 7

现在我们已经能训练最小 language model。

但 Bigram Model 只知道当前 token，看不到真正的上下文。

Week 7 会加入 Attention：

当前 Token 的 Query



与前面 Tokens 的 Keys 比较



得到 Attention Weights



加权汇总 Values



Context-aware Representation

Week 7 - Attention: 让 Token 主动查找相关上下文

Week 7 核心目标

Week 6 的 Bigram Model 只能根据当前 token 预测下一个 token。

但语言中的答案经常依赖更早的内容：

小猫没有吃鱼，因为它不饿。

理解“它”时，模型需要回头寻找相关信息。

Attention 的核心问题是：

对当前 token 来说，前面哪些 tokens 重要？应该从它们那里取回多少信息？

本周重点：Query、Key、Value、Attention Score、Softmax、Causal Mask。

Attention 为什么会出现

在 Attention 之前，sequence model 也能通过固定窗口、循环状态或卷积聚合 context，但很远的信息要经过许多中间步骤才能传到当前位置。Attention 提供了一条内容相关的直接路径：当前 token 可以在一次 operation 中与允许查看的所有 tokens 比较，并决定向谁读取更多信息。

它解决的核心不是“存储更多文字”，而是“在已有 context 中动态路由信息”。

1. 最简单的 Context Aggregation

假设前面三个 token vectors 是：

$$x_1, x_2, x_3$$

最简单的方法是取平均：

$$context = \frac{x_1 + x_2 + x_3}{3}$$

问题是它认为每个 token 同样重要。

在“它不饿”中，“它”与“小猫”的关系可能比“鱼”更重要。模型需要根据当前 token 动态分配权重。

平均值还有另一个问题：不同查询得到同一个 context summary。无论当前 token 是“它”、还是“不饿”，都只能拿到相同平均结果。Attention 则让每个 query position 产生自己的一组 weights，所以 context 是针对当前需求定制的。

2. 用搜索系统理解 Q、K、V

可以把 Attention 类比为一次内存检索：

- Query：我正在寻找什么？
- Key：每条记录用什么特征接受匹配？
- Value：匹配后真正取回什么内容？

数据库类比：

Query 与 Keys 比较

↓

得到匹配分数

↓

把分数转换成权重

↓

按权重汇总 Values

Query、Key、Value 都由输入 vector 经过不同的可学习 projections 得到：

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

其中 W_Q 、 W_K 、 W_V 都是 parameters。

Projection 为什么是可学习的

原始 embedding 需要同时承载很多信息。用于“寻找主语”的特征，不一定等于被找到后应该传递的全部内容。三个 Linear projections 让训练自行建立三个工作空间：匹配需求、匹配标签、传输内容。

开始训练时这些 matrices 几乎是随机的。只要某种 Q/K 匹配与 V 传递方式能降低 next-token Loss, Backprop 就会逐步强化对应参数。

3. 为什么要有三份表示

同一个 token 在检索过程中扮演不同角色：

Query → 它此刻想找什么

Key → 它可以怎样被别人找到

Value → 被找到后提供什么信息

如果只用同一份 vector，模型就无法分别学习“匹配规则”和“取回内容”。

一个数据库 analogy 可以帮助区分：搜索“Melbourne 的天气”时，Query 里有地点与主题；记录的 Key 可能只包含地点、日期、类别；真正返回的 Value 才是温度和天气内容。匹配字段和返回内容承担不同职责，没必要完全相同。

Q、K、V 也不是从模型外部提供的三份数据。在 self-attention 中，它们都来自同一个 x，只是经过三组不同的 learned transformations。

4. Query 与 Key 的 Dot Product

两个 vectors 的 dot product 可以作为匹配分数：

$$score(q, k) = q \cdot k$$

例如当前 query：

$$q = [1, 0]$$

三个 keys：

$$k_1 = [1, 0], \quad k_2 = [0, 1], \quad k_3 = [1, 1]$$

计算：

$$q \cdot k_1 = 1$$

$$q \cdot k_2 = 0$$

$$q \cdot k_3 = 1$$

得到 scores:

$$[1, 0, 1]$$

第一和第三个 token 与当前 query 更匹配。

Dot Product 为什么能表示匹配

Dot product 把对应维度相乘再相加。如果 Query 和 Key 在重要维度上同号且都较大，总分上升；方向冲突的维度会降低总分。Training 可以调整 projection matrices，使“对任务有用的匹配关系”在这个空间中产生较高 dot product。

这里的 similarity 不是人类预先定义的词义相似度。某个 head 可以学习对 next-token prediction 有帮助的任意关系，例如局部搭配、分隔符、重复模式或远距离依赖。

5. Softmax 把 Scores 变成 Weights

对 scores 使用 Softmax:

$$\text{softmax}([1, 0, 1]) \approx [0.422, 0.155, 0.422]$$

这些 Attention Weights:

- 都大于 0;
- 总和为 1;
- 匹配分数更高的 token 获得更大权重。

Softmax 的作用不仅是归一化，还让 tokens 彼此竞争：一个位置获得更多 weight，其他位置的总份额就会减少。Weights 因而表示当前 query 在本次 forward 中如何分配有限的读取注意力。

Attention weight 高，说明该 Value 对当前 output 的线性贡献较大，但不能单独证明模型“理解了”某种人类概念；后面还有 output projection、Residual、FFN 和更多 layers 继续改变 representation。

6. 加权汇总 Values

假设:

$$v_1 = [1, 0], \quad v_2 = [0, 1], \quad v_3 = [1, 1]$$

输出:

$$\text{output} = 0.422v_1 + 0.155v_2 + 0.422v_3$$

$$\approx [0.844, 0.577]$$

当前 token 得到的新 representation，不再只包含自己，而是融合了它认为相关的 context。

注意真正被加权相加的是 Values，不是 Keys。Keys 只参加寻址，Values 承载返回内容。这个分工解释了为什么公式先用 QK^T 产生 weights，最后才乘 V 。

7. Scaled Dot-Product Attention

完整公式：

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

拆开：

QK^T → 每个 Query 与所有 Keys 的匹配分数
 $\div \sqrt{d_k}$ → 缩放 scores
 Softmax → 变成 attention weights
 $\times V$ → 加权汇总 values

公式看起来长，但只有“比较、归一化、取回”三个动作。

可以把完整公式拆成四个有明确 shape 的中间变量：

$$S = QK^T$$

$$S_{scaled} = \frac{S}{\sqrt{d_k}}$$

$$A = softmax(S_{scaled})$$

$$O = AV$$

S 是原始匹配分数，A 是读取权重，O 是读取后的新 representation。Debug 时分别检查这四步，比盯着一整行公式有效得多。

8. 为什么除以 $\sqrt{d_k}$

当 vector dimension 很大时，dot product 的绝对值容易变大。

非常大的 scores 进入 Softmax 后，概率可能过度接近 0 或 1，使 gradients 变弱、training 不稳定。

除以 $\sqrt{d_k}$ 是一个尺度校正：让不同 dimension 下的 score 范围更稳定。

目前记住用途即可，不需要推导统计证明。

一个数字直觉

如果 Query 和 Key 有很多彼此独立、方差相近的维度，dot product 是大量乘积之和。Dimension 越多，和的波动范围通常越大。除以 $\sqrt{d_k}$ 能把尺度拉回比较稳定的区间，避免 Softmax 过早变成几乎 one-hot。

Softmax 一旦极度饱和，小 score 的改变很难明显改变 probabilities，gradient 会变弱。因此 scaling 的直接目的不是改变谁最大，而是改善训练时的数值与 gradient 条件。

9. Self-Attention 的 Shape

输入：

X: [B, T, C]

投影后：

Q: [B, T, d_k]
 K: [B, T, d_k]
 V: [B, T, d_v]

Score matrix:

$Q @ K.transpose(-2, -1)$

[B, T, d_k] @ [B, d_k, T]
 ↓
 [B, T, T]

[B,T,T] 的两个 T 分别表示：

row → 哪个 token 正在查询

column → 它正在查看哪个 token

最后：

[B, T, T] @ [B, T, d_v]
 ↓
 [B, T, d_v]

每个 token 获得一个 context-aware vector。

[T,T] Matrix 应该怎样读

假设 T=4，score matrix 有 4 行、4 列。第 3 行不是“第三个 feature”，而是第 3 个 query token 对所有 4 个 key tokens 的看法。对最后一个 dimension 做 Softmax，表示每一行分别形成一组总和为 1 的读取 weights。

如果 Softmax 错做在 query dimension，代码仍可能运行，但语义会变成“多个 queries 竞争同一个 key”，不再是每个 query 独立选择 context。

10. 为什么 GPT 需要 Causal Mask

训练句子：

我 喜欢 AI

当模型在位置 1 预测“喜欢”时，不能提前看到位置 2 的“AI”。否则答案已经泄露。

Causal Mask 规定：

位置 0 只能看位置 0

位置 1 可以看位置 0, 1

位置 2 可以看位置 0, 1, 2

矩阵形式：

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

这是 GPT 保持 next-token prediction 因果性的关键。

Training 为什么尤其需要 Mask

为了并行效率，训练会一次输入完整 [B,T] sequence，并同时计算所有位置的 predictions。Causal mask 让这种并行计算仍然符合逐 token 生成时的信息边界：位置 t 的 output 只能依赖 ≤t 的 inputs。

如果没有 mask，training loss 会异常容易下降，因为每个位置可以从右边直接读取即将成为 target 的 token；但 inference 时未来 token 并不存在，模型就无法复现训练条件。这类问题叫 information leakage。

11. 为什么 Mask 在 Softmax 前使用 $-\infty$

不允许查看的位置被替换成负无穷：

```
scores = scores.masked_fill(mask == 0, float("-inf"))
```

因为：

$$e^{-\infty} = 0$$

经过 Softmax 后，这些位置的 attention weight 正好为 0。

如果只把 score 改成 0，它经过 Softmax 后仍可能获得非零 probability。

例如允许位置 score 是 $[-2, -3]$ ，若被 mask 的未来位置设成 0，它反而会成为最大的 score。 $-\infty$ 表示无论其他 scores 是多少，该位置指数后都必须为 0，从概率分布中彻底移除。

12. 一个 Attention Head 的 PyTorch 实现

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class AttentionHead(nn.Module):
    def __init__(self, model_dim, head_dim, context_length):
        super().__init__()

        self.query = nn.Linear(model_dim, head_dim, bias=False)
        self.key = nn.Linear(model_dim, head_dim, bias=False)
        self.value = nn.Linear(model_dim, head_dim, bias=False)

        mask = torch.tril(
            torch.ones(context_length, context_length)
        )

        self.register_buffer("causal_mask", mask)

    def forward(self, x):
        B, T, C = x.shape

        q = self.query(x)
        k = self.key(x)
        v = self.value(x)

        scores = q @ k.transpose(-2, -1)
        scores = scores / math.sqrt(k.size(-1))

        mask = self.causal_mask[:T, :T]
        scores = scores.masked_fill(mask == 0, float("-inf"))

        weights = F.softmax(scores, dim=-1)
        output = weights @ v

        return output
```

13. 代码与公式逐行对应

Projection

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

```
q = self.query(x)
k = self.key(x)
v = self.value(x)
```

Similarity 与 Scaling

$$S = \frac{QK^T}{\sqrt{d_k}}$$

```
scores = q @ k.transpose(-2, -1)
scores = scores / math.sqrt(k.size(-1))
```

Weighting

$$A = \text{softmax}(S)$$

$$O = AV$$

```
weights = F.softmax(scores, dim=-1)
output = weights @ v
```

14. 为什么叫 Self-Attention

因为 Q、K、V 都来自同一个 input sequence:

```
X → Q
X → K
X → V
```

序列内部的 tokens 相互查找信息，所以叫 Self-Attention。

Cross-Attention 则让 Query 来自一个 sequence，Key 和 Value 来自另一个 sequence。GPT 的核心是 causal self-attention。

15. Multi-Head Attention

一个 attention head 只有一套 W_Q 、 W_K 、 W_V 。

Multi-Head Attention 并行建立多套 projections:

```
Head 1 → 一种匹配空间
Head 2 → 另一种匹配空间
Head 3 → 另一种匹配空间
...
```

每个 head 的结果被拼接，再经过 output projection:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

不要把每个 head 强行解释成固定语法角色。它们的功能由 training 共同形成。

为什么一头不够

单个 head 在一次读取中只产生一组 attention weights 和一个 value subspace。语言中可能同时存在多种有用关系：当前词附近的搭配、较远的主语、标点边界、段落主题。多个 heads 提供多套独立 projections，让模型并行形成不同的读取方案，再由 output projection 混合。

如果 model dimension 是 $C=64$ 、heads 是 $H=4$ ，通常每个 head 处理 $D=16$ 个 channels，拼接后仍回到 64：

$4 \text{ heads} \times 16 \text{ channels} = 64 \text{ channels}$

因此 Multi-Head 增加的是多个子空间的并行视角，不是把 block 的外部 shape 变成四倍。

16. Attention 不是什么

Attention：

- 不是外部数据库搜索；
- 不是永久记忆；
- 不是完整可解释性证明；
- 不会自动知道 token 顺序。

它只是在当前 forward pass 中，根据当前 representations 进行动态加权信息交换。

尤其不要把 attention weights 直接当成完整解释。最终 prediction 受到多层、多个 heads、Residual paths、FFN 和 output layer 的共同影响；一个局部 weight matrix 只是整个因果链中的一部分。

17. Attention 的计算成本

Score matrix shape 是 $[T,T]$ ，所以 sequence 越长，计算和内存增长很快：

$$O(T^2)$$

这是长 context 模型需要专门优化的重要原因。当前只需知道来源是每个 token 都要与其他 tokens 比较。

例如 sequence length 从 1,000 增到 2,000，score matrix 元素数从约 100 万增到约 400 万，而不是 200 万。Context 翻倍，attention pair 数约变四倍。

$O(T^2)$ 描述主要随 sequence length 的增长趋势，不表示 Transformer 的所有计算都恰好只有这一项。FFN、projections、batch size、heads 和 model dimension 同样影响实际时间与显存。

18. Week 7 最应该理解的 7 件事

1. Attention 让每个 token 动态选择相关 context。
2. Query 表示“我要找什么”。
3. Key 表示“我怎样被匹配”。
4. Value 表示“匹配后提供什么”。
5. Dot Product 产生 similarity scores。
6. Softmax 把 scores 转成总和为 1 的 weights。
7. Causal Mask 防止 GPT 偷看未来 token。

19. Week 7 → Week 8

Attention 解决了 token 之间的信息交换，但它还不是完整 Transformer。

Transformer Block 还需要：

Attention

- + Residual Connection
- + Layer Normalization
- + Feed-Forward Network

Week 8 会解释这些零件为什么同时存在，以及它们如何保持同一个 [B,T,C] shape 反复堆叠。

Week 8 - Transformer: 把 Attention 组装成可训练的网络

Week 8 核心目标

Week 7 学会了一个 Attention Head。

Transformer 不是一个新的单独公式，而是一套可重复堆叠的 architecture：



本周重点：Position、Residual Connection、LayerNorm、Feed-Forward Network 和完整 Transformer Block。

1. Attention 还缺什么

Self-Attention 能让 tokens 交换信息，但完整网络还要解决：

1. token 的位置怎样进入模型；
2. 深层网络怎样保留原信息并稳定训练；
3. 每个 token 汇总 context 后，怎样独立进行更复杂变换；
4. 如何把同一套 block 堆叠多次。

Transformer 的其他组件正是围绕这些问题设计的。

可以把一个 Block 看成维持同一接口的处理单元：输入和输出都是 [B,T,C]，但每次经过它，token representation 都能融合更多 context 并完成更复杂的非线性变换。相同 shape 并不表示内容没变，就像一个方法接收并返回同一 DTO 类型，内部字段含义仍可被更新。

2. Token Embedding 不包含位置

Token embedding 只由 token ID 决定：

“猫” 在位置 1 → 相同 token embedding
“猫” 在位置 8 → 相同 token embedding

但语言顺序很重要：

猫追狗
狗追猫

tokens 相同，含义不同。

Attention 自己为什么不能可靠知道顺序

如果没有 position information，同时对 sequence 的 token rows 做同样排列变化，Q/K/V 和 attention outputs 也会跟着排列；operation 本身主要根据内容匹配，不天然拥有“第一个、前一个、相距三步”的坐标系统。

语言却高度依赖顺序。因此必须在进入 blocks 前或 attention 计算中注入位置关系，否则“猫追狗”和“狗追猫”包含相同 token identities，模型很难区分角色关系。

3. Position Embedding

最直观的方法是为每个位置也保存一个可学习 vector：

Token Embedding [B, T, C]
Position Embedding [T, C]

相加 [B, T, C]

代码：

```
token_vectors = token_embedding(token_ids)

positions = torch.arange(T, device=token_ids.device)
position_vectors = position_embedding(positions)

x = token_vectors + position_vectors
```

为什么相加而不是拼接？相加保持 model dimension c 不变，让后面的 blocks 使用同一个接口。

现代模型也可能使用 Rotary Position Embedding 等方案。当前 Mini GPT 使用 learned position embedding，先把核心主线学清。

Position Embedding 学到什么

Learned position embedding 的每一行也是 parameter：第 0 行代表位置 0 的可学习向量，第 1 行代表位置 1，以此类推。它不是人工编码“这是第二个词”，而是让 training 学会各位置怎样影响 representation。

它的直接限制是只为配置中的有限 positions 建表，所以超过 context_length 没有现成 row。RoPE 等方案改变位置注入方式，但要解决的仍是同一个基本问题：让 Attention 感知顺序和距离。

4. Transformer Block 的两个主要计算单元

Self-Attention

让一个 token 从其他相关 tokens 读取信息。

Feed-Forward Network

让每个 token 对自己已经汇总的信息进行独立的非线性变换。

可以理解为：

Attention → tokens 之间通信
FeedForward → 每个 token 自己思考

这只是帮助理解的类比，但很好地说明了两者的分工。

为什么两者都需要？Attention output 本质上是 Values 的加权组合，擅长从其他 positions 搬运和混合信息；FFN 则能在每个 position 的 feature dimensions 内重新编码这些信息。只通信而不处理，或只处理自己而不读取 context，表达能力都会受到限制。

5. Feed-Forward Network

典型形式：

$$FFN(x) = W_2 GELU(W_1 x + b_1) + b_2$$

通常先扩大 dimension，再缩回：

$C \rightarrow 4C \rightarrow C$

代码：

```
self.ffn = nn.Sequential(
    nn.Linear(model_dim, 4 * model_dim),
    nn.GELU(),
    nn.Linear(4 * model_dim, model_dim),
)
```

它应用在每个 token position 上，但所有 positions 共享同一组 weights。

为什么先扩大到 4C

中间更宽的空间给模型更多“临时工作维度”来组合 features，GELU 再选择性保留不同响应，最后投影回 C 以满足 residual interface。4C 是常见设计比例，不是数学定律；不同模型会使用不同 expansion ratio 或 gated FFN。

同一组 FFN weights 应用于所有 token positions，类似同一个纯函数被批量调用：每个 position 输入不同，因此输出不同，但参数共享使模型能把学到的 feature transformation 应用于任意位置。

6. 为什么需要 GELU

如果两层 Linear 之间没有 activation：

$$W_2(W_1 x + b_1) + b_2$$

仍然可以合并成一次 Linear Transformation。

GELU 引入 non-linearity，使 FFN 能表达更复杂关系。

GELU 的精确公式现在不是重点。记住它在 Transformer 中扮演与 Week 3 ReLU 类似的角色即可。

Activation 为什么放在两次 Linear 之间

两次 Linear 若直接相连，整体仍是一次 Linear：

$$W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$$

无论堆多少次都无法产生真正的非线性 decision boundary。GELU 让不同输入区域受到不同程度的响应，使网络能够表达无法被一个 matrix 完成的条件关系。

与 ReLU 的硬门槛相比，GELU 是更平滑的 gating。当前关键不是背它的精确曲线，而是理解它阻止两层 Linear 被合并。

7. Residual Connection

Residual Connection：

$$y = x + F(x)$$

代码：

```
x = x + attention(x)
x = x + feed_forward(x)
```

它让一个 layer 学习“在原信息上增加什么变化”，而不是每层都重建全部 representation。
 它还为 gradient 提供更直接的路径：

$$\frac{\partial y}{\partial x} = 1 + \frac{\partial F(x)}{\partial x}$$

即使 F 的 gradient 很弱，仍然有 identity path 中的 1。

Residual 为什么既帮助信息也帮助 Gradient

没有 residual 时，每个 layer 必须把所有有用信息完整传给下一层；某层一旦破坏信息，后面很难恢复。 $x + F(x)$ 保留原始 x ，分支只需学习有用的增量。

Backward 时，gradient 也有两条路径：一条经过复杂的 F ，另一条直接经过加法回到 x 。路径汇合时 gradients 相加，所以即使 F 路径很弱，identity path 仍能提供稳定通道。这直接连接 Week 4 的“路径上相乘、分支处相加”。

8. Residual 对 Shape 的要求

要执行：

```
x + attention_output
```

两者 shape 必须相同：

```
x      : [B, T, C]
attention_output : [B, T, C]
result  : [B, T, C]
```

Feed-Forward 最后也必须回到 $[B, T, C]$ 。

这就是为什么 Transformer Block 可以重复堆叠：每个 block 输入输出接口相同。

如果 attention head 拼接后得到 $[B, T, H \times D]$ ，output projection 的重要职责之一就是把它映射回 C 。FFN 虽然中间扩大到 $4C$ ，最后也必须回到 C 。Residual addition 因而像一个强制的 architecture contract，约束每个 sublayer 的外部接口。

9. Layer Normalization 的直觉

网络很深时，每层输入数值范围可能不断漂移，训练会变得不稳定。

LayerNorm 对每个 token 的 C 个 features 做标准化：

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

然后再使用可学习 parameters 调整：

$$y_i = \gamma \hat{x}_i + \beta$$

直觉：

先把每个 token 的内部数值拉回稳定尺度
 再让模型学习适合的缩放和平移

不需要手算完整统计过程，重点是它稳定每个 token representation 的尺度。

LayerNorm 到底在哪个范围计算

对 [B,T,C]，LayerNorm 通常对每个 batch、每个 token 自己的最后 C 个 features 计算 mean 和 variance：

token 1 的 C 个数 → 一组 μ 、 σ^2

token 2 的 C 个数 → 另一组 μ 、 σ^2

它不会把不同 examples 的统计混在一起，因此 training 和单条 inference 都能使用一致逻辑。

例如 token vector [1,2,3] 的 mean 是 2；中心化后为 [-1,0,1]，再除以标准差，使整体尺度稳定。最后的 γ 和 β 是可学习 parameters，保证 normalization 不会永久剥夺模型所需的缩放与偏移能力。

10. Pre-Norm Transformer Block

Mini GPT 常用 Pre-Norm：先 LayerNorm，再进入主要计算。

```
x = x + self.attention(self.ln1(x))
```

```
x = x + self.ffn(self.ln2(x))
```

数学：

$$x' = x + \text{Attention}(\text{LN}(x))$$

$$y = x' + \text{FFN}(\text{LN}(x'))$$

注意 Residual 保存的是 normalization 前的原始路径。

为什么使用 Pre-Norm

Post-Norm 与 Pre-Norm 都是有效架构选择。Pre-Norm 把 normalized input 送入复杂分支，同时让 residual identity path 保持直接，深层模型通常更容易获得稳定 gradient flow。

读代码时不要只看有没有 LayerNorm，还要确认它位于分支前还是分支后。顺序改变会改变实际 function 和训练行为。

11. 完整 Transformer Block

```
class TransformerBlock(nn.Module):
    def __init__(self, model_dim, num_heads, context_length):
        super().__init__()

        self.ln1 = nn.LayerNorm(model_dim)
        self.attention = CausalSelfAttention(
            model_dim=model_dim,
            num_heads=num_heads,
            context_length=context_length,
        )

        self.ln2 = nn.LayerNorm(model_dim)
        self.ffn = nn.Sequential(
            nn.Linear(model_dim, 4 * model_dim),
            nn.GELU(),
            nn.Linear(4 * model_dim, model_dim),
        )

    def forward(self, x):
        x = x + self.attention(self.ln1(x))
        x = x + self.ffn(self.ln2(x))
        return x
```

从外面看，它只是：

$[B, T, C] \rightarrow \text{Block} \rightarrow [B, T, C]$

内部却完成了 context communication 和 nonlinear transformation。

这四行 forward 很短，但每行都有明确职责：

ln1 → 稳定送入 attention 的尺度

attention → 跨 tokens 读取信息

first + → 保留原信息并加入 context update

ln2/ffn → 对每个 token 的 features 做非线性处理

second + → 再次保留 identity path

“代码短”不等于“概念简单”；理解 Block 的关键是能解释每个组件为什么不能随意删除。

12. 堆叠多个 Blocks

```
self.blocks = nn.Sequential(
    *[
        TransformerBlock(
            model_dim=model_dim,
            num_heads=num_heads,
            context_length=context_length,
        )
        for _ in range(num_layers)
    ]
)
```

数据流：

```
[B, T, C]
↓ Block 1
[B, T, C]
↓ Block 2
[B, T, C]
↓ Block 3
[B, T, C]
```

更深的 layers 可以在前面形成的 representations 上继续组合信息。

第一层看到的是 token + position 的初始 representation；后面层看到的是已经融合过 context 的 representation。于是后层 attention 不只比较原始词义，还能比较前层形成的短语、关系和结构特征。Depth 的价值来自这种逐层组合，而不是简单重复同一个静态结果。

13. 从 Transformer Output 到 Vocabulary Logits

最后使用 Linear Layer 把每个 token 的 C 维 representation 映射到 V 个 vocabulary logits：

$$\text{logits} = xW_{\text{vocab}} + b$$

Shape：

```
x : [B, T, C]
W_vocab: [C, V]
logits: [B, T, V]
```

这就是 Week 6 Cross Entropy 需要的格式。

LM Head 存在是因为内部 C 维 model space 与外部 V 个 vocabulary choices 是两种不同接口。每个 token position 的 representation 经过同一组 output weights，与 vocabulary 中每个候选形成一个 logit。

它不直接选择 token；training 时 logits 交给 Cross Entropy，generation 时最后一个位置的 logits 经过 sampling 策略。Architecture 提供分布，使用场景决定怎样消费分布。

14. GPT Forward Pass 的完整 Shape

假设：

B = 4
T = 8
C = 32
V = 100

那么：

token IDs [4, 8]
↓ Embedding
token representations [4, 8, 32]
↓ Blocks
context representations [4, 8, 32]
↓ LM Head
logits [4, 8, 100]

Targets:

[4, 8]

Cross Entropy 把它看成 $4 \times 8 = 32$ 个 next-token classification tasks。

15. Encoder、Decoder 和 Decoder-only

只需要知道三种常见结构：

Encoder-only

通常允许双向查看 context，适合理解和表示任务。

Encoder-Decoder

一部分读取输入，另一部分逐步生成输出，常用于 sequence-to-sequence。

Decoder-only

使用 causal self-attention，从左到右预测下一个 token。

GPT 采用 Decoder-only architecture，因为它的训练目标就是 autoregressive next-token prediction。

“Decoder-only” 不表示 GPT 只做机械解码，也不表示它内部没有理解输入。这个名称来自 Transformer 架构分类：它使用带 causal mask 的 stack，从左到右建立 representations 并预测后续 token。

Encoder-only 允许每个位置同时查看左右 context，因此不适合直接用同一 forward 做严格 autoregressive generation；Encoder-Decoder 则把输入表示和输出生成分成两个 stacks。选择哪种结构取决于信息可见性和任务接口。

16. Transformer 的能力来自哪里

不是某个单独公式产生“智能”。能力来自整个系统共同作用：

大量训练数据
+ 大量 Parameters
+ Attention 信息交换
+ FFN 非线性变换
+ 多层堆叠
+ Loss 与 Backpropagation
+ 足够计算资源

底层仍然没有离开 Week 2：寻找能够降低 Loss 的 parameters。

17. 需要知道但暂不展开的概念

- Dropout: training 时随机关闭部分路径, 帮助 regularization;
- Rotary Position Embedding: 另一种注入相对位置信息的方法;
- KV Cache: generation 时复用过去 tokens 的 Keys 和 Values;
- Flash Attention: 在不改变 Attention 数学结果的前提下优化计算和内存访问。

它们重要, 但不是理解基础 Transformer 的前置条件。

这里的取舍是“关键知识详细、扩展知识定位清楚”:

- Dropout 解决 regularization;
- RoPE 解决位置关系表示;
- KV Cache 解决逐 token inference 重复计算;
- Flash Attention 解决 Attention kernel 的内存访问效率。

它们分别作用于不同层面, 不能互相替代。先能完整解释基础 Block, 再进入这些优化, 学习路径会更清晰。

18. Week 8 最应该理解的 7 件事

1. Position information 让模型区分 token 顺序。
2. Attention 负责 tokens 之间的信息交换。
3. FFN 对每个 token 做非线性变换。
4. Residual Connection 保留原路径并帮助 gradient 流动。
5. LayerNorm 稳定每个 token representation 的尺度。
6. 每个 block 保持 [B,T,C] shape, 因此可以堆叠。
7. LM Head 把 [B,T,C] 映射成 [B,T,V] logits。

19. Week 8 → Week 9

我们已经看到 token IDs 如何进入 Transformer。

但还有一个被暂时隐藏的问题:

文字究竟应该怎样切成 tokens? 为什么一个 token 不一定等于一个 word?

Week 9 会实现 Tokenizer 和训练数据切片, 并解释 Tokenizer 为什么必须与 Model 绑定。

Week 9 - Tokenizer: 文字怎样变成 GPT 的输入

Week 9 核心目标

Transformer 接收的是 token IDs:

```
[18, 42, 7, 91, ...]
```

Tokenizer 负责在文字和 IDs 之间转换:

```
Text --encode--> Token IDs  
Text <--decode-- Token IDs
```

本周重点: Token、Vocabulary、Subword、BPE 直觉、Encode/Decode、Special Tokens, 以及如何构造训练 batches。

Tokenizer 为什么是独立组件

Neural Network 的输入接口是有限范围的整数 IDs, 用户输入却是开放的 Unicode text。Tokenizer 位于二者之间, 负责定义稳定协议: 哪些 text fragments 属于 vocabulary、怎样编号、遇到新组合怎样拆分、怎样把 IDs 还原成 text。

它像编译器的 lexer 加 symbol table。Model 不直接读取字符文件, 而是读取 tokenizer 输出的 token stream。协议一旦改变, 同一个 ID 的含义就可能改变, 因此 tokenizer 必须与 checkpoint 一起版本化。

1. Token 不等于 Word

一句话可以有不同切法。

Character-level

```
learning → l e a r n i n g
```

Word-level

```
I love learning → I | love | learning
```

Subword-level

```
learning → learn | ing
```

GPT 常使用 subword tokenizer。一个 token 可能是:

- 一个完整单词;
- 单词的一部分;
- 标点;
- 空格加单词片段;
- 某些语言中的一个或多个字符。

所以 token count 与 word count 不是同一件事。

Token 是计算单位, 不是语言学真理

Tokenizer 的切分目标主要是计算效率和覆盖能力, 不是完美复原人类对“单词”的定义。英文中的空格、中文的汉字组合、emoji、代码缩进和标点可能使用不同切分策略。

同一句文本换 tokenizer 后可能得到不同 token count。这会影响 context window 中能容纳的实际文字量、API 计费、training steps 中的 target 数量和 generation 速度。

2. 为什么不简单按 Word 切

Word-level vocabulary 会遇到：

```
learn
learning
learned
learner
```

如果每个变体都作为独立 token，vocabulary 会非常大，而且新词难处理。

如果只用 characters，vocabulary 很小，但 sequence 会变长：

```
transformer
→ t r a n s f o r m e r
```

Subword 是折中：常见片段可以整体保存，罕见词可以拆成较小片段。

例如模型没见过完整词 unhappiness，仍可能把它拆成：

```
un | happi | ness
```

这样既能复用常见片段的统计，又不需要为所有可能的新词预留独立 vocabulary entry。Subword 解决的是开放文本与有限 vocabulary 之间的工程矛盾。

3. Vocabulary Size 的 Trade-off

Vocabulary 较小

优点：Embedding 和输出层较小

缺点：同一文本需要更多 tokens，sequence 更长

Vocabulary 较大

优点：常见文字可用更少 tokens 表示

缺点：Embedding 和 LM Head 参数更多

Tokenizer design 会同时影响：

- sequence length;
- context window 能容纳多少文字;
- model parameter count;
- training 和 inference 成本。

用 Parameter Count 看 Trade-off

Token Embedding 和 LM Head 的大小都与 v 有关。若 model dimension 是 C ，两部分大致包含 $v \times C$ 级别的 weights。Vocabulary 从 30,000 增到 60,000，会显著增加输入/输出 layers 的 parameters。

但 vocabulary 太小会让同一段文字拆成更多 tokens；Attention 的主要成本又随 sequence length 近似平方增长。因此不存在脱离 language、data 和 model size 的“最佳 vocabulary size”。

4. Character Tokenizer：先理解完整闭环

为了从零实现 Mini GPT，可以先用 character tokenizer。

语料：

```
text = "我喜欢AI, AI帮助学习。"
```

建立 vocabulary:

```
characters = sorted(set(text))

token_to_id = {
    token: index
    for index, token in enumerate(characters)
}

id_to_token = {
    index: token
    for token, index in token_to_id.items()
}
```

Encode:

```
def encode(value):
    return [token_to_id[ch] for ch in value]
```

Decode:

```
def decode(ids):
    return "".join(id_to_token[i] for i in ids)
```

必须满足基本 round-trip:

```
sample = "AI学习"
assert decode(encode(sample)) == sample
```

为什么 Character Tokenizer 适合教学

它把 tokenization 的完整闭环暴露出来: 建立 vocabulary、双向 mapping、encode、decode、unknown character 和 batch construction 都能直接看到。没有 BPE merge rules 的额外复杂度, 便于确认 model pipeline。

它不适合直接代表工业 GPT tokenizer。Character-level sequence 往往更长, 对不同 Unicode writing systems 的处理也不均衡。教学中使用它, 是为了先验证接口和学习原理, 不是因为它性能最佳。

5. Unicode 与 UTF-8

Python string 保存 Unicode characters; 文件需要使用明确 encoding 读取:

```
with open("sample_corpus.txt", "r", encoding="utf-8") as file:
    text = file.read()
```

如果训练和运行环境用不同编码读取同一文件, 可能产生乱码或不同 token sequence。

Tokenizer 不直接处理“字形”, 而是处理解码后的 text units。Character tokenizer 中一个 Python character 也不保证等于用户视觉上看到的完整 grapheme, 但教学项目先不展开 Unicode segmentation。

例如某些 emoji 由多个 Unicode code points 组合, 带重音字符也可能有不同规范化形式。视觉上相同的字符串若底层 code points 不同, character vocabulary 可能认为它们不同。

这就是为什么 production tokenizer 需要明确 UTF-8、normalization 和 byte fallback 策略。编码问题不仅会显示乱码, 还可能悄悄改变 token IDs, 使 checkpoint 输入完全错位。

6. 未见 Token 问题

Character tokenizer 如果遇到 vocabulary 中没有的 character:

```
encode("新字符")
```

会失败。

常见处理方式：

- 使用 <UNK>；
- 使用覆盖范围更广的 byte-level tokenizer；
- 使用 subword / byte fallback。

我们的 Mini GPT vocabulary 来自完整训练语料，因此在同一语料内实验不会遇到这个问题。

<UNK> 能保证编码继续进行，却把多个未知文本压成同一个 ID，信息会丢失。Byte-level fallback 则能用有限的 byte units 表示任何 UTF-8 输入，代价是罕见内容可能产生更长 sequence。选择策略是在覆盖能力、序列长度与可读性之间取舍。

7. BPE 的核心直觉

BPE (Byte Pair Encoding) 的思想是：

从较小单位开始，反复把语料中最常出现的相邻 pair 合并成新 token。

假设初始语料中经常出现：

```
low
lower
```

如果 lo 最常见，先合并：

```
low
lower
```

如果 low 又很常见，再合并：

```
low
lower
```

经过多轮后，常见字符串会成为长 token；罕见字符串仍能由较小单位组合。

手工再走一轮 BPE

假设 corpus 中片段出现次数为：

```
low   × 5
lower × 2
new   × 2
```

Pair lo 出现 7 次，可能先合并成 lo；随后 lo w 也出现 7 次，再合并成 low。Vocabulary 多了 lo、low 两个 entries，旧的较小单位仍保留用于组合其他字符串。

关键是 merge rules 来自 corpus frequency。更换领域或语言，常见 pairs 变化，最终 token vocabulary 也会变化。

8. Tokenizer Training 与 Text Encoding 不同

Tokenizer Training

从大语料统计并决定 vocabulary 和 merge rules。

Encoding

使用已经固定的 vocabulary 和 rules 把新文本转换成 IDs。

模型部署时不能随意重新训练 tokenizer。因为同一个 ID 必须始终代表同一个 token。

Tokenizer training 类似生成数据库 schema；encoding 类似按固定 schema 写入数据。重新 training 后即使 vocabulary size 碰巧相同，ID-to-token mapping 和 merge priority 也可能完全不同，旧 model weights 不会自动适配。

9. Tokenizer 与 Model 必须绑定

假设训练时：

ID 42 → "ing"

另一个 tokenizer 中：

ID 42 → "cat"

即使 model architecture 完全一样，输入含义也已破坏。

所以一个可用 checkpoint 至少需要匹配：

```
model weights
+ architecture config
+ tokenizer vocabulary/rules
```

Embedding row 42 学到的是“训练 tokenizer 中 ID 42 的使用方式”。如果加载另一个 tokenizer，row 42 的数值还在，却被错误地赋给另一 token。程序常常不会 shape error，但输出会严重异常，这是一种语义层面的 schema mismatch。

10. Special Tokens

常见 special tokens：

```
<BOS> Beginning of Sequence
<EOS> End of Sequence
<PAD> Padding
<UNK> Unknown
```

它们不是普通文字，但在 sequence 中占据 token ID。

例如 <EOS> 可以教模型：

一段文本在这里结束。

Chat model 还可能使用表示 role、message boundary 或 tool call 的 special tokens。

Special tokens 的意义来自 training data 和 model objective，而不是尖括号文字本身。只有语料构造和 loss 让 <EOS> 与“结束”共同出现，模型才会学会在合适位置提高其 probability。

它们必须提前保留固定 IDs，并与普通 tokenizer rules 协调，否则 special marker 可能被拆成多个普通 tokens，失去协议作用。

11. Padding 与 Attention Mask

不同 examples 长度不同：

```
[A, B, C]
[D, E]
```

为了放进同一个矩形 batch，可以 padding：

```
[A, B, C]
[D, E, PAD]
```

Padding mask 告诉模型不要把 <PAD> 当正常内容。

它与 Causal Mask 目的不同：

```
Causal Mask → 不看未来
Padding Mask → 不看补齐位置
```

固定长度的连续文本训练可以直接切块，不一定需要 padding。

Padding mask 与 causal mask 可能同时存在。前者处理 batch 的矩形对齐，后者处理时间方向的信息边界。一个位置即使属于过去，如果它只是 <PAD> 也不应被读取；一个位置即使是真实 token，如果位于未来也不应被读取。

12. 把 Corpus 变成 Token Tensor

```
token_ids = encode(text)
data = torch.tensor(token_ids, dtype=torch.long)
```

Shape:

```
[total_token_count]
```

这是一个连续 token stream。

为什么常把多个文档连接成 stream？这样可以高效切出大量固定长度 windows。不过生产系统通常还会加入 document boundary token，防止模型把两个无关文档的拼接误当成自然连续文本。

13. Train / Validation Split

```
split_index = int(0.9 * len(data))

train_data = data[:split_index]
validation_data = data[split_index:]
```

Training data 用于更新 parameters；Validation data 只用于衡量模型面对未参与更新的数据时表现如何。

不要先随机打乱整个 token stream，因为这样会破坏文本的局部顺序。

Split 也要防止数据泄漏。如果同一文档的重复内容同时出现在 train 和 validation，validation loss 可能过于乐观。教学项目按连续位置切分足够展示概念；严谨实验还会按文档或时间划分，并去除重复样本。

14. 从 Token Stream 取一个 Example

假设 context length:

```
T = 4
```

从位置 s 取:

```
x = data[s : s + T]
y = data[s + 1 : s + T + 1]
```

如果片段是:

```
[我, 喜欢, AI, 学习, 数学]
```

那么:

```
x = [我, 喜欢, AI, 学习]
y = [喜欢, AI, 学习, 数学]
```

Targets 正好错开一位。

一个长度为 4 的 input 实际提供 4 个目标:

```
位置 0: 看 “我”      预测 “喜欢”
位置 1: 看 “我 喜欢”   预测 “AI”
位置 2: 看 “我 喜欢 AI” 预测 “学习”
位置 3: 看 “我 喜欢 AI 学习” 预测 “数学”
```

Causal mask 保证每个 position 虽然在同一次 parallel forward 中计算，仍只能读取自己左侧允许的 context。

15. 构造一个 Batch

```
def get_batch(source, batch_size, context_length):
    starts = torch.randint(
        low=0,
        high=len(source) - context_length,
        size=(batch_size,),
    )

    inputs = torch.stack([
        source[i : i + context_length]
        for i in starts
    ])

    targets = torch.stack([
        source[i + 1 : i + context_length + 1]
        for i in starts
    ])

    return inputs, targets
```

Shape:

```
inputs : [B, T]
targets: [B, T]
```

随机 start positions 让训练过程中模型看到语料的不同片段。

Batch 不是把 tokens 任意混在一起。每一行内部仍保持连续顺序，只是不同 rows 可以从 stream 的不同 start positions 取样。torch.stack 增加 batch dimension，把多个独立 windows 组成 [B,T]。

边界条件也很重要：最高 start 必须保证还能读取 T+1 个 tokens，因为 target 比 input 多向右访问一个位置。Off-by-one 在这里通常会表现为偶发 slice 长度不一致。

16. Tokenizer 不负责理解语言

Tokenizer 只决定如何切分和编号。

```
Tokenizer → representation boundary
Model    → learns patterns from token sequences
```

好的 tokenizer 能提高效率，但知识和语言能力主要来自 model、data 和 training。

Tokenizer 会影响模型学习难度，却不会自己生成 semantic representation。它只是决定原始 text 被切成哪些离散符号；Embedding 和 Transformer 才根据 contexts 学习这些符号怎样关联。

17. Week 9 最应该理解的 7 件事

1. Token 不一定等于 word。
2. Subword 在 vocabulary size 与 sequence length 之间折中。
3. Encode 把 text 变成 IDs，Decode 做反向转换。
4. Tokenizer training 与 text encoding 是两个阶段。
5. Tokenizer 必须与 model checkpoint 严格匹配。
6. Inputs 和 targets 从同一 token stream 错开一位。
7. 一个 batch 的基本 shape 是 [B,T]。

18. Week 9 → Week 10

现在完整数据入口已经准备好：

Text
↓ Tokenizer
[B,T] Token IDs

Week 10 会把此前的 Embedding、Attention、Transformer Block 和 LM Head 组合成一个完整 Mini GPT class，并用 shape 和 causal tests 检查 architecture 是否正确。

Week 10 - GPT Architecture: 把所有零件组装起来

Week 10 核心目标

前面已经分别理解：

Tokenizer
Embedding
Position
Attention
Feed-Forward
Residual
LayerNorm
Cross Entropy

Week 10 不再引入新的核心数学，而是解决软件架构问题：这些组件怎样组成一个可训练、可保存、可生成文本的 GPT model。

这一步很重要，因为“理解每个零件”和“能保证整个系统正确组合”是两种能力。GPT 中最常见的实现错误往往不是不会写 Softmax，而是 shape、mask、module registration、configuration 或 checkpoint contract 在组件边界上不一致。

1. 先定义 Configuration

```
from dataclasses import dataclass
```

```
@dataclass
class GPTConfig:
    vocab_size: int
    context_length: int = 128
    model_dim: int = 128
    num_heads: int = 4
    num_layers: int = 4
    dropout: float = 0.1
```

这些是 hyperparameters，不是 training 自动学出的 parameters。

它们定义 architecture 的 shape 契约。

Configuration 为什么需要集中定义

这些值会同时影响多个组件。例如 model_dim 必须被 token embedding、position embedding、LayerNorm、Attention、FFN 和 LM Head 共同认可；散落成多个 magic numbers 很容易造成不兼容。

Config 相当于 model 的 schema：

它决定哪些 tensors 可以连接
它决定 state_dict 中 parameter shapes
它决定 checkpoint 能否被重新加载
它决定大致 parameter count 和 compute cost

Hyperparameter 不参与普通 optimizer update，但一旦 training 开始，随意改变 shape 类配置通常会使旧 weights 无法加载。

配套 mini_gpt.py 使用了论文和开源实现中常见的缩写，含义完全对应：

教程语义名	配套代码字段	含义
context_length	block_size	一次 forward 的最大 token 数
model_dim	n_embd	每个 token representation 的宽度
num_heads	n_head	Attention heads 数量

教程语义名	配套代码字段	含义
num_layers	n_layer	Transformer blocks 数量

先用语义名理解 architecture，再看到缩写时做这层映射即可。

2. Configuration 的关键约束

Multi-Head Attention 通常要求：

$$model_dim \bmod num_heads = 0$$

例如：

```
model_dim = 128
num_heads = 4
head_dim = 32
```

代码：

```
assert config.model_dim % config.num_heads == 0
```

如果约束不成立，channels 无法平均拆给每个 head。

这个 constraint 背后的含义是每个 head 获得同样数量的 feature channels。C=128,H=4 时每头 32 维；如果 C=130,H=4，无法用简单 reshape 平均分組。

num_heads 增大并不自动增加总 model dimension：在固定 C 下，heads 更多意味着每个 head 更窄。Architecture 调整总是在“并行视角数量”和“每个视角容量”之间取舍。

3. GPT 的顶层数据流

```
Token IDs [B,T]
↓
Token Embedding + Position Embedding
↓
[B,T,C]
↓
Transformer Blocks × N
↓
Final LayerNorm
↓
LM Head
↓
Logits [B,T,V]
```

如果提供 targets，还会继续：

```
Logits + Targets
↓
Cross Entropy
↓
Scalar Loss
```

为什么同一个 Forward 同时支持 Training 与 Inference

无论是否 training，model 的核心任务都是把 token IDs 变成 logits。Targets 只是可选输入：存在时额外计算 Loss，不存在时直接返回 logits。

这种接口把 architecture 与生命周期逻辑分开：

model.forward 负责 predictions
 training loop 负责 backward 和 update
 generation loop 负责 sampling 和 append

Model 不应在普通 forward 内偷偷调用 optimizer，否则难以测试、复用和执行 inference。

4. Causal Self-Attention Class

为了减少样板代码，可以一次生成 Q、K、V：

一次使用 Linear(C,3C) 与分别使用三个 Linear(C,C) 在表达能力上可以等价。Fused QKV projection 把三个相同 input 的矩阵运算合并，便于底层实现提高效率；chunk(3) 再把最后 dimension 分成 Q、K、V。

```
class CausalSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()

        assert config.model_dim % config.num_heads == 0

        self.num_heads = config.num_heads
        self.head_dim = config.model_dim // config.num_heads

        self.qkv = nn.Linear(
            config.model_dim,
            3 * config.model_dim,
            bias=False,
        )

        self.output = nn.Linear(
            config.model_dim,
            config.model_dim,
            bias=False,
        )

        self.dropout = config.dropout

    def forward(self, x):
        B, T, C = x.shape

        qkv = self.qkv(x)
        q, k, v = qkv.chunk(3, dim=-1)

        q = q.view(B, T, self.num_heads, self.head_dim)
        k = k.view(B, T, self.num_heads, self.head_dim)
        v = v.view(B, T, self.num_heads, self.head_dim)

        q = q.transpose(1, 2)
        k = k.transpose(1, 2)
        v = v.transpose(1, 2)

        attention = F.scaled_dot_product_attention(
            q,
            k,
            v,
            is_causal=True,
            dropout_p=self.dropout if self.training else 0.0,
        )

        attention = attention.transpose(1, 2).contiguous()
        attention = attention.view(B, T, C)

        return self.output(attention)
```

5. 拆分 Heads 时的 Shape

输入：

```
x : [B,T,C]
qkv : [B,T,3C]
```

拆开：

```
q, k, v: [B,T,C]
```

拆 heads 并 transpose：

```
[B,T,C]
→ [B,T,H,D]
→ [B,H,T,D]
```

其中：

$$C = H \times D$$

Attention 后再反向合并：

```
[B,H,T,D]
→ [B,T,H,D]
→ [B,T,C]
```

这是整段 attention 代码最容易出错的地方。

view 与 transpose 分别改变什么

view(B,T,H,D) 把一个 C dimension 解释成 $H \times D$ ，并没有改变元素值。transpose(1,2) 则交换 sequence 与 head axes，让 attention kernel 接收 [B,H,T,D]。

Attention 完成后必须按相反顺序还原。transpose 可能产生非连续内存 view，因此代码在最终 view(B,T,C) 前调用 .contiguous()。这不是新的数学，而是 tensor memory layout 的工程要求。

调试时建议逐行 assert：

```
assert q.shape == (B, H, T, D)
assert attention.shape == (B, H, T, D)
```

否则两个 dimension 大小碰巧相同时，错误 transpose 可能不会立即报错。

6. FeedForward 与 Block

```
class FeedForward(nn.Module):
    def __init__(self, config):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(config.model_dim, 4 * config.model_dim),
            nn.GELU(),
            nn.Linear(4 * config.model_dim, config.model_dim),
            nn.Dropout(config.dropout),
        )

    def forward(self, x):
        return self.net(x)
```

```
class TransformerBlock(nn.Module):
    def __init__(self, config):
        super().__init__()

        self.ln1 = nn.LayerNorm(config.model_dim)
```

```

self.attention = CausalSelfAttention(config)

self.ln2 = nn.LayerNorm(config.model_dim)
self.ffn = FeedForward(config)

def forward(self, x):
    x = x + self.attention(self.ln1(x))
    x = x + self.ffn(self.ln2(x))
    return x

```

每个 block 输入和输出都是 [B,T,C]。

FeedForward 与 Attention 被封装成独立 modules，不只是为了代码整洁。它确保每组 parameters 被注册、可以单独测试，并让 Block 的 forward 清楚表达两条 residual branches。

Architecture code 的阅读顺序可以固定为：先确认输入输出 shape，再确认信息可见性，最后才看优化细节。只要 [B,T,C] contract 和 causal constraint 正确，组件就能稳定堆叠。

7. 完整 GPT Model

```

class MiniGPT(nn.Module):
    def __init__(self, config):
        super().__init__()

        self.config = config

        self.token_embedding = nn.Embedding(
            config.vocab_size,
            config.model_dim,
        )

        self.position_embedding = nn.Embedding(
            config.context_length,
            config.model_dim,
        )

        self.blocks = nn.ModuleList([
            TransformerBlock(config)
            for _ in range(config.num_layers)
        ])

        self.final_norm = nn.LayerNorm(config.model_dim)

        self.lm_head = nn.Linear(
            config.model_dim,
            config.vocab_size,
            bias=False,
        )

    def forward(self, token_ids, targets=None):
        B, T = token_ids.shape

        if T > self.config.context_length:
            raise ValueError("sequence is longer than context length")

        positions = torch.arange(T, device=token_ids.device)

        x = (
            self.token_embedding(token_ids)
            + self.position_embedding(positions)
        )

        for block in self.blocks:
            x = block(x)

        x = self.final_norm(x)

        logits = self.lm_head(x)

        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))

```

```

x = self.final_norm(x)
logits = self.lm_head(x)

loss = None

if targets is not None:
    B, T, V = logits.shape

    loss = F.cross_entropy(
        logits.reshape(B * T, V),
        targets.reshape(B * T),
    )

return logits, loss

```

这段 Model 代码的 Ownership Map

```

Tokenizer   拥有 text ↔ ID 规则
MiniGPT     拥有可训练 modules
Config      拥有 architecture choices
Forward     拥有 ID → logits / loss 的数据流
Training Loop 拥有 optimizer 和 step
Generation  拥有 sampling policy

```

边界清楚后，替换 tokenizer、改变 sampling 或恢复 checkpoint 时更容易判断哪些状态必须一起变化。

8. ModuleList 为什么重要

如果把 blocks 放进普通 Python list，PyTorch 不会自动注册其中的 parameters。

nn.ModuleList 让 framework 知道：

```

这些 blocks 属于 model
它们的 parameters 要出现在 model.parameters()
它们要进入 state_dict
它们要跟随 model.to(device)

```

这是 architecture code 与普通 Python object graph 的一个关键区别。

同理，直接把 Tensor 放进普通 list 也不会自动变成 registered Parameter。框架只有通过 nn.Module、nn.ModuleList、nn.Parameter 等机制才能发现状态。

一个很危险的 bug 是：forward 确实使用了普通 list 中的 layers，loss 也能计算，但 optimizer 看不到它们，weights 永远不更新。检查 `sum(p.numel() for p in model.parameters())` 和 `state_dict().keys()` 可以及时发现 registration 缺失。

9. Parameter 到底在哪里

主要 parameters 来自：

```

Token Embedding
Position Embedding
Q/K/V Projections
Attention Output Projections
Feed-Forward Linear Layers
LayerNorm scale and bias
LM Head

```

查看总数：

```

parameter_count = sum(
    parameter.numel()
)

```

```
for parameter in model.parameters()
)

print(parameter_count)
```

所谓“模型有多少 parameters”，就是把所有可训练 tensors 的元素数量相加。

Parameter count 表示模型需要保存和训练多少可调数字，不等于一次 forward 的 operation count，也不等于 checkpoint 的精确字节数。后者还受 dtype、optimizer state、metadata 等影响。

例如 float32 parameter 通常占 4 bytes；AdamW 还可能为每个 parameter 保存两组移动统计，所以 training memory 远大于只存 model weights 的 inference memory。

10. 手算主要 Parameter Count

假设：

V = vocabulary size
T = context length
C = model dimension
N = layer count

大致：

Token Embedding

$$V \times C$$

Position Embedding

$$T \times C$$

每个 Block 的 Attention

QKV projection 约：

$$C \times 3C = 3C^2$$

Output projection：

$$C \times C = C^2$$

每个 Block 的 FFN

$$C \times 4C + 4C \times C = 8C^2$$

所以每个 block 的主要 matrix parameters 约：

$$12C^2$$

这解释了为什么增加 model dimension 会快速增加 parameter count。

带入一个具体配置

若 $C=128$ ，每个 block 的主要 matrix parameters 约为：

$$12C^2 = 12 \times 128^2 = 196,608$$

四个 blocks 约 786,432 个，尚未计算 embeddings、LayerNorm 和 biases。若把 C 从 128 翻倍到 256， C^2 项会约变为四倍，不是两倍。

因此扩大 width 是非常昂贵的 architecture decision。Config 中看似简单的一个数字，会同时改变 memory、compute、data requirement 和 training time。

11. Weight Tying: 知道用途即可

Token Embedding 把 token ID 映射到 model space；LM Head 把 model space 映射回 vocabulary。

有些语言模型共享两者的 weight：

```
self.lm_head.weight = self.token_embedding.weight
```

这叫 Weight Tying，可以减少 parameters，并在输入输出 representation 之间建立联系。教学项目可以选择使用或不使用，核心 architecture 不依赖它。

为什么能够共享？两者 shapes 都是 $[V, C]$ （Linear weight 在实现中按 $\text{output} \times \text{input}$ 存储）。输入端学习 token 的 representation，输出端学习 representation 与 token candidates 的匹配，共享能减少重复参数。

Weight tying 是设计选择，不是“必须正确”的唯一方案。是否使用要与 checkpoint 定义一致，不能加载后临时改变而期待得到相同输出。

12. 正确性首先检查 Shape

最小 smoke test：

```
config = GPTConfig(vocab_size=100)
model = MiniGPT(config)

inputs = torch.randint(0, 100, (4, 16))
targets = torch.randint(0, 100, (4, 16))

logits, loss = model(inputs, targets)

assert logits.shape == (4, 16, 100)
assert loss.ndim == 0
```

它验证：

```
[B,T] → [B,T,V]
loss → scalar
```

Smoke test 解决的是结构正确性，不证明 model 已经学会语言。Random inputs 和 random targets 仍应满足 shape、finite loss 和 backward；这些基础条件不成立时，继续长时间 training 没有意义。

建议再检查：

```
assert torch.isfinite(loss)
loss.backward()
assert any(p.grad is not None for p in model.parameters())
```

从 pipeline test 到 learning test 分层验证，定位问题会容易很多。

13. 必须检查 Causal 性

一个位置的 output 不应该受到未来 token 影响。

测试思路：

1. 建立两个 sequences;
2. 保持前半部分相同;
3. 只修改未来 positions;
4. 比较较早位置的 logits;
5. 较早位置应该保持一致（在 eval 模式且无 dropout 时）。

如果未来 token 改变了过去位置的 logits，说明 causal mask 有问题。

为什么必须在 eval() 下测？Dropout 会让两次 forward 即使输入相同也产生随机差异，掩盖 causal comparison。正确测试应固定 model state、关闭 dropout，并只比较位于修改点之前的 logits。

这是比“看 mask matrix 好像正确”更有力的行为测试：它直接验证 architecture 对信息泄漏的最终承诺。

14. Architecture 与 State Dict

保存 weights：

```
torch.save(model.state_dict(), "model.pt")
```

加载时必须先建立兼容 architecture：

```
model = MiniGPT(config)
model.load_state_dict(torch.load("model.pt"))
```

Checkpoint 不是完整程序。它需要匹配：

```
model code
configuration
state dict
tokenizer
```

state_dict 只保存命名 tensors，不保存 class 的完整 forward 逻辑。如果代码改变了 residual 顺序，即使 parameter names 和 shapes 还能加载，model function 也可能已经不同。

可靠 checkpoint 应记录 config、tokenizer identity、training step、必要的软件版本或代码版本。继续 training 时还需 optimizer state，才能延续 AdamW 已累积的统计。

15. 从 Week 2 的公式到 GPT

Week 2：

$$\hat{y} = wx + b$$

GPT 中到处仍然是：

$$Y = XW + b$$

区别只是：

- x 和 w 变成高维 tensors;
- Linear calculations 被 Attention、Activation、Residual 和 Normalization 组合;
- Loss 变成 vocabulary classification 的 Cross Entropy;
- parameters 数量从几个增加到许多。

Training 原理没有换掉。

16. Week 10 最应该理解的 7 件事

1. Config 定义 architecture 的 shape 契约。
2. Token 和 position embeddings 把 $[B,T]$ 变成 $[B,T,C]$ 。
3. 每个 Transformer Block 保持 $[B,T,C]$ 。
4. LM Head 把 $[B,T,C]$ 变成 $[B,T,V]$ 。
5. Targets 存在时计算 scalar Cross Entropy loss。
6. ModuleList 确保 child modules 被 PyTorch 注册。
7. Model、config、state dict 和 tokenizer 必须匹配。

17. Week 10 → Week 11

Architecture 已经完整，但随机初始化的 model 还不会生成有意义文本。

Week 11 要真正运行：

```
get_batch
↓
forward
↓
loss
↓
backward
↓
optimizer.step
```

然后学习如何 validation、保存 checkpoint，以及用 temperature 和 top-k 逐 token generation。

Week 11 - Training 与 Inference: 让 Mini GPT 学会生成文本

Week 11 核心目标

Week 10 完成了 architecture。

Week 11 关注模型生命周期的两个阶段：

Training

- 使用 targets 计算 loss
- backward
- 更新 parameters

Inference

- 没有 targets
- 不 backward
- parameters 不改变
- 逐 token 生成

本周重点：Training Loop、Validation、AdamW、Checkpoint、Temperature、Top-k Sampling。

这一章要区分三类状态：model parameters 是被学习的长期状态；optimizer statistics 是帮助更新的训练状态；当前 batch 的 activations/gradients 是一次 step 的临时状态。理解 ownership 后，保存、恢复和 inference 的差异会自然清楚。

1. 一个 Batch 提供很多训练信号

Inputs 和 targets:

inputs : [B,T]
targets: [B,T]

每个位置都是一个 next-token prediction task。

例如 B=4、T=128：

$$4 \times 128 = 512$$

一个 batch 同时提供 512 个 target tokens。

Model 输出：

logits: [4,128,V]

Cross Entropy 对这些位置的 loss 求平均，得到一个 scalar。

求平均的作用是让 loss scale 不随着 $B \times T$ 线性增大。这样 batch 或 context length 改变时，learning rate 的含义相对稳定。每个位置仍有自己的 prediction error；Backward 会把所有位置对共享 parameters 的 gradients 汇总。

一个 parameter 会在很多 token positions 被重复使用，因此本次 gradient 表示它对整个 batch 平均目标的总责任，而不是某一个 token 的单独意见。

2. Step 与 Epoch

Training Step

处理一个 batch，并执行一次 parameter update。

Epoch

模型完整遍历 dataset 一次。

随机从连续 token stream 采样的语言模型训练通常更常按 step 管理：

```
step 1
step 2
...
step 5000
```

因为“完整看完一次”在随机切块场景下不总是清晰。

Step 是 optimizer state 发生一次变化的最小单位。一个 forward/backward 如果没有 optimizer.step(), 只计算了 gradient, 并没有完成训练更新。

Epoch 更适合能明确枚举所有 samples 的 dataset。连续 token stream 若随机抽取 overlapping windows, 某些 token 可能被多次看到、某些暂时没抽到, 因此用 total steps 或 processed tokens 更容易准确描述训练进度。

3. 为什么常用 AdamW

Week 2 使用最简单 SGD:

$$\theta := \theta - \alpha g$$

AdamW 会为每个 parameter 维护 gradient 的移动统计, 并使用 decoupled weight decay。

直觉上:

SGD → 所有 parameters 直接按当前 gradient 更新

AdamW → 根据近期 gradient 行为, 自适应每个 parameter 的步幅

我们不需要现在推导完整公式。核心仍然没有改变: gradient 告诉 optimizer 怎样更新 parameter。

AdamW 为什么比“直接乘当前 Gradient”更稳定

SGD 只看当前 gradient; 某个 batch 的 noise 可能让方向突然变化。AdamW 为每个 parameter element 维护两组移动统计:

first moment → gradient 的平滑方向, 类似 momentum

second moment → gradient 平方的平滑尺度

更新时, 长期一致的方向会被保留; 经常出现巨大 gradient 的 parameter 会得到相对缩小的有效步幅。于是不同 parameters 可以拥有自适应 update scale。

简化地看:

$$m_t \approx \text{movingAverage}(g_t)$$

$$v_t \approx \text{movingAverage}(g_t^2)$$

$$\text{update} \propto \frac{m_t}{\sqrt{v_t} + \epsilon}$$

这里仍然全部建立在 Backprop 产生的一阶 gradient 上, 没有要求完整二阶导数。

W 代表什么

AdamW 中的 W 指 decoupled Weight Decay。除了按 gradient 更新，还让部分 weights 逐步向 0 收缩，作为 regularization。它与把 L2 penalty 混进 Adam gradient 的效果并不完全相同，因此单独命名为 AdamW。

通常不会对所有 parameter types 使用相同 decay，例如 biases 和 LayerNorm parameters 常被排除。教学代码先使用统一配置，重点理解其目的。

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4,
    weight_decay=0.01,
)
```

4. 一个标准 Training Step

```
inputs, targets = get_batch(
    train_data,
    batch_size,
    context_length,
)

inputs = inputs.to(device)
targets = targets.to(device)

optimizer.zero_grad(set_to_none=True)

logits, loss = model(inputs, targets)

loss.backward()

torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=1.0,
)

optimizer.step()
```

顺序：

```
Batch
↓
Zero Gradients
↓
Forward
↓
Loss
↓
Backward
↓
Optional Gradient Clipping
↓
Optimizer Step
```

为什么这个顺序不能打乱

Forward 必须使用同一版本的 parameters 产生 prediction；Backward 必须针对这次 forward 的 graph 计算 gradients；Optimizer 最后才用这些 gradients 修改 parameters。如果在 backward 前先 step，forward values 与 parameter state 不再一致。

Gradient clipping 必须放在 backward() 之后，因为之前还没有 gradients；也要放在 optimizer.step() 之前，否则过大的 update 已经发生。

`zero_grad()` 则要在本 `step backward` 之前完成，避免旧 batch 残留。每一行都对应明确生命周期边界。

5. 完整 Training Loop

```
for step in range(max_steps):
    model.train()

    inputs, targets = get_batch(
        train_data,
        batch_size,
        context_length,
    )

    inputs = inputs.to(device)
    targets = targets.to(device)

    optimizer.zero_grad(set_to_none=True)

    _, loss = model(inputs, targets)

    loss.backward()

    torch.nn.utils.clip_grad_norm_(
        model.parameters(),
        1.0,
    )

    optimizer.step()

    if step % evaluation_interval == 0:
        validation_loss = estimate_validation_loss()

        print(
            step,
            loss.item(),
            validation_loss,
        )
```

这就是前 10 周知识最终落入的循环。

Training loop 本身不学习知识；它只是反复执行同一状态转换：

```
(parameters, optimizer state, batch)
    ↓
(updated parameters, updated optimizer state)
```

真正定义“学什么”的是 dataset 与 loss，定义“能表示什么”的是 architecture，定义“怎样走”的是 optimizer。三者职责不同。

6. 初始 Loss 大约应该是多少

如果模型初始化后对 V 个 tokens 接近均匀猜测：

$$p \approx \frac{1}{V}$$

Cross Entropy：

$$Loss \approx -\log\left(\frac{1}{V}\right) = \log(V)$$

例如 vocabulary size 是 100:

$$\log(100) \approx 4.605$$

如果初始 loss 与这个数量级极不相符，应先检查 target shift、vocabulary size、logits shape 或 loss calculation。

这个 baseline 很有用，因为它是能在训练前预测的 sanity check。若 $V=100$ ，初始 loss 是 0.1，通常不是模型天生很聪明，而更可能是 label leakage、loss shape 或 reduction 有问题；若 loss 极大或 NaN，则可能有 logits scale、dtype 或 numerical instability 问题。

7. Validation 为什么必要

Training Loss 只说明模型越来越适合训练 batches。

Validation Loss 回答：

模型对没有参与 parameter update 的数据是否也变好？

常见情况：

train loss ↓
validation loss ↓
→ 正常学习

train loss ↓
validation loss ↑
→ 可能 overfitting

Validation 为什么不能更新 Parameters

Validation 的价值来自“模型没有用这些 targets 调整自己”。如果对 validation batch 执行 optimizer step，它就变成 training data，指标失去独立性。

Validation loss 也不是最终产品质量的全部。它衡量同一 objective 上的 generalization；生成连贯性、安全性、事实性等还需要额外 evaluation。但在教学训练循环中，它是最直接的过拟合信号。

8. 正确的 Validation 模式

```
@torch.no_grad()
def estimate_loss(model, source, eval_steps):
    model.eval()

    losses = []

    for _ in range(eval_steps):
        inputs, targets = get_batch(
            source,
            batch_size,
            context_length,
        )

        inputs = inputs.to(device)
        targets = targets.to(device)

        _, loss = model(inputs, targets)
        losses.append(loss.item())

    model.train()

    return sum(losses) / len(losses)
```

关键点：

- `model.eval()` 关闭 Dropout 等 training-only 行为；
- `torch.no_grad()` 不建立 backward graph；
- 多个 batches 求平均，减少随机波动；
- 评估后恢复 `model.train()`。

评估多个 batches 是为了降低 sampling noise。单个 random validation window 可能碰巧简单或困难；平均后才能比较长期趋势。实际记录时还应保持 evaluation protocol 不变，否则不同时间点的数字不可直接比较。

9. Gradient Clipping

如果 gradient norm 突然很大，一次 update 可能破坏 parameters。

```
torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    max_norm=1.0,
)
```

它会在整体 gradient norm 超过阈值时按比例缩小 gradients。

Gradient Clipping 是安全措施，不会修复错误 architecture、错误数据或不合适的 learning rate。

`clip_grad_norm_` 先把所有 selected gradients 看成一个长向量，计算整体 norm。若 norm 超过阈值，就按同一比例缩小，使方向大体保持不变、长度受到限制。

它限制的是 update 前的 gradient signal，不是强行把每个 parameter value 限制在某个范围。频繁触发 clipping 可能提示 learning dynamics 不稳定，应该进一步检查 learning rate、initialization 或 data。

10. Learning Rate 仍然是关键

太大 → loss 波动、发散、NaN
太小 → loss 下降非常慢

调试顺序建议：

1. 先确认 model 能 overfit 一个 batch；
2. 再观察 training loss 是否稳定下降；
3. 再调整 learning rate、batch size 和 model size；
4. 最后才加入复杂 schedule。

不要用复杂 optimizer 设置掩盖基础 bug。

Learning-rate schedule 会让步幅随训练阶段变化，例如先 warm up，再逐渐 decay。它有实际价值，但前提是固定 learning rate 下 pipeline 已经能学习。否则 schedule 只会增加难以定位的变量。

11. Overfit One Batch Test

这是最有价值的训练测试之一。

固定一个很小 batch，反复训练：

```
inputs, targets = get_batch(
    train_data,
    batch_size=2,
    context_length=8,
)

for step in range(1000):
    optimizer.zero_grad(set_to_none=True)
    _, loss = model(inputs, targets)
```



```
loss.backward()
optimizer.step()
```

一个有足够 capacity 的小模型应该能把这个固定 batch 的 loss 降得很低。

如果做不到，优先检查：

- targets 是否错开一位；
- causal mask 是否正确；
- parameters 是否注册进 optimizer；
- gradients 是否存在；
- train/eval 状态；
- dtype、device 和 shape。

为什么这个测试如此有效？Generalization 很难，但记住一个很小的固定 batch 对有足够 capacity 的 model 应该很容易。若连它都做不到，问题更可能在 data flow、gradient flow 或 update，而不是“数据量不够”。

反过来，成功 overfit one batch 也不证明模型能 generalize。它只证明最核心的训练管线能够把 Loss 的信号传到 parameters 并改变 predictions。

12. Checkpoint 应保存什么

如果只做 inference：

```
torch.save({
    "model": model.state_dict(),
    "config": config,
    "tokenizer": tokenizer_state,
}, "mini_gpt.pt")
```

如果以后继续 training，还需要：

```
torch.save({
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "config": config,
    "tokenizer": tokenizer_state,
    "step": step,
}, "mini_gpt_checkpoint.pt")
```

Optimizer state 会保存 AdamW 的移动统计。缺少它也能重新训练，但不能精确延续原训练状态。

Checkpoint 是“可恢复训练状态的快照”，不仅是一个 weights 文件。要精确 resume，至少需要 model、optimizer、step、config、tokenizer，若使用 scheduler 或 mixed precision，还要保存它们的状态。

若只做 inference，可以省去 optimizer state，大幅减小文件。保存内容应由恢复目标决定，而不是把所有 runtime objects 盲目序列化。

13. Inference 与 Training 的区别

Inference：

```
model.eval()

with torch.no_grad():
    logits, _ = model(token_ids)
```

不提供 targets，不计算 loss，不 backward，不 optimizer step。

Parameters 保持不变。

Training 和 inference 使用同一组 learned parameters，但执行环境不同：inference 不需要 targets、activation graph、parameter gradients 或 optimizer statistics，因此可以更节省内存，并针对 latency、batching、KV cache 做优化。

model.eval() 处理 Dropout 等 layer behavior，torch.no_grad() 关闭 graph recording；二者职责不同，通常同时使用。

14. 为什么生成时只取最后一个位置

Model 输出：

```
logits: [B,T,V]
```

每个位置都有“它的下一个 token”分数。

当前要继续整段 prompt，只需要最后位置的 prediction：

```
next_token_logits = logits[:, -1, :]
```

Shape：

```
[B,V]
```

前面 positions 的 logits 不是无用，它们在 training 时提供并行 targets；generation 时那些 predictions 已经对应过去时刻，当前只需要 context 末尾位置对“下一个 token”的分布。

15. Temperature

生成前调整 logits：

$$z'_i = \frac{z_i}{\text{temperature}}$$

然后 Softmax。

temperature < 1 → 分布更尖锐、更保守

temperature = 1 → 原分布

temperature > 1 → 分布更平、更随机

Temperature 不是“创造力参数”的完整定义。它只是调整采样分布的相对尖锐程度。

Temperature 不改变 logits 的排序，只改变差距。小于 1 时高分候选更占优势；大于 1 时分布更平。若 temperature 接近 0，数值会非常尖锐，因此实现通常需要验证它必须大于 0。

它也不会修复 bad model：如果合理 token 本来就没有较高 logits，改变 temperature 只能重新分配已有候选的概率。

16. 手算 Temperature 的直觉

原 logits：

[2, 1, 0]

当 temperature 为 0.5：

[4, 2, 0]

最大 logit 的优势被放大。

当 temperature 为 2：

[1, 0.5, 0]

不同 logits 更接近，低概率 tokens 更有机会被选中。

17. Top-k Sampling

Top-k 只保留概率最高的 k 个候选，其余设置为负无穷：

```
values, _ = torch.topk(logits, k)
threshold = values[:, -1].unsqueeze(-1)

logits = logits.masked_fill(
    logits < threshold,
    float("-inf"),
)
```

然后 Softmax 和 sampling。

它减少极低概率 token 造成的离谱输出，同时保留一定随机性。

Top-k 与 temperature 分工不同：temperature 改变保留候选之间的相对概率，top-k 直接删除排名之外的候选。二者常组合使用，但 k 太小会让输出僵硬，太大则几乎不经过滤作用。

Top-p 是另一种动态候选策略：保留累计 probability 达到阈值的最小集合。本课程只需详细掌握 top-k，知道 top-p 解决“固定 k 不适应分布宽窄”的问题即可。

18. 完整 Generation Loop

```
@torch.no_grad()
def generate(
    model,
    token_ids,
    max_new_tokens,
    temperature=1.0,
    top_k=None,
):
    model.eval()

    for _ in range(max_new_tokens):
        context = token_ids[:, -model.config.context_length:]

        logits, _ = model(context)
        logits = logits[:, -1, :]
        logits = logits / temperature

        if top_k is not None:
            values, _ = torch.topk(
                logits,
                min(top_k, logits.size(-1)),
            )
            cutoff = values[:, -1].unsqueeze(-1)

            logits = logits.masked_fill(
                logits < cutoff,
                float("-inf"),
            )

        probabilities = F.softmax(logits, dim=-1)

        next_id = torch.multinomial(
            probabilities,
```

```

    num_samples=1,
)

token_ids = torch.cat(
    [token_ids, next_id],
    dim=1,
)

return token_ids

```

19. 为什么要截断 Context

Model 只有固定 context length:

```
context = token_ids[:, -context_length:]
```

超过窗口的早期 tokens 不会直接进入本次 forward。

这也解释了 context window 的实际含义：模型一次计算能直接查看的最大 token 数。

截断不等于模型主动总结了早期内容。被移出窗口的 tokens 不再直接参与本次 forward，除非它们的信息已经通过其他机制压缩进仍在窗口中的文字或外部 state。

朴素 generation 每一步重算整个截断窗口；KV Cache 可以复用过去 attention 的 Keys/Values，提高 inference 效率，但不会扩大模型本身的 context limit。

20. 常见 Bug Checklist

Loss 不下降

- targets 是否 shift 一位？
- optimizer.zero_grad() 是否调用？
- loss.backward() 是否调用？
- parameters 是否交给 optimizer？
- learning rate 是否极端？

输出提前看到答案

- causal mask 是否正确？

加载后乱码或完全异常

- tokenizer 是否与 checkpoint 匹配？
- encoding 是否一致？

Device Error

- model、inputs、targets 是否在同一 device？

Validation 波动很大

- 是否平均多个 batches？
- 是否调用 model.eval()？

21. Week 11 最应该理解的 7 件事

1. 一个 $[B,T]$ batch 提供 $B \times T$ 个 next-token targets。
2. AdamW 仍然使用 Backprop 得到的 gradients 更新 parameters。
3. Validation 不参与 parameter update。
4. 初始 random loss 约为 $\log(V)$ 。
5. Overfit one batch 是检查训练管线的关键测试。
6. Inference 不计算 gradients，不改变 parameters。
7. Generation 是“最后位置 logits $\rightarrow$ sampling $\rightarrow$ append $\rightarrow$ repeat”。

22. Week 11 $\rightarrow$ Week 12

所有核心零件已经就位。

Week 12 不再孤立讲组件，而是从一个字符串开始，完整追踪：

```
Text
→ Tokens
→ Batch
→ Embeddings
→ Transformer
→ Logits
→ Loss
→ Backprop
→ Checkpoint
→ Generated Text
```

Week 12 - 完整 Mini GPT: 从 Text 到 Training 再到 Generation

Week 12 核心目标

这一周的目标不是再学一组名词，而是把前 11 周连成一个可运行系统。

最终主线：

```
Corpus
↓
Tokenizer
↓
Token Stream
↓
Training Batches
↓
Mini GPT
↓
Cross Entropy Loss
↓
Backpropagation
↓
AdamW Update
↓
Checkpoint
↓
Autoregressive Generation
```

这一周的重点不是把所有代码背下来，而是建立端到端 ownership：每一步都知道输入是谁产生的、输出交给谁、为什么需要它，以及出现异常时应该在哪里检查。只要能沿这条链定位一个 token 的变化如何影响最终 Loss，就真正把前面的章节连起来了。

1. 项目结构

```
AI_First_Principles_12_Weeks/
├── mini_gpt.py
├── sample_corpus.txt
└── requirements.txt
```

安装：

```
python -m pip install -r requirements.txt
```

先确认 PyTorch：

```
python -c "import torch; print(torch.__version__)"
```

2. 第一次运行先验证 Pipeline

不要一开始就追求高质量文字。

先用小配置验证：

```
Tokenizer 能 encode/decode
Batch shape 正确
Forward 能运行
Loss 是 finite scalar
Backward 能得到 gradients
```

Optimizer 能更新 parameters
 Checkpoint 能保存与加载
 Generation 能输出 token

这相当于端到端 smoke test。

每一项都对应一个可观察断点：

encode/decode → 数据协议
 batch shape → 训练样本
 forward → architecture
 finite loss → objective 数值
 gradients → backward connectivity
 step → parameter mutation
 checkpoint → 可恢复性
 generation → inference contract

不要把“loss 能打印出来”误认为整个 pipeline 正确；loss 可能计算成功，但 parameters 没有注册、targets 泄漏未来，或 generation 使用了错误 tokenizer。

3. 从一个字符串开始

假设 corpus：

机器学习通过数据调整参数。
 神经网络通过梯度下降降低损失。

Tokenizer：

```
token_ids = encode(text)
```

得到：

```
[12, 7, 31, 4, ...]
```

然后：

```
data = torch.tensor(token_ids, dtype=torch.long)
```

Shape：

```
[N]
```

其中 N 是总 token 数。

从这里开始，模型不再看到原始字符串，只看到整数序列。为了保留可追溯性，实际项目应同时保存 tokenizer vocabulary、编码版本和原始 corpus 的来源信息；否则后面看到一个 ID 很难判断它代表什么。

4. 形成 Training Batch

设：

```
B = 4  
T = 8
```

```
inputs, targets = get_batch(  
    train_data,  
    batch_size=4,  
    context_length=8,  
)
```

Shape：

```
inputs : [4,8]  
targets: [4,8]
```

每一行都是一个连续文本窗口，target 比 input 向后移动一个 token。

inputs 和 targets 看似只差一个 slice，却定义了整个 language-model objective。若把 target 错位、反向或与 input 完全相同，后面的 transformer 可能仍然产生合法 tensor，问题只会在模型学到错误行为时暴露。

5. Embedding Lookup

假设：

V = 80
C = 64

Token Embedding:

[4,8] → [4,8,64]

Position Embedding:

[8] → [8,64]

相加后通过 broadcasting:

[4,8,64] + [8,64] → [4,8,64]

现在每个 token 同时包含 token identity 和 position information。

这一步把两种互补信息合并：token embedding 回答“是什么”，position embedding 回答“在哪里”。二者相加而不是拼接，是因为后续 Block 的接口固定为 C 维；如果拼接，model dimension 会改变，所有 projection shapes 都必须重新定义。

6. 进入 Transformer Block

每个 block:

```
x [4,8,64]
↓ LayerNorm
Attention [4,8,64]
↓ Residual Add
[4,8,64]
↓ LayerNorm
FeedForward [4,8,64]
↓ Residual Add
[4,8,64]
```

无论堆叠多少 blocks，外部 shape 不变。

这就是可组合 architecture 的关键：每个 Block 内部可以复杂，外部仍遵守同一 contract。像软件中的 middleware chain，调用者不需要知道内部每个步骤，只要输入输出协议稳定，就可以重复组合。

7. Q/K/V Shape Trace

假设：

num_heads = 4
head_dim = 16
model_dim = 64

```
x [4,8,64]
qkv projection [4,8,192]
split q, k, v each [4,8,64]
split heads each [4,4,8,16]
attention scores [4,4,8,8]
attention output [4,4,8,16]
merge heads [4,8,64]
```

Score matrix 中：


```
4 batches
× 4 heads
× 8 query positions
× 8 key positions
```

Causal mask 会把每个 query 右侧的未来 key positions 设为不可见。

这张 [4,4,8,8] score tensor 里，最后两个 dimensions 是“query position × key position”。检查 causal 性时要沿最后一维确认右上角为 masked；把 mask 广播到 batch 和 heads 后，每个样本、每个 head 都遵守同一信息边界。

8. LM Head 产生 Logits

Transformer 最终 output:

```
[4,8,64]
```

LM Head:

```
Linear(64 → 80)
```

得到:

```
logits [4,8,80]
```

每个 batch、每个 position 都为 vocabulary 中 80 个 tokens 提供分数。

80 不是“模型认为有 80 个答案”，而是 vocabulary 中每个候选都拥有一个相对分数。Cross Entropy 会拿 target ID 作为索引，要求对应候选相对于其他候选获得足够高的 probability。

9. Cross Entropy 产生一个 Loss

Reshape:

```
logits [4,8,80] → [32,80]
targets [4,8] → [32]
```

```
loss = F.cross_entropy(
    logits.reshape(32, 80),
    targets.reshape(32),
)
```

输出是 scalar:

```
loss shape = []
```

这 32 个 token predictions 共同决定一次 update。

一次 scalar loss 把 32 个局部训练信号聚合起来；Backprop 再它们对共享 weights 的贡献相加或平均。于是同一个 attention weight 可能同时受到同一 batch 中许多 positions 的反馈。

10. Backward Trace

```
loss.backward()
```

Gradient 反向流过:

```
Loss
↓
LM Head
↓
Final LayerNorm
↓
Last Transformer Block
```

```

↓
...
↓
First Transformer Block
↓
Token / Position Embeddings

```

Autograd 使用 Chain Rule 为所有参与本次 forward 的 parameters 计算 gradients。

这就是 Week 4 的两层手算在更大 graph 上自动执行。

如果某个 module 的 parameter 没有出现在这条 graph，loss.backward() 不会神奇地为它创造 gradient。检查 .grad is None 是发现断开连接、错误 detach 或未使用 module 的直接方法。

11. 为什么只有部分 Embedding Rows 更新

Embedding lookup 只读取当前 batch 中出现的 token IDs。

因此当前 step 的 loss 只依赖这些 rows；没有被 lookup 的 rows 通常不会从这条路径收到 gradient。

随着不同 batches 被采样，更多 token rows 逐步参与训练。

这是一种稀疏的 lookup gradient：当前 step 只更新被读取的 rows，但 Transformer 与 LM Head 的很多 parameters 会从所有 positions 收到密集 gradient。两种更新模式同时存在，解释了为什么 embedding 的某些 rows 在单步检查中保持没有 gradient，并不代表 embedding 永远不会学习。

12. Optimizer Step

```

optimizer.zero_grad(set_to_none=True)

_, loss = model(inputs, targets)
loss.backward()

torch.nn.utils.clip_grad_norm_(
    model.parameters(),
    1.0,
)

optimizer.step()

```

这个循环做的仍然是：

注意 optimizer.step() 读取的是当前 step 已计算的 gradients；它不会重新计算 forward。若在 step 后忘记 zero_grad()，下一次 batch 的 gradients 会累加到旧值上，更新就不再代表当前 batch 的 loss。

Parameters → Prediction → Loss → Gradients → Better Parameters

13. Training 前后的分布变化

Training 前，next-token probabilities 接近随机。

Training 后，模型逐渐提高语料中合理 continuation 的 probability。

例如看到：

神经网络通过

模型可能逐渐更倾向：

梯度
训练
参数

而不是无关字符。

模型不是保存一套 if/else rules，而是通过大量 parameter values 表示这些统计关系。

因此“训练后能生成语料中的句子”要谨慎解释。小 corpus 上的 Mini GPT 可能只是记住局部 patterns，不能据此推断它获得了通用语言能力。用 validation loss、新 prompt 和不同随机 seed 检查，才能区分学习到的统计规律与简单 memorization。

14. Autoregressive Generation Trace

Prompt:

机器学习

流程:

```

encode prompt
↓
forward
↓
take last-position logits
↓
temperature / top-k
↓
softmax
↓
sample next token
↓
append token
↓
repeat
↓
decode all IDs
  
```

每次只生成一个新 token，但一次 forward 会处理窗口内所有 context tokens。

这与 training 的并行方式形成对比：training 已知完整 target sequence，可以同时计算所有 positions；generation 的未来 tokens 尚不存在，只能顺序追加。因此 generation 通常受 token-by-token latency 影响，而不是只看一次 forward 的吞吐量。

15. 为什么当前实现会重复计算过去

朴素 generation 每次都重新计算整个 context:

```

Step 1: [A]
Step 2: [A,B]
Step 3: [A,B,C]
  
```

前面 tokens 的 Keys 和 Values 被反复计算。

KV Cache 会保存过去 layers 的 Keys 和 Values，只计算新 token 的部分。

这是重要 inference optimization，但不改变训练时学习到的 Attention 关系，也不是完成教学版 Mini GPT 的必要条件。

KV Cache 保存的是各层历史 K/V tensors，而不是最终文字答案。新 token 到来时，query 仍需与历史 keys 比较，并把历史 values 汇总进新 output；只是历史 keys/values 不必重新投影。理解这一点能避免把 cache 误认为“模型记忆”或参数更新。

16. 三阶段实验方法

Stage 1: Pipeline Test

小 model
少量 steps
目标: 所有 shape 和流程正确

Stage 2: Learning Test

固定一个 batch
反复训练
目标: loss 能显著下降

Stage 3: Generalization Experiment

train / validation split
更长训练
定期评估
目标: 观察 train 与 validation loss

每个 stage 都有明确的失败解释:

Stage 1 失败 → shape、dtype、mask、registration 或 checkpoint bug
Stage 2 失败 → gradient、optimizer、learning rate 或 data shift bug
Stage 3 失败 → data quality、capacity、regularization 或 generalization 问题

不要在 Stage 1 失败时盲目增大 model; 参数越多只会让错误更难观察。

这种分阶段方式比一次启动大训练更容易定位问题。

17. 换成自己的 Corpus

基本步骤:

1. 使用 UTF-8 保存文本;
2. 清理明显损坏或不希望学习的内容;
3. 重新建立 tokenizer;
4. 确认 encode/decode round-trip;
5. 创建 train/validation split;
6. 从小配置开始;
7. 保存 tokenizer 与 checkpoint。

模型只会从提供的数据分布中学习。增加数据量不等于自动提高数据质量。

换 corpus 时还要问三个问题:

1. 文本是否包含你希望模型模仿的风格与事实?
2. train/validation 是否真正独立?
3. tokenizer 是否能稳定覆盖文本中的 Unicode、标点和边界?

如果数据包含大量模板、重复段落或乱码, 模型会忠实学习这些模式。数据清理不是训练外的行政工作, 而是直接决定 Loss 可以奖励什么行为。

18. 从 Character Tokenizer 升级 Subword

Model 接口基本不用改变:

input 仍然是 [B,T] token IDs
output 仍然是 [B,T,V] logits

需要改变的是:

- tokenizer implementation;

- vocabulary size;
- encode/decode rules;
- checkpoint 中的 tokenizer metadata;
- Token Embedding 和 LM Head 的 first/last dimension。

这说明 Tokenizer 与 Model 是独立组件，但它们通过 vocabulary IDs 严格耦合。

独立意味着可以替换实现；耦合意味着替换时必须同步更新 v、embedding rows、LM Head columns 和 checkpoint metadata。接口独立不等于语义可以任意互换。

19. 如何安全地扩大 Model

可以逐步增加：

```
model_dim
num_heads
num_layers
context_length
batch_size
training steps
```

每次只改变少量因素，并记录：

- parameter count;
- training/validation loss;
- step time;
- memory usage;
- sample quality。

更大的 model 需要更多 data 和 compute，不保证自动得到更好的 validation 结果。

扩大时应一次只改变少量变量，并保留可比较的 baseline。否则即使 loss 改善，也无法知道原因是 width、depth、context、training steps 还是更合适的 learning rate。

20. Mini GPT 的测试清单

Tokenizer

```
assert decode(encode(sample)) == sample
```

Batch

```
assert inputs.shape == targets.shape
assert torch.equal(inputs[:, 1:], targets[:, :-1])
```

Model

```
assert logits.shape == (
    batch_size,
    context_length,
    vocab_size,
)
assert loss.ndim == 0
```

Gradient

```
loss.backward()
assert any(
```

```
p.grad is not None
for p in model.parameters()
)
```

Checkpoint

保存、重新加载，并确认相同 input 在 eval() 下产生相同 logits。

Causal Test

修改未来 tokens，不应改变过去 positions 的 logits。

21. 十二周知识怎样汇合

Week 1

Vector、Matrix、Shape、Dot Product，让公式能被翻译成代码。

Week 2

Prediction、Loss、Gradient Descent，解释机器怎样学习 parameters。

Week 3

把 $wx+b$ 扩展成 neuron、activation、layer 和 neural network。

Week 4

用 Chain Rule 把 Loss 的责任反向分配给所有 parameters。

Week 5

使用 Tensor、Autograd、nn.Module 和 Optimizer 自动执行训练。

Week 6

把 text 转成 embedding，并用 Cross Entropy 学习 next-token prediction。

Week 7

Attention 让 tokens 动态查找相关 context。

Week 8

把 Attention、FFN、Residual 和 LayerNorm 组成 Transformer Block。

Week 9

Tokenizer、Vocabulary 和 batching 建立完整数据入口。

Week 10

把所有 components 组装成 Mini GPT architecture。

Week 11

训练、验证、保存 checkpoint，并逐 token generation。

Week 12

完成从 corpus 到 generated text 的 end-to-end system。

22. 最终 Mental Model

```

Text
↓
Tokenizer
↓
Token IDs
↓
Embeddings
↓
Transformer Blocks
↓
Logits
↓
Cross Entropy Loss
↓
Backpropagation
↓
Optimizer
↓
Updated Parameters

```

Generation:

```

Prompt
↓
Same Model Forward
↓
Next-token Distribution
↓
Sample One Token
↓
Append and Repeat

```

23. 最终理解测试

如果你可以用自己的语言回答下面问题，说明地基已经建立：

1. Token ID 为什么不能直接当作有大小含义的 feature?
2. Embedding Matrix 中什么是 parameter?
3. [B,T,C] 三个 dimensions 分别是什么?
4. Q、K、V 各自承担什么角色?
5. Causal Mask 为什么要在 Softmax 前使用?
6. Residual Connection 为什么要求 shape 相同?
7. LM Head 为什么输出 [B,T,V]?
8. Cross Entropy 在奖励什么、惩罚什么?
9. Backpropagation 与 Optimizer 有什么区别?
10. Training 与 Inference 是否会修改同一组 parameters?

理想的回答不仅是“training 会、inference 不会”，还应说明原因：training 通过 targets 形成 scalar Loss，再 backward 和 optimizer.step；inference 只执行 forward 与 sampling，使用固定参数，不建立用于 update 的 graph。

24. 整个课程最重要的一句话

Week 2 中的核心没有被 Transformer 推翻：

$$Learning = finding parameters that minimize loss$$

GPT 的 architecture 更复杂、parameters 更多、data 更大，但它仍然在重复：

```
Prediction
↓
Loss
↓
Gradient
↓
Parameter Update
↓
Better Prediction
```

现在你已经能够从代码、shape、公式和 training loop 四个角度理解一个 Mini GPT 怎样工作。

如果再向下追一层，你会发现每个复杂名字最终都落回几种基本动作：整数 lookup、矩阵乘法、加法、非线性函数、归一化、概率损失和梯度更新。理解这些动作如何组合，比记住某个模型版本的类名更能迁移到新的 architecture。

附录 A - 完整 Mini GPT 参考代码

下面代码与 Week 9-12 的 tokenizer、architecture、training、checkpoint 和 generation 主线配套。

```
"""A small, readable GPT implementation for learning from first principles.

Usage:
python mini_gpt.py train --input sample_corpus.txt --steps 1000
python mini_gpt.py generate --checkpoint mini_gpt_checkpoint.pt --prompt "Learning"
"""

from __future__ import annotations

import argparse
import math
import random
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Sequence

import torch
import torch.nn as nn
import torch.nn.functional as F

@dataclass
class GPTConfig:
    vocab_size: int
    block_size: int = 64
    n_embd: int = 128
    n_head: int = 4
    n_layer: int = 4
    dropout: float = 0.1

    def validate(self) -> None:
        if self.vocab_size <= 0:
            raise ValueError("vocab_size must be positive")
        if self.block_size <= 0:
            raise ValueError("block_size must be positive")
        if self.n_embd <= 0:
            raise ValueError("n_embd must be positive")
        if self.n_head <= 0:
            raise ValueError("n_head must be positive")
        if self.n_layer <= 0:
            raise ValueError("n_layer must be positive")
        if self.n_embd % self.n_head != 0:
            raise ValueError("n_embd must be divisible by n_head")
        if not 0.0 <= self.dropout < 1.0:
            raise ValueError("dropout must be in [0, 1)")

class CharacterTokenizer:
    def __init__(self, chars: Sequence[str]) -> None:
        unique_chars = sorted(set(chars))
        if not unique_chars:
            raise ValueError("Tokenizer vocabulary cannot be empty")

        self.chars = unique_chars
        self.stoi = {char: index for index, char in enumerate(self.chars)}
        self.itos = {index: char for char, index in self.stoi.items()}

    @classmethod
    def from_text(cls, text: str) -> "CharacterTokenizer":
```

```

    return cls(text)

@property
def vocab_size(self) -> int:
    return len(self.chars)

def encode(self, text: str) -> list[int]:
    unknown = sorted({char for char in text if char not in self.stoi})
    if unknown:
        rendered = ", ".join(repr(char) for char in unknown)
        raise ValueError(f"Prompt contains unknown characters: {rendered}")
    return [self.stoi[char] for char in text]

def decode(self, token_ids: Sequence[int]) -> str:
    return "".join(self.itos[int(token_id)] for token_id in token_ids)

class CausalSelfAttention(nn.Module):
    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        config.validate()

        self.n_head = config.n_head
        self.head_size = config.n_embd // config.n_head

        self.qkv = nn.Linear(config.n_embd, 3 * config.n_embd, bias=False)
        self.output_projection = nn.Linear(config.n_embd, config.n_embd)

        self.attention_dropout = nn.Dropout(config.dropout)
        self.residual_dropout = nn.Dropout(config.dropout)

        mask = torch.tril(torch.ones(config.block_size, config.block_size))
        self.register_buffer(
            "causal_mask",
            mask.view(1, 1, config.block_size, config.block_size),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        batch_size, sequence_length, channels = x.shape

        query, key, value = self.qkv(x).chunk(3, dim=-1)

        query = query.view(
            batch_size,
            sequence_length,
            self.n_head,
            self.head_size,
        ).transpose(1, 2)

        key = key.view(
            batch_size,
            sequence_length,
            self.n_head,
            self.head_size,
        ).transpose(1, 2)

        value = value.view(
            batch_size,
            sequence_length,
            self.n_head,
            self.head_size,
        ).transpose(1, 2)

        scores = query @ key.transpose(-2, -1)
        scores = scores * (self.head_size ** -0.5)

```

```

scores = scores.masked_fill(
    self.causal_mask[:, :, :sequence_length, :sequence_length] == 0,
    float("-inf"),
)

weights = F.softmax(scores, dim=-1)
weights = self.attention_dropout(weights)

output = weights @ value
output = output.transpose(1, 2).contiguous()
output = output.view(batch_size, sequence_length, channels)
output = self.output_projection(output)

return self.residual_dropout(output)

```

```

class FeedForward(nn.Module):
    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(config.n_embd, 4 * config.n_embd),
            nn.GELU(),
            nn.Linear(4 * config.n_embd, config.n_embd),
            nn.Dropout(config.dropout),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)

```

```

class TransformerBlock(nn.Module):
    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        self.ln1 = nn.LayerNorm(config.n_embd)
        self.attention = CausalSelfAttention(config)
        self.ln2 = nn.LayerNorm(config.n_embd)
        self.feed_forward = FeedForward(config)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.attention(self.ln1(x))
        x = x + self.feed_forward(self.ln2(x))
        return x

```

```

class MiniGPT(nn.Module):
    def __init__(self, config: GPTConfig) -> None:
        super().__init__()
        config.validate()
        self.config = config

        self.token_embedding = nn.Embedding(config.vocab_size, config.n_embd)
        self.position_embedding = nn.Embedding(config.block_size, config.n_embd)
        self.embedding_dropout = nn.Dropout(config.dropout)

        self.blocks = nn.ModuleList(
            [TransformerBlock(config) for _ in range(config.n_layer)]
        )

        self.final_norm = nn.LayerNorm(config.n_embd)
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)

        self.apply(self._init_weights)
        self.lm_head.weight = self.token_embedding.weight

```

```
@staticmethod
```

```

def _init_weights(module: nn.Module) -> None:
    if isinstance(module, nn.Linear):
        nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        nn.init.normal_(module.weight, mean=0.0, std=0.02)

def forward(
    self,
    idx: torch.Tensor,
    targets: torch.Tensor | None = None,
) -> tuple[torch.Tensor, torch.Tensor | None]:
    batch_size, sequence_length = idx.shape

    if sequence_length > self.config.block_size:
        raise ValueError(
            f"Sequence length {sequence_length} exceeds "
            f"block_size {self.config.block_size}"
        )

    positions = torch.arange(sequence_length, device=idx.device)
    x = self.token_embedding(idx) + self.position_embedding(positions)
    x = self.embedding_dropout(x)

    for block in self.blocks:
        x = block(x)

    x = self.final_norm(x)
    logits = self.lm_head(x)

    loss = None
    if targets is not None:
        vocab_size = logits.shape[-1]
        loss = F.cross_entropy(
            logits.reshape(batch_size * sequence_length, vocab_size),
            targets.reshape(batch_size * sequence_length),
        )

    return logits, loss

@torch.no_grad()
def generate(
    self,
    idx: torch.Tensor,
    max_new_tokens: int,
    temperature: float = 1.0,
    top_k: int | None = None,
) -> torch.Tensor:
    if temperature <= 0:
        raise ValueError("temperature must be greater than zero")
    if max_new_tokens < 0:
        raise ValueError("max_new_tokens cannot be negative")

    was_training = self.training
    self.eval()

    for _ in range(max_new_tokens):
        context = idx[:, -self.config.block_size:]
        logits, _ = self(context)
        next_logits = logits[:, -1, :] / temperature

        if top_k is not None:
            k = max(1, min(top_k, next_logits.shape[-1]))
            top_values, _ = torch.topk(next_logits, k)

```

```

        threshold = top_values[:, [-1]]
        next_logits = next_logits.masked_fill(
            next_logits < threshold,
            float("-inf"),
        )

        probabilities = F.softmax(next_logits, dim=-1)
        next_token = torch.multinomial(probabilities, num_samples=1)
        idx = torch.cat((idx, next_token), dim=1)

    if was_training:
        self.train()

    return idx

def choose_device(requested: str) -> torch.device:
    if requested != "auto":
        return torch.device(requested)
    if torch.cuda.is_available():
        return torch.device("cuda")
    if hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")

def set_seed(seed: int) -> None:
    random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def get_batch(
    source: torch.Tensor,
    batch_size: int,
    block_size: int,
    device: torch.device,
) -> tuple[torch.Tensor, torch.Tensor]:
    if len(source) <= block_size:
        raise ValueError(
            "Dataset split must contain more tokens than block_size. "
            "Use a larger corpus or a smaller block_size."
        )

    starts = torch.randint(
        low=0,
        high=len(source) - block_size,
        size=(batch_size,),
    )

    x = torch.stack([source[i : i + block_size] for i in starts])
    y = torch.stack([source[i + 1 : i + block_size + 1] for i in starts])

    return x.to(device), y.to(device)

@torch.no_grad()
def estimate_losses(
    model: MiniGPT,
    train_data: torch.Tensor,
    val_data: torch.Tensor,
    eval_steps: int,
    batch_size: int,
    device: torch.device,

```

```

) -> dict[str, float]:
    was_training = model.training
    model.eval()
    result: dict[str, float] = {}

    for name, source in (("train", train_data), ("val", val_data)):
        losses = []
        for _ in range(eval_steps):
            x, y = get_batch(
                source,
                batch_size,
                model.config.block_size,
                device,
            )
            _, loss = model(x, y)
            if loss is None:
                raise RuntimeError("Expected a loss during evaluation")
            losses.append(loss.item())

        result[name] = sum(losses) / len(losses)

    if was_training:
        model.train()

    return result

def save_checkpoint(
    output_path: Path,
    model: MiniGPT,
    optimizer: torch.optim.Optimizer,
    tokenizer: CharacterTokenizer,
    step: int,
    losses: dict[str, float],
) -> None:
    checkpoint = {
        "model_state": model.state_dict(),
        "optimizer_state": optimizer.state_dict(),
        "config": asdict(model.config),
        "tokenizer_chars": tokenizer.chars,
        "step": step,
        "losses": losses,
    }
    torch.save(checkpoint, output_path)

def load_checkpoint(
    checkpoint_path: Path,
    device: torch.device,
) -> tuple[MiniGPT, CharacterTokenizer, dict]:
    try:
        checkpoint = torch.load(
            checkpoint_path,
            map_location=device,
            weights_only=False,
        )
    except TypeError:
        checkpoint = torch.load(checkpoint_path, map_location=device)

    config = GPTConfig(**checkpoint["config"])
    tokenizer = CharacterTokenizer(checkpoint["tokenizer_chars"])

    if tokenizer.vocab_size != config.vocab_size:
        raise ValueError("Checkpoint tokenizer and model vocabulary do not match")

```

```

model = MiniGPT(config).to(device)
model.load_state_dict(checkpoint["model_state"])

return model, tokenizer, checkpoint

def count_parameters(model: nn.Module) -> int:
    return sum(parameter.numel() for parameter in model.parameters())

def train(args: argparse.Namespace) -> None:
    set_seed(args.seed)
    device = choose_device(args.device)

    text = args.input.read_text(encoding="utf-8")
    if not text:
        raise ValueError("Training file is empty")

    tokenizer = CharacterTokenizer.from_text(text)
    encoded = torch.tensor(tokenizer.encode(text), dtype=torch.long)

    split_index = int(len(encoded) * args.train_split)
    train_data = encoded[:split_index]
    val_data = encoded[split_index:]

    minimum_split_size = args.block_size + 1
    if len(train_data) < minimum_split_size or len(val_data) < minimum_split_size:
        raise ValueError(
            "Both train and validation splits must be larger than block_size. "
            "Use a larger corpus, smaller block_size, or adjusted train_split."
        )

    config = GPTConfig(
        vocab_size=tokenizer.vocab_size,
        block_size=args.block_size,
        n_embd=args.n_embd,
        n_head=args.n_head,
        n_layer=args.n_layer,
        dropout=args.dropout,
    )

    model = MiniGPT(config).to(device)
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=args.learning_rate,
        weight_decay=args.weight_decay,
    )

    initial_expected_loss = math.log(tokenizer.vocab_size)
    print(f"device: {device}")
    print(f"tokens: {len(encoded):,}")
    print(f"vocabulary: {tokenizer.vocab_size}")
    print(f"parameters: {count_parameters(model):,}")
    print(f"uniform baseline loss: {initial_expected_loss:.4f}")

    last_losses = {"train": float("nan"), "val": float("nan")}
    model.train()

    for step in range(args.steps + 1):
        if step % args.eval_interval == 0 or step == args.steps:
            last_losses = estimate_losses(
                model=model,
                train_data=train_data,
                val_data=val_data,
                eval_steps=args.eval_steps,

```

```

        batch_size=args.batch_size,
        device=device,
    )
    print(
        f"step {step:6d} | "
        f"train {last_losses['train']:.4f} | "
        f"val {last_losses['val']:.4f}"
    )

    if step == args.steps:
        break

    x, y = get_batch(
        train_data,
        args.batch_size,
        config.block_size,
        device,
    )

    _, loss = model(x, y)
    if loss is None:
        raise RuntimeError("Expected a loss during training")

    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
    optimizer.step()

    save_checkpoint(
        output_path=args.output,
        model=model,
        optimizer=optimizer,
        tokenizer=tokenizer,
        step=args.steps,
        losses=last_losses,
    )
    print(f"checkpoint: {args.output}")

    seed_character = text[0]
    context = torch.tensor(
        [tokenizer.encode(seed_character)],
        dtype=torch.long,
        device=device,
    )
    generated = model.generate(
        context,
        max_new_tokens=args.sample_tokens,
        temperature=args.temperature,
        top_k=args.top_k,
    )
    print("\n--- sample ---")
    print(tokenizer.decode(generated[0].tolist()))

def generate(args: argparse.Namespace) -> None:
    set_seed(args.seed)
    device = choose_device(args.device)

    model, tokenizer, checkpoint = load_checkpoint(args.checkpoint, device)
    model.eval()

    if not args.prompt:
        raise ValueError("Prompt must contain at least one character")

    prompt_ids = tokenizer.encode(args.prompt)

```



```

context = torch.tensor(
    [prompt_ids],
    dtype=torch.long,
    device=device,
)

generated = model.generate(
    context,
    max_new_tokens=args.max_new_tokens,
    temperature=args.temperature,
    top_k=args.top_k,
)

print(f"checkpoint step: {checkpoint.get('step', 'unknown')}")
print(tokenizer.decode(generated[0].tolist()))

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    subparsers = parser.add_subparsers(dest="command", required=True)

    train_parser = subparsers.add_parser("train", help="Train a character Mini GPT")
    train_parser.add_argument("--input", type=Path, required=True)
    train_parser.add_argument(
        "--output",
        type=Path,
        default=Path("mini_gpt_checkpoint.pt"),
    )
    train_parser.add_argument("--steps", type=int, default=2000)
    train_parser.add_argument("--batch-size", type=int, default=32)
    train_parser.add_argument("--block-size", type=int, default=64)
    train_parser.add_argument("--n-embd", type=int, default=128)
    train_parser.add_argument("--n-head", type=int, default=4)
    train_parser.add_argument("--n-layer", type=int, default=4)
    train_parser.add_argument("--dropout", type=float, default=0.1)
    train_parser.add_argument("--learning-rate", type=float, default=3e-4)
    train_parser.add_argument("--weight-decay", type=float, default=0.01)
    train_parser.add_argument("--grad-clip", type=float, default=1.0)
    train_parser.add_argument("--train-split", type=float, default=0.9)
    train_parser.add_argument("--eval-interval", type=int, default=100)
    train_parser.add_argument("--eval-steps", type=int, default=20)
    train_parser.add_argument("--sample-tokens", type=int, default=300)
    train_parser.add_argument("--temperature", type=float, default=0.8)
    train_parser.add_argument("--top-k", type=int, default=20)
    train_parser.add_argument("--seed", type=int, default=42)
    train_parser.add_argument("--device", default="auto")
    train_parser.set_defaults(func=train)

    generate_parser = subparsers.add_parser(
        "generate",
        help="Generate text from a saved checkpoint",
    )
    generate_parser.add_argument("--checkpoint", type=Path, required=True)
    generate_parser.add_argument("--prompt", required=True)
    generate_parser.add_argument("--max-new-tokens", type=int, default=300)
    generate_parser.add_argument("--temperature", type=float, default=0.8)
    generate_parser.add_argument("--top-k", type=int, default=20)
    generate_parser.add_argument("--seed", type=int, default=42)
    generate_parser.add_argument("--device", default="auto")
    generate_parser.set_defaults(func=generate)

    return parser

def main() -> None:

```

```
parser = build_parser()
args = parser.parse_args()
args.func(args)

if __name__ == "__main__":
    main()
```